Java Programming

Course Code: IT201

B. Tech. (IT/CSE)

Module-1

Books & References

1. Java : The Complete Reference

– Patrick Naughton, Herbert Schildt

2. Thinking in Java (http://www.mindview.net/Books/TIJ/)

– Bruce Eckel

3. Richard G Baldwin's "Introductory Java Programming Tutorial" on: http://www.dickbaldwin.com/tocint.htm

BOOKS

1. JAVA The Complete Reference by PATRICK NAUGHTON & HERBERT SCHILD, TMH.
2. Introduction to JAVA Programming a primar, Balaguruswamy.
3. K. Arnold and J. Gosling, "The JAVA programming language", Pearson Education.
4. Timothy Budd, "Understanding Object-oriented programming with Java", Pearson Education

OBJECTIVES

After completing this section, you will be able to
▪ Understand the OOP concepts and difference in C++ and Java languages
▪ Understand data types, variables, operators and their use
▪ Differentiate among - JDK, - $JRE\ and$ - JVM
▪ Understand the steps for creating, compiling and executing a Java program

Introduction

- Java is a high-level programming language originally developed by Sun Microsystems and released in 1995.
- Java runs on a variety of platforms, such as Windows, Mac OS, and the various versions of UNIX.
- Java is a programming language and a platform.
- James Gosling is known as the father of Java.
- Before Java, its name was Oak.
- Since Oak was already a registered company, so James Gosling and his team changed the Oak name to Java.

Platform?

Any hardware or software environment in which a program runs, is known as a platform.

Since Java has a runtime environment (JRE) and API, it is called a platform.

Java: Applications

According to Sun, 3 billion devices run Java.

Desktop Applications such as acrobat reader, media player, antivirus, etc.

Web Applications such as irctc.co.in etc.

Enterprise Applications such as banking applications.

- Mobile
- Embedded System
- Smart Card
- Robotics
- Games, etc.

Why Java?

Object Oriented − In Java, everything is an Object. Java can be easily extended since it is based on the Object model.

Platform Independent − Unlike many other programming languages including C and C++, when Java is compiled, it is not compiled into platform specific machine, rather into platform independent byte code. This byte code is distributed over the web and interpreted by the Virtual Machine (JVM) on whichever platform it is being run on.

Simple − Java is designed to be easy to learn. If you understand the basic concept of OOP Java, it would be easy to master.

Secure − With Java's secure feature it enables to develop virus-free, tamper-free systems. Authentication techniques are based on public-key encryption.

Architecture-neutral − Java compiler generates an architecture-neutral object file format, which makes the compiled code executable on many processors, with the presence of Java runtime system.

Portable − Being architecture-neutral and having no implementation dependent aspects of the specification makes Java portable. Compiler in Java is written in ANSI C with a clean portability boundary, which is a POSIX subset.

Robust − Java makes an effort to eliminate error prone situations by emphasizing mainly on compile time error checking and runtime checking.

Multithreaded − With Java's multithreaded feature it is possible to write programs that can perform many tasks simultaneously. This design feature allows the developers to construct interactive applications that can run smoothly.

Interpreted − Java byte code is translated on the fly to native machine instructions and is not stored anywhere. The development process is more rapid and analytical since the linking is an incremental and light-weight process.

High Performance − With the use of Just-In-Time compilers, Java enables high performance.

Distributed − Java is designed for the distributed environment of the internet.

Dynamic − Java is considered to be more dynamic than C or C++ since it is designed to adapt to an evolving environment. Java programs can carry extensive amount of run-time information that can be used to verify and resolve accesses to objects on run-time.

Java SE Vs. Java EE Vs. Java ME

- Java SE – Standard Edition
  - Java Platform, Standard Edition (Java SE) is **a computing platform for development of portable code for desktop and server environments**. Java SE was formerly known as Java 2 Platform, Standard Edition (J2SE). The platform uses Java programming language and is part of the Java software-platform family.
- Java EE – Enterprise Edition
  - Jakarta EE, formerly Java Platform, Enterprise Edition and Java 2 Platform, Enterprise Edition, is a set of specifications, extending Java SE with specifications for enterprise features such as distributed computing and web services.
- Java ME - Micro Edition
  - Java Platform, Micro Edition or Java ME is a computing platform for development and deployment of portable code for embedded and mobile devices.

Java SE, Java EE, Java ME



Types of Java Applications

There are mainly 4 types of applications that can be created using Java programming:

1) Standalone Application- Standalone applications are also known as desktop applications or window-based applications. These are traditional software that we need to install on every machine. Examples of standalone application are Media player, antivirus, etc. AWT and Swing are used in Java for creating standalone applications.

2) Web Application- An application that runs on the server side and creates a dynamic page is called a web application. Currently, Servlet, JSP, Struts, Spring, Hibernate, JSF, etc. technologies are used for creating web applications in Java.

3) Enterprise Application- An application that is distributed in nature, such as banking applications, etc. is called enterprise application. It has advantages of the high-level security, load balancing, and clustering. In Java, EJB is used for creating enterprise applications.

4) Mobile Application- An application which is created for mobile devices is called a mobile application. Currently, Android and Java ME are used for creating mobile applications.

Program 1:

```
public class MyFirstJavaProgram {

    /* This is my first java program.
    * This will print 'Hello World' as the output
    */

    public static void main(String []args) {
    System.out.println("Hello World"); // prints Hello World
    }
}
```

Steps to create a Java Program

For executing any java program:

S1: Install the JDK
S2: Set path of the jdk/bin directory.

If you are saving the Java source file inside the JDK/bin directory, the path is not required to be set because all the tools will be available in the current directory.

Way 1: set path=C:\Program Files\Java\jdk1.6.0_23\bin

Way 2: Go to MyComputer properties -> advanced tab -> environment variables -> new tab of user variable -> write path in variable name -> write path of bin folder in variable value -> ok -> ok -> ok

S3: Create the java program Using Notepad editor or some other editor.
S4: Save this file as Simple.java
S5: Compile and run the java program.

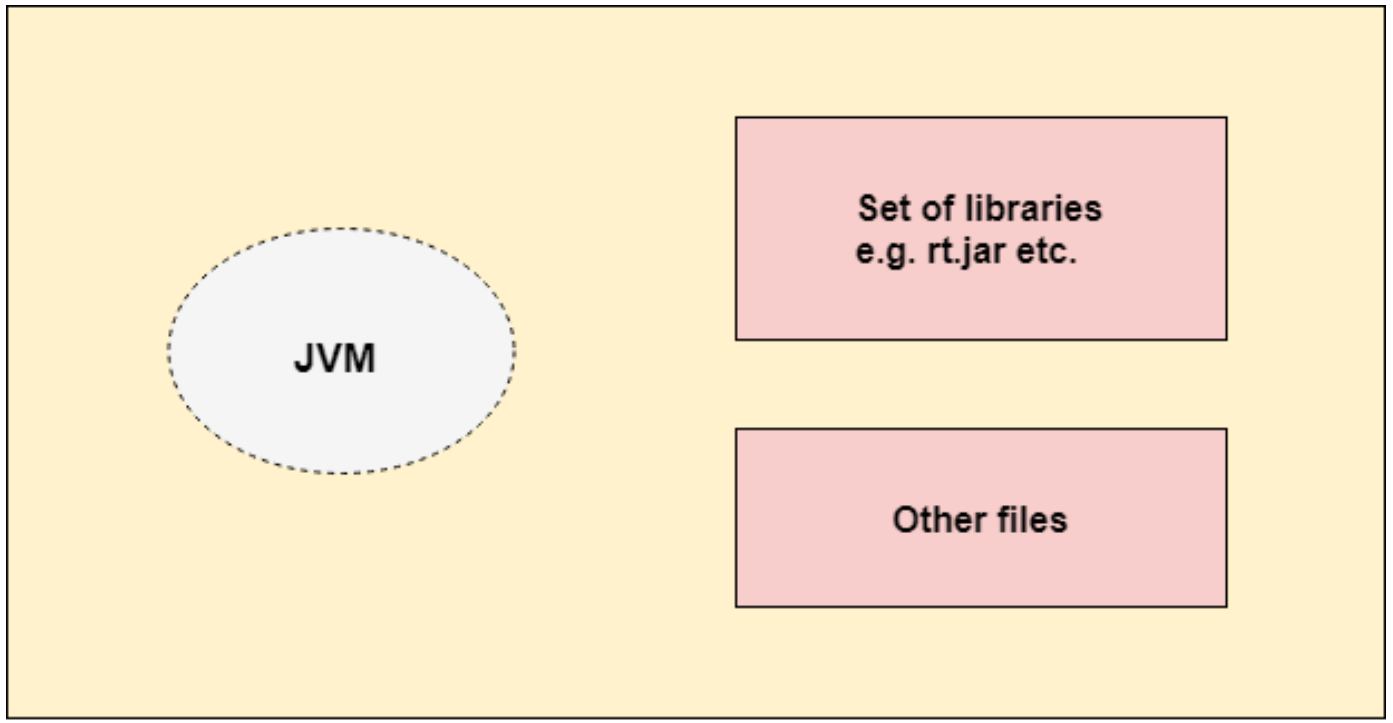
To compile:	javac Simple.java
To execute:	java Simple
Output:	Hello World

Difference among JDK, JRE, and JVM

JVM

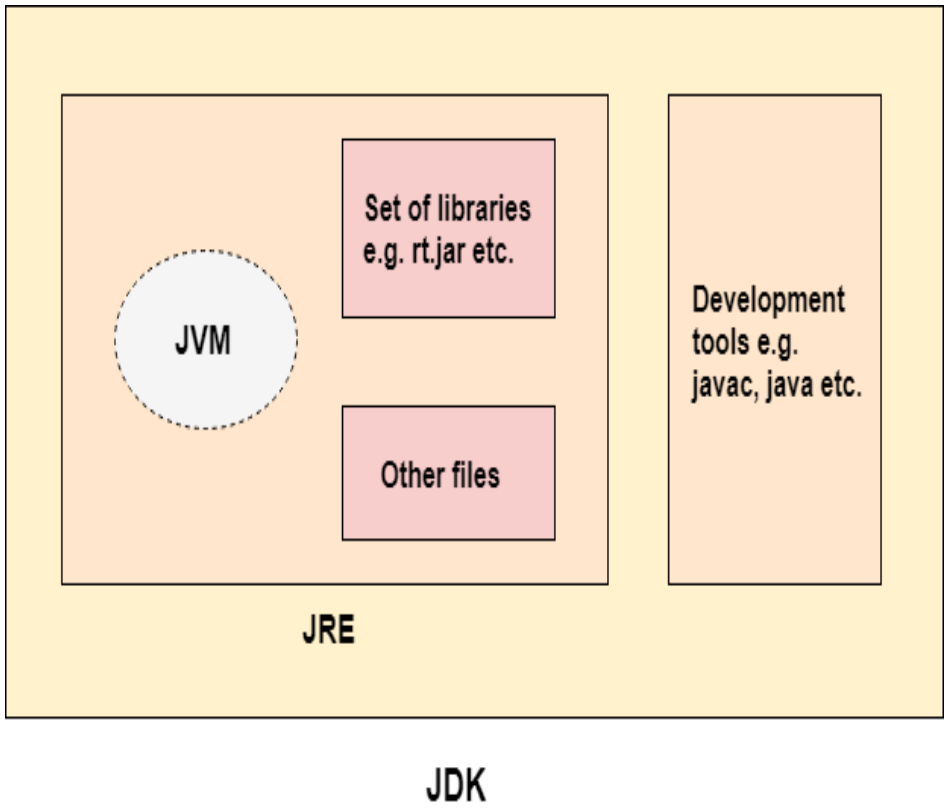
- JVM (Java Virtual Machine) is an abstract machine.
- It is called a virtual machine because it doesn't physically exist.
- It is a specification that provides a runtime environment in which Java bytecode can be executed.
- It can also run those programs which are written in other languages and compiled to Java bytecode.
- JVMs are available for many hardware and software platforms. JVM, JRE, and JDK are platform dependent because the configuration of each OS is different from each other. However, Java is platform independent. There are three notions of the JVM: *specification*, *implementation*, and *instance*.

JRE



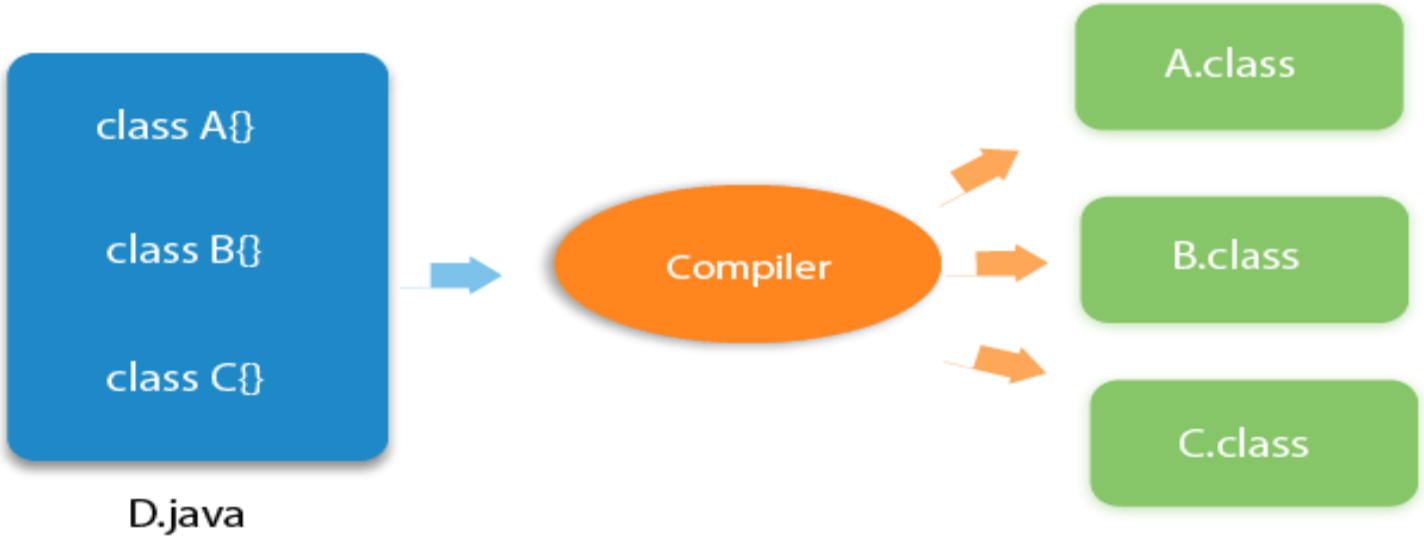
JRE

JDK


Set of libraries
e.g. rt.jar etc.
JVM
Other files
Development
tools e.g.
javac, java etc.
JRE
JDK

Can you have multiple classes in a java source file?

Yes



Java Variables

A variable is a container which holds the value while the Java program is executed. A variable is assigned with a data type.

Variable is a name of memory location.

There are three types of variables in java: local, instance and static.

There are two types of data types in Java: primitive and non-primitive.

1) Local Variable

A variable declared inside the body of the method is called local variable. You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

A local variable cannot be defined with "static" keyword.

2) Instance Variable

A variable declared inside the class but outside the body of the method, is called instance variable. It is not declared as static.

It is called instance variable because its value is instance specific and is not shared among instances.

3) Static variable

A variable which is declared as static is called static variable. It cannot be local. You can create a single copy of static variable and share among all the instances of the class. Memory allocation for static variable happens only once when the class is loaded in the memory.

Example:

```
class A{
int data=50;//instance variable
static int m=100;//static variable
void method(){
int n=90;//local variable
}
}//end of class
```

Data Types in Java

Data types specify the different sizes and values that can be stored in the variable.

There are two types of data types in Java:

Primitive data types: The primitive data types include boolean, char, byte, short, int, long, float and double.

Non-primitive data types: The non-primitive data types include Classes, Interfaces, and Arrays.

Primitive Data Types- These are the most basic data types available in Java language.

Data Type	Default Value	Default size
boolean	false	1 bit
char	'\u0000'	2 byte
byte	0	1 byte
short	0	2 byte
int	0	4 byte
long	0L	8 byte
float	0.0f	4 byte
double	0.0d	8 byte

Operators in Java

Operator in Java is a symbol which is used to perform operations. For example: +, -, *, / etc.

There are many types of operators in Java which are given below:

Operator Type	Category	Precedence
Unary	postfix	*expr*++ *expr*--
	prefix	++*expr* --*expr* +*expr* -*expr* ~ !
Arithmetic	multiplicative	* / %
	additive	+ -
Shift	shift	<< >> >>>
Relational	comparison	< > <= >= instanceof
	equality	== !=
Bitwise	bitwise AND	&
	bitwise exclusive OR	^
	bitwise inclusive OR	|
Logical	logical AND	&&
	logical OR	||
Ternary	ternary	? :
Assignment	assignment	= += -= *= /= %= &= ^= |= <<= >>= >>>=

Some Assignments ☺

1. Write a program which prints the following information about at least 5 persons:

   NAME MAIL-ID EMPLOYEE-CODE PHONE

   Eg. Umesh umesh@cse p03161 25764728

   Salil salil@cse p03160 25764728

   Each entry should be on a separate line.

2. Write a program that prints the following line on the screen along with quotes.

   "Can we print '\' with System.out.println() statement?"

End

Thank you…

REFRENCES

1. Java : The Complete Reference , Patrick Naughton, Herbert Schildt

2. Thinking in Java (http://www.mindview.net/Books/TIJ/) , Bruce Eckel

3. https://www.javatpoint.com/java-tutorial

4. https://docs.oracle.com/javase/tutorial/

Java Programming

Course Code: IT201

B. Tech. (IT/CSE)

Module-1

Lecture-2

OBJECTIVES

After completing this section, you will be able to
▪ Understand Operators and their use
▪ Understand the use of Modifiers and access control
▪ Understand the use of Decision control statements

Operators in Java

Operator in Java is a symbol which is used to perform operations. For example: +, -, *, / etc.

There are many types of operators in Java which are given below:

Operators in Java

Operator Type	Category	Precedence
Unary	postfix	*expr*++ *expr*--
	prefix	++*expr* --*expr* +*expr* -*expr* ~ !
Arithmetic	multiplicative	* / %
	additive	+ -
Shift	shift	<< >> >>>
Relational	comparison	< > <= >= instanceof
	equality	== !=
Bitwise	bitwise AND	&
	bitwise exclusive OR	^
	bitwise inclusive OR	|
Logical	logical AND	&&
	logical OR	||
Ternary	ternary	? :
Assignment	assignment	= += -= *= /= %= &= ^= |= <<= >>= >>>=

Operator Examples

```
class OperatorExample{
public static void main(String args[]){
int x=10;
System.out.println(x++);//10 (11)
System.out.println(++x);//12
System.out.println(x--);//12 (11)
System.out.println(--x);//10
}
}
```

```
class OperatorExample{
public static void main(String args[]){
int a=10;
int b=10;
System.out.println(a++ + ++a);//10+12=22
System.out.println(b++ + b++);//10+11=21
}
}
```

~ and !

```
class OperatorExample{
public static void main(String args[]){
int a=10;
int b=-10;
boolean c=true;
boolean d=false;
System.out.println(~a);//-11 (minus of total positive value which
starts from 0)
System.out.println(~b);//9 (positive of total minus, positive starts from 0)
System.out.println(!c);//false (opposite of boolean value)
System.out.println(!d);//true
}
}
```

<< Vs. >>

```
class OperatorExample{
public static void main(String args[]){
System.out.println(10<<2);              //10*2^2=10*4=40
System.out.println(10<<3); //10*2^3=10*8=80
System.out.println(20<<2); //20*2^2=20*4=80
System.out.println(15<<4); //15*2^4=15*16=240
}
}
```

```
class OperatorExample{
public static void main(String args[]){
System.out.println(10>>2); //10/2^2=10/4=2
System.out.println(20>>2); //20/2^2=20/4=5
System.out.println(20>>3); //20/2^3=20/8=2
}
}
```

Logical && vs Bitwise &


```
class OperatorExample{
public static void main(String args[]){
int a=10;
int b=5;
int c=20;
System.out.println(a<b&&a++<c);//false && true = false
System.out.println(a);//10 because second condition is not checked
System.out.println(a<b&a++<c);//false && true = false
System.out.println(a);//11 because second condition is checked
}
}
```

Ternary Operator

Ternary operator is used as one liner replacement for if-then-else statement and used a lot in Java programming. it is the only conditional operator which takes three operands.

```
class OperatorExample{
public static void main(String args[]){
int a=10;
int b=5;
int min=(a<b)?a:b;
System.out.println(min);
}
}
```

Modifiers

Modifiers are keywords that you add to those definitions to change their meanings.

Java language has a wide variety of modifiers, including the following :

- Java Access Modifiers
- Non Access Modifiers

To use a modifier, you include its keyword in the definition of a class, method, or variable. The modifier precedes the rest of the statement.

E.g.: public class className {

Access Control Modifiers

Java provides a number of access modifiers to set access levels for classes, variables, methods and constructors.

The four access levels are −

-Visible to the package, the default. No modifiers are needed.

-Visible to the class only (private).

-Visible to the world (public).

-Visible to the package and all subclasses (protected).

Non-Access Modifiers

- Java provides a number of non-access modifiers to achieve many other functionality.
- The *static* modifier for creating class methods and variables.
- The *final* modifier for finalizing the implementations of classes, methods, and variables.
- The *abstract* modifier for creating abstract classes and methods.
- The *synchronized* and *volatile* modifiers, which are used for threads.

Control Statements: If Statement

The *if statement* is used to test the condition. It checks boolean condition: *true* or *false*. There are various types of if statement in Java:

- if statement
- if-else statement
- if-else-if ladder
- nested if statement

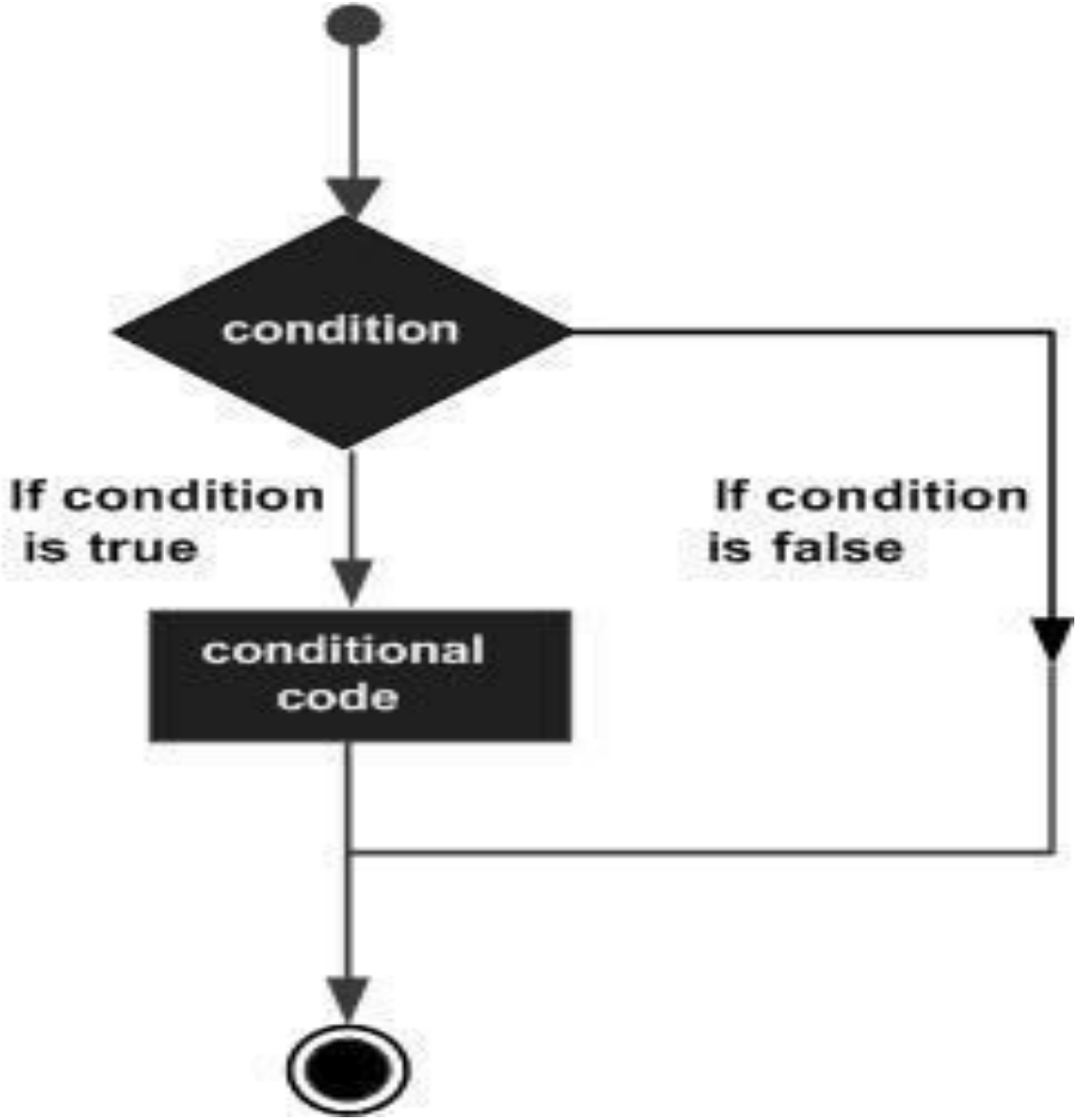
if Statement

The Java if statement tests the condition. It executes the *if block* if condition is true.

Syntax:

```
if(condition){
//code to be executed
}
```

If Statement



Example

```
//Java Program to demonstate the use of if statement.
public class IfExample {
public static void main(String[] args) {
    //defining an 'age' variable
    int age=20;
    //checking the age
    if(age>18){
        System.out.print("Age is greater than 18");
    }
}
}
```

Output: Age is greater than 18

if-else Statement

The Java if-else statement also tests the condition. It executes the *if block* if condition is true otherwise *else block* is executed.

Syntax:

```
if(condition){
//code if condition is true
}else{
//code if condition is false
}
```

Example

```
//A Java Program to demonstrate the use of if-else statement.
//It is a program of odd and even number.
public class IfElseExample {
public static void main(String[] args) {
    //defining a variable
    int number=13;
    //Check if the number is divisible by 2 or not
    if(number%2==0){
        System.out.println("even number");
    }else{
        System.out.println("odd number");
    }
}
}
```

Output: odd number

if-else-if ladder Statement

The if-else-if ladder statement executes one condition from multiple statements.

Syntax:

```
if(condition1){
//code to be executed if condition1 is true
}else if(condition2){
//code to be executed if condition2 is true
}
else if(condition3){
//code to be executed if condition3 is true
}
...
else{
//code to be executed if all the conditions are false
}
```

Nested if statement

The nested if statement represents the *if block within another if block*. Here, the inner if block condition executes only when outer if block condition is true.

Syntax:

```
if(condition){
    //code to be executed
        if(condition){
            //code to be executed
    }
}
```

Example

```
//Java Program to demonstrate the use of Nested If Statement.
public class JavaNestedIfExample2 {
public static void main(String[] args) {
    //Creating two variables for age and weight
    int age=25;
    int weight=48;
    //applying condition on age and weight
    if(age>=18){
        if(weight>50){
            System.out.println("You are eligible to donate blood");
        } else{
            System.out.println("You are not eligible to donate blood");
        }
    } else{
      System.out.println("Age must be greater than 18");
    }
} }
```

Output: You are not eligible to donate blood

Switch Statement

- The Java *switch statement* executes one statement from multiple conditions.
- It is like if-else-if ladder statement.
- The switch statement works with byte, short, int, long, enum types, String and some wrapper types like Byte, Short, Int, and Long.
- Since Java 7, you can use strings in the switch statement.
- There can be *one or N number of case values* for a switch expression.

Switch Statement

- The case value must be of switch expression type only. The case value must be *literal or constant*. It doesn't allow variables.
- The case values must be *unique*. In case of duplicate value, it renders compile-time error.
- The Java switch expression must be of *byte, short, int, long (with its Wrapper type), enums and string*.
- Each case statement can have a *break statement* which is optional. When control reaches to the break statement, it jumps the control after the switch expression. If a break statement is not found, it executes the next case.
- The case value can have a *default label* which is optional.

Example

```
public class SwitchExample {
public static void main(String[] args) {
    //Declaring a variable for switch expression
    int number=20;
    //Switch expression
    switch(number){
    //Case statements
    case 10: System.out.println("10");
    break;
    case 20: System.out.println("20");
    break;
    case 30: System.out.println("30");
    break;
    //Default case statement
    default:System.out.println("Not in 10, 20 or 30");
    }
}
}
```

Output: 20

Switch Statement

Java allows us to use strings in switch expression since Java SE 7. The case statement should be string literal.

```
//Java Program to demonstrate the use of Java Switch
//statement with String
public class SwitchStringExample {
public static void main(String[] args) {
    //Declaring String variable
    String levelString="Expert";
    int level=0;
```

Switch Statement


```
 //Using String in Switch expression
    switch(levelString){
    //Using String Literal in Switch case
    case "Beginner": level=1;
    break;
    case "Intermediate": level=2;
    break;
    case "Expert": level=3;
    break;
    default: level=0;
    break;
    }
    System.out.println("Your Level is: "+level);
}
```

Output: Your Level is: 3

Use of Enum


```
//Java Program to demonstrate the use of Enum
//in switch statement


public class JavaSwitchEnumExample {
      public enum Day {  Sun, Mon, Tue, Wed, Thu, Fri, Sat  }
      public static void main(String args[])
      {
        Day[] DayNow = Day.values();
          for (Day Now : DayNow)
          {
              switch (Now)
              {
                  case Sun:
                      System.out.println("Sunday");
                      break;
                  case Mon:
                      System.out.println("Monday");
                      break;
                  case Tue:
                      System.out.println("Tuesday");
                      break;
```

Use of Enum


```
        case Wed:
                        System.out.println("Wednesday");
                        break;
                    case Thu:
                        System.out.println("Thursday");
                        break;
                    case Fri:
                        System.out.println("Friday");
                        break;
                    case Sat:
                        System.out.println("Saturday");
                        break;
                }
            }
        }
}
```

Some Assignments ☺

int a_number=1; // (range: 1 to 5 including both)
Print the value of *a_number* in word. For example, it should print “Four” if *a_number* contains 4.

1. Use equality ‘= =’ operator.
2. Do not use equality ‘= =’ operator.

3. WAP to check whether a year entered by user is leap year or not. (Hint: A year is leap, if it is divisible by 4 and 400. But, not by 100.)

4. WAP to create a Grading System for Students. The program will print the grade according to the marks of student and grading system should include following grades fail, D grade, C grade, B grade, A grade and A+. (Hint: use if-else ladder)

5. WAP to check whether a number is POSITIVE, NEGATIVE or ZERO.

End

Thank you…

REFRENCES

1. Java : The Complete Reference , Patrick Naughton, Herbert Schildt

2. Thinking in Java (http://www.mindview.net/Books/TIJ/) , Bruce Eckel

3. https://www.javatpoint.com/java-tutorial

4. https://docs.oracle.com/javase/tutorial/

Java Programming

Course Code: IT201

B. Tech. (IT/CSE)

Module-1

Lecture-3

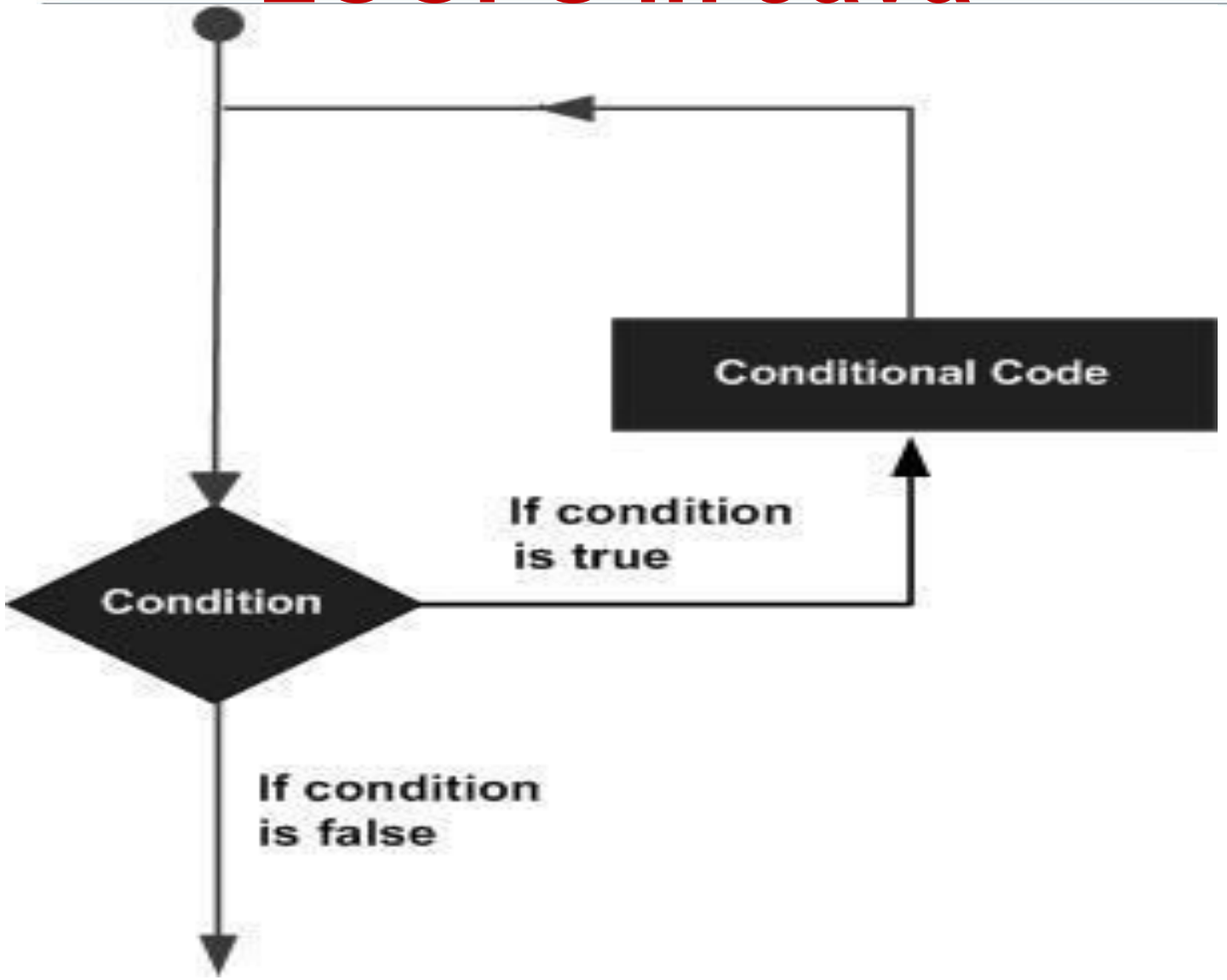
OBJECTIVES

After completing this section, you will be able to
▪ Understand the use of Loops
▪ Understand the use of break and continue statements
▪ Understand the use of Comments

LOOPS in Java

- Sometimes we need to execute a block of code several number of times.
- A **loop** statement allows us to execute a statement or group of statements multiple times.
- Following is the general form of a loop statement in most of the programming languages −

LOOPS in Java



LOOPS in Java

- There are three types of loops in Java.
  - for loop
  - while loop
  - do-while loop

1.) For Loop:

- The Java *for loop* is used to iterate a part of the program several times.
- If the number of iteration is fixed, it is recommended to use for loop.

LOOPS in Java

- Simple For Loop
- For-each or Enhanced For Loop
- Labeled For Loop

- A simple for loop is the same as C/C++. We can initialize the variable, check condition and increment/decrement value.
- It consists of four parts:
- **Initialization**: It is the initial condition which is executed once when the loop starts. Here, we can initialize the variable, or we can use an already initialized variable. It is an optional condition.
- **Condition**: It is the second condition which is executed each time to test the condition of the loop. It continues execution until the condition is false. It must return boolean value either true or false. It is an optional condition.
- **Statement**: The statement of the loop is executed each time until the second condition is false.
- **Increment/Decrement**: It increments or decrements the variable value. It is an optional condition.

LOOPS in Java

Syntax:

```
for(initialization;condition;incr/decr)
{
//statement or code to be executed
}
```

Nested For Loop

- If we have a for loop inside another loop, it is known as nested for loop.
- The inner loop executes completely whenever outer loop executes.

Example

```
public class NestedForExample {
public static void main(String[] args) {
//loop of i
for(int i=1;i<=3;i++){
//loop of j
for(int j=1;j<=3;j++){
        System.out.println(i+" "+j);
}//end of i
}//end of j
}
}
```

Output:

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

Example

```
public class PyramidExample {
public static void main(String[] args) {
for(int i=1;i<=5;i++){
for(int j=1;j<=i;j++){
        System.out.print("* ");
}
System.out.println();//new line
}
}
}
```

Output:

```
*
* *
* * *
* * * *
* * * * *
```

for-each Loop

- The for-each loop is used to traverse array or collection in java. It is easier to use than simple for loop because we don't need to increment value and use subscript notation.
- It works on elements basis not index. It returns element one by one in the defined variable.

Syntax:

```
for(Type var:array){
//code to be executed
}
```

Example

```
public class ForEachExample {
public static void main(String[] args) {
    //Declaring an array
    int arr[]={12,23,44,56,78};
    //Printing array using for-each loop
    for(int i:arr){
        System.out.println(i);
    }
}
}
```

Output:

```
12
23
44
56
78
```

Labeled For Loop

- We can have a name of each Java for loop. To do so, we use label before the for loop. It is useful if we have nested for loop so that we can break/continue specific for loop.
- Usually, break and continue keywords breaks/continues the innermost for loop only.

Syntax:

```
labelname:
for(initialization;condition;incr/decr){
//code to be executed
}
```

Example


```
public class LabeledForExample {
public static void main(String[] args) {
    //Using Label for outer and for loop
    aa:
        for(int i=1;i<=3;i++){
            bb:
                for(int j=1;j<=3;j++){
                    if(i==2&&j==2){
                        break aa;
                    }
                    System.out.println(i+" "+j);
                }
        }
}
}
```

Output:

```
1 1
1 2
1 3
2 1
```

While Loop

- The Java *while loop* is used to iterate a part of the program several times. If the number of iteration is not fixed, it is recommended to use while loop.

Syntax:

```
while(condition){
//code to be executed
}
```

do-while Loop

- The Java *do-while loop* is used to iterate a part of the program several times. If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use do-while loop.
- The Java *do-while loop* is executed at least once because condition is checked after loop body.

Syntax:

```
do{
//code to be executed
}while(condition);
```

Break Statement

- When a break statement is encountered inside a loop, the loop is immediately terminated and the program control resumes at the next statement following the loop.
- The Java *break* statement is used to break loop or switch statement. It breaks the current flow of the program at specified condition. In case of inner loop, it breaks only inner loop.
- We can use Java break statement in all types of loops such as for loop, while loop and do-while loop.

Syntax:

jump-statement;

break;

Continue Statement

- The continue statement is used in loop control structure when you need to jump to the next iteration of the loop immediately. It can be used with for loop or while loop.
- The Java *continue statement* is used to continue the loop. It continues the current flow of the program and skips the remaining code at the specified condition. In case of an inner loop, it continues the inner loop only.
- We can use Java continue statement in all types of loops such as for loop, while loop and do-while loop.

Syntax:

jump-statement;

continue;

```
public class ContinueExample {
public static void main(String[] args) {
    //for loop
    for(int i=1;i<=10;i++){
        if(i==5){
            //using continue statement
            continue;//it will skip the rest statement
        }
        System.out.println(i);
    }
}
}
```

Output:

```
1
2
3
4
6
7
8
9
10
```

Comments

- The Java comments are the statements that are not executed by the compiler and interpreter. The comments can be used to provide information or explanation about the variable, method, class or any statement. It can also be used to hide program code.

- Types of Java Comments:

There are three types of comments in Kava.

  – Single Line Comment
  – Multi Line Comment
  – Documentation Comment

Comments

- The single line comment is used to comment only one line.

  Syntax: //This is single line comment

- The multi line comment is used to comment multiple lines of code.

Syntax: /*
This
is
multi line
comment
*/

Comments

- The documentation comment is used to create documentation API. To create documentation API, you need to use **javadoc tool**.
- This type of comments are used generally when writing code for a project/software package, since it helps to generate a documentation page for reference, which can be used for getting information about methods present, its parameters, etc.

- For example http://docs.oracle.com/javase/7/docs/api/java/util/Scanner.html is an auto generated documentation page which is generated by using documentation comments and a javadoc tool for processing the comments.

Syntax:

```
/**
This
is
documentation
comment
*/
```

Example

```
/** The Calculator class provides methods to get addition and subtraction of given 2 numbers.*/
public class Calculator {
/** The add() method returns addition of given numbers.*/
public static int add(int a, int b){return a+b;}
/** The sub() method returns subtraction of given numbers.*/
public static int sub(int a, int b){return a-b;}
}
```

Compile it by javac tool:

```
javac Calculator.java
```

Create Documentation API by javadoc tool:

```
javadoc Calculator.java
```

Now, there will be HTML files created for your Calculator class in the current directory. Open the HTML files and see the explanation of Calculator class provided through documentation comment.

Command Line Arguments

- When command line arguments are supplied to JVM, JVM wraps these and supply to args[].
- It can be confirmed that they are actually wrapped up in args array by checking the length of args using args.length.
- A Java application can accept any number of arguments from the command line. This allows the user to specify configuration information when the application is launched.
- The user enters command-line arguments when invoking the application and specifies them after the name of the class to be run. For example, suppose a Java application called Sort sorts lines in a file.

Command Line Arguments

- To sort the data in a file named friends.txt, a user would enter:

  java Sort friends.txt

- When an application is launched, the runtime system passes the command-line arguments to the application's main method via an array of Strings.

- In the previous example, the command-line arguments passed to the Sort application in an array that contains a single String: "friends.txt".

Command Line Arguments

- Echoing Command-Line Arguments
- The Echo example displays each of its command-line arguments on a line by itself:

```
public class Echo {
    public static void main (String[] args) {
        for (String s: args) {
            System.out.println(s);
        }
    }
}
```

Command Line Arguments

The following example shows how a user might run Echo. User input is in italics.

java Echo Drink Hot Java
Drink
Hot
Java
Note that the application displays each word — Drink, Hot, and Java — on a line by itself. This is because the space character separates command-line arguments. To have Drink, Hot, and Java interpreted as a single argument, the user would join them by enclosing them within quotation marks.

java Echo "Drink Hot Java"
Drink Hot Java

Command Line Arguments

- Parsing Numeric Command-Line Arguments
- If an application needs to support a numeric command-line argument, it must convert a String argument that represents a number, such as "34", to a numeric value. Here is a code snippet that converts a command-line argument to an int:

```
int firstArg;
if (args.length > 0) {
    try {
        firstArg = Integer.parseInt(args[0]);
    } catch (NumberFormatException e) {
        System.err.println("Argument" + args[0] + " must be an integer.");
        System.exit(1);
    }
}
```

- parseInt throws a NumberFormatException if the format of args[0] isn't valid. All of the Number classes — Integer, Float, Double, and so on — have parseXXX methods that convert a String representing a number to an object of their type.

Some Assignments ☺

1. Write a program which prints the following information about at least 5 persons:

   NAME MAIL-ID EMPLOYEE-CODE PHONE

   Eg. Umesh umesh@cse p03161 25764728

   Salil salil@cse p03160 25764728

   Each entry should be on a separate line.

2. Write a program that prints the following line on the screen along with quotes.

   "Can we print '\' with System.out.println() statement?"

End

Thank you…

REFRENCES

1. Java : The Complete Reference , Patrick Naughton, Herbert Schildt

2. Thinking in Java (http://www.mindview.net/Books/TIJ/) , Bruce Eckel

3. https://www.javatpoint.com/java-tutorial

4. https://docs.oracle.com/javase/tutorial/

Classes and Objects

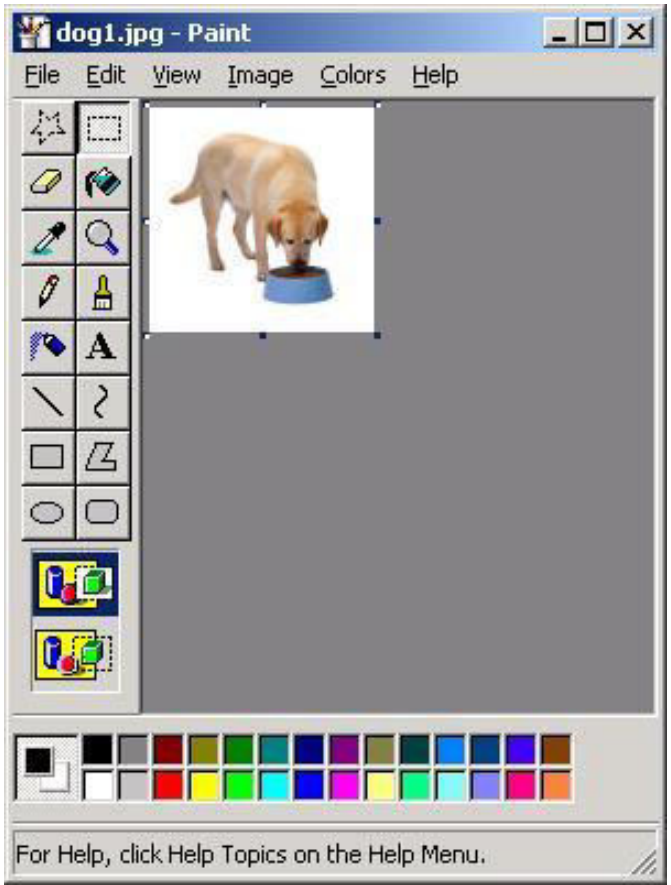
- A Java program consists of one or more classes
- A class is an abstract description of objects
- Here is an example class:
  - `class Dog { ...description of a dog goes here... }`
- Here are some objects of that class:



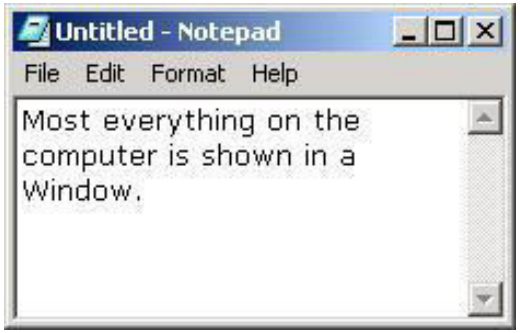



More Objects

- Here is another example of a class:
  - class Window { … }
- Here are some examples of Windows:







Classes contain data definitions

- Classes describe the data held by each of its objects
- Example:
  - class Dog {

```
String name;
int age;
```

    ...rest of the class...

    }



- A class may describe any number of objects
  - Examples: "Fido", 3; "Rover", 5; "Spot", 3;
- A class may describe a single object, or even no objects at all

Classes contain methods

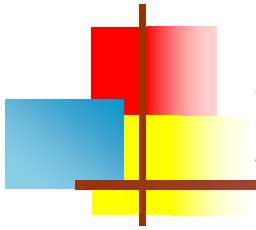
- A class may contain methods that describe the behavior of objects
- Example:
  - class Dog {

```
    ...
    void bark() {
        System.out.println("Woof!");
    }
}
```

Methods usually go after the data

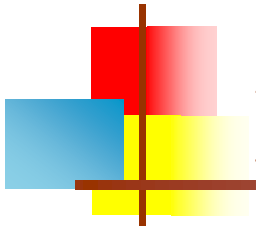


- When we ask a *particular* Dog to bark, it says "Woof!"
- Only Dog *objects* can bark; the *class* Dog cannot bark



Methods contain statements

- A statement causes the object to do something
  - (A better word would be “command”—but it isn’t)
- Example:
  - `System.out.println("Woof!");`
  - This causes the particular `Dog` to “print” (actually, display on the screen) the characters `Woof!`



Methods may contain temporary data

- Data described in a class exists in all objects of that class
  - Example: Every Dog has its own name and age
- A method may contain local *temporary* data that exists only until the method finishes
- Example:
  - ```
    void wakeTheNeighbors( ) {
        int i = 50;       // i is a temporary variable
        while (i > 0) {
            bark( );
            i = i - 1;
        }
    }
    ```
    ```

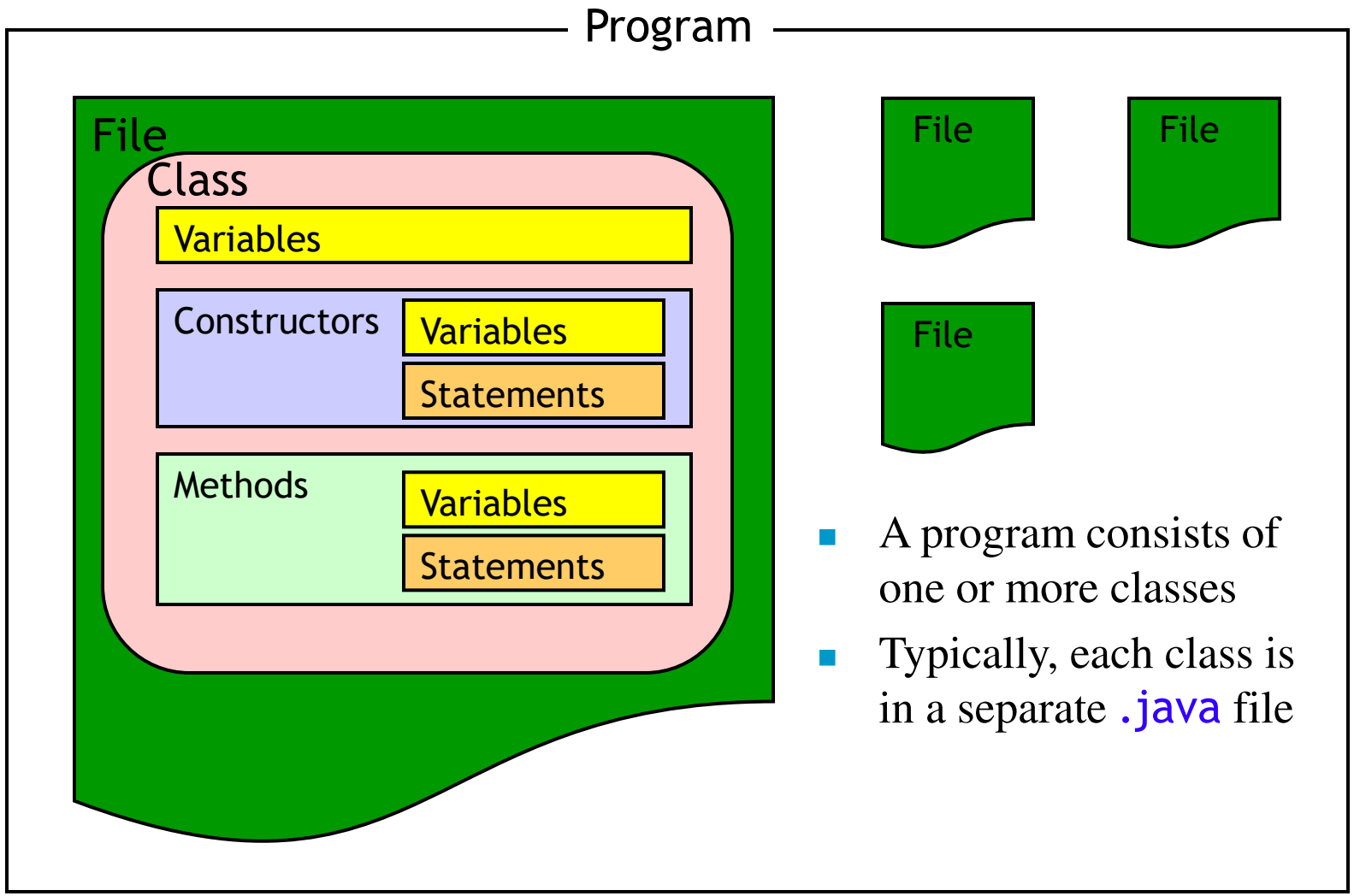
Classes *always* contain constructors

- A constructor is a piece of code that "constructs," or creates, a new object of that class
- If you don't write a constructor, Java defines one for you (behind the scenes)
- You can write your own constructors
- Example:

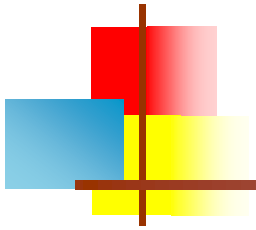
```
class Dog {
    String name;
    int age;
    Dog(String n, int age) {
        name = n;
        this.age = age;
    }
}
```

(This part is the constructor)

Diagram of program structure

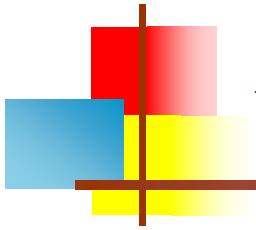


- A program consists of one or more classes
- Typically, each class is in a separate `.java` file



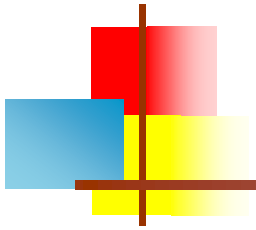
Summary

- A program consists of one or more classes
- A class is a description of a kind of object
  - In most cases, it is the objects that do the actual work
- A class describes data, constructors, and methods
  - An object's data is information about that object
  - An object's methods describe how the object behaves
  - A constructor is used to create objects of the class
- Methods (and constructors) may contain temporary data and statements (commands)



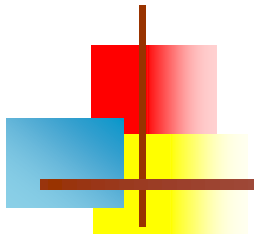
Writing and running programs

- When you *write* a program, you are writing classes and all the things that go into classes
- Your program typically contains commands to create objects (that is, "calls" to constructors)
- When you *run* a program, it creates objects, and those objects interact with one another and do whatever they do to cause something to happen
  - Analogy: *Writing* a program is like writing the rules to a game; *running* a program is like actually playing the game
- You never know how well the rules are going to work until you try them out



Getting started

- Question: Where do objects come from?
  - Answer: They are created by other objects.
- Question: Where does the *first* object come from?
  - Answer: Programs have a special main method, not part of any object, that is executed in order to get things started
- ```
  public static void main(String[ ] args) {
      Dog fido = new Dog("Fido", 5); // creates a Dog
  }
  ```
- The special keyword static says that the main method belongs to *the class itself*, not to objects of the class
  - Hence, the main method can be "called" before any objects are created
  - Usually, the main method gets things started by creating one or more objects and telling them what to do
  ```

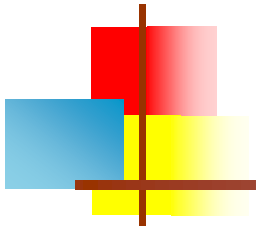


Classes & Objects – Part II

Java Programming

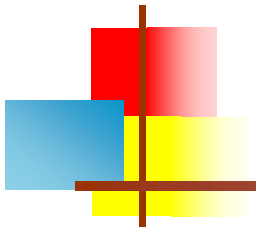
Module I





Creating an Object

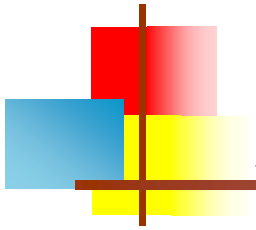
- A class provides the blueprints for objects
- An object is created from a class.
- In Java, the new keyword is used to create new objects.
- There are three steps when creating an object from a class −
  - **Declaration** − A variable declaration with a variable name with an object type.
  - **Instantiation** − The 'new' keyword is used to create the object.
  - **Initialization** − The 'new' keyword is followed by a call to a constructor. This call initializes the new object.



e.g.

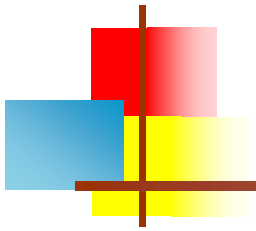
```
public class Puppy {
   public Puppy(String name) {
      // This constructor has one parameter, name.
      System.out.println("Passed Name is :" + name );
   }

   public static void main(String []args) {
      // Following statement would create an object myPuppy
      Puppy myPuppy = new Puppy( "tommy" );
   }
}
```



Accessing Instance Variables and Methods

- Instance variables and methods are accessed via created objects.
- To access an instance variable −
  - /* First create an object */
    - ObjectReference = new Constructor();
  - /* Now call a variable as follows */
    - ObjectReference.variableName;
  - /* Now you can call a class method as follows */
    - ObjectReference.MethodName();



```
public class Puppy {
  int puppyAge;

  public Puppy(String name) {
    // This constructor has one parameter, name.
    System.out.println("Name chosen is :" + name );
  }

  public void setAge( int age ) {
    puppyAge = age;
  }

  public int getAge( ) {
    System.out.println("Puppy's age is :" + puppyAge );
    return puppyAge;
  }

  public static void main(String []args) {
    /* Object creation */
    Puppy myPuppy = new Puppy( "tommy" );

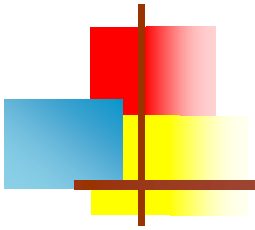
    /* Call class method to set puppy's age */
    myPuppy.setAge( 2 );

    /* Call another class method to get puppy's age */
    myPuppy.getAge( );

    /* You can access instance variable as follows as well */
    System.out.println("Variable Value :" + myPuppy.puppyAge );
  }
}
```

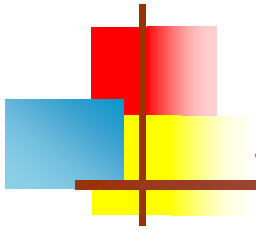
Output:

Name chosen is :tommy
Puppy's age is :2
Variable Value :2



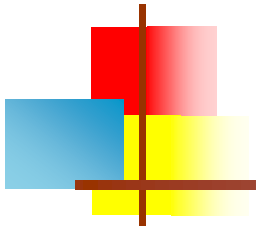
Source File Declaration Rules

- There can be only one public class per source file.
- A source file can have multiple non-public classes.
- The public class name should be the name of the source file as well which should be appended by **.java** at the end. For example: the class name is *public class Employee{}* then the source file should be as Employee.java.
- If the class is defined inside a package, then the package statement should be the first statement in the source file.
- If import statements are present, then they must be written between the package statement and the class declaration. If there are no package statements, then the import statement should be the first line in the source file.
- Import and package statements will imply to all the classes present in the source file. It is not possible to declare different import and/or package statements to different classes in the source file.



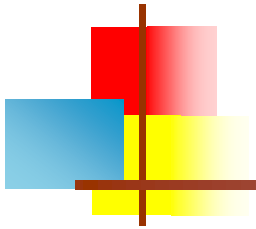
Java Package

- In simple words, it is a way of categorizing the classes and interfaces.
- When developing applications in Java, hundreds of classes and interfaces will be written, therefore categorizing these classes is a must as well as makes life much easier.



Import Statements

- In Java if a fully qualified name, which includes the package and the class name is given, then the compiler can easily locate the source code or classes.
- Import statement is a way of giving the proper location for the compiler to find that particular class.
- E.g.
  - import java.io.*;



```
import java.io.*;
public class Employee {

  String name;
  int age;
  String designation;
  double salary;

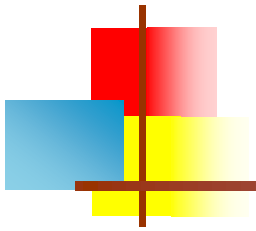
  // This is the constructor of the class Employee
  public Employee(String name) {
    this.name = name;
  }

  // Assign the age of the Employee  to the variable age.
  public void empAge(int empAge) {
    age = empAge;
  }

  /* Assign the designation to the variable designation.*/
  public void empDesignation(String empDesig) {
    designation = empDesig;
  }

  /* Assign the salary to the variable              salary.*/
  public void empSalary(double empSalary) {
    salary = empSalary;
  }

  /* Print the Employee details */
  public void printEmployee() {
    System.out.println("Name:"+ name );
    System.out.println("Age:" + age );
    System.out.println("Designation:" + designation );
    System.out.println("Salary:" + salary);
  }
}
```



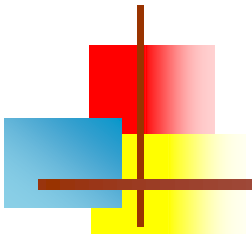
```
import java.io.*;
public class EmployeeTest {
  public static void main(String args[]) {
    /* Create two objects using constructor */
    Employee empOne = new Employee("James Smith");
    Employee empTwo = new Employee("Mary Anne");

    // Invoking methods for each object created
    empOne.empAge(26);
    empOne.empDesignation("Senior Software Engineer");
    empOne.empSalary(1000);
    empOne.printEmployee();

    empTwo.empAge(21);
    empTwo.empDesignation("Software Engineer");
    empTwo.empSalary(500);
    empTwo.printEmployee();
  }
}
```

Output:

```
C:\> javac Employee.java
C:\> javac EmployeeTest.java
C:\> java EmployeeTest
Name:James Smith
Age:26
Designation:Senior Software
Engineer
Salary:1000.0
Name:Mary Anne
Age:21
Designation:Software Engineer
Salary:500.0
```

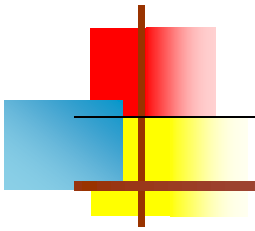


Strings

Java Programming

Module I





Strings

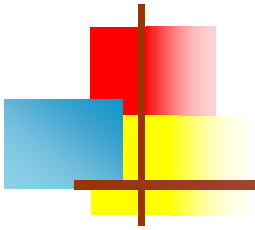
- Java string is a sequence of characters. They are objects of type String.
- Once a String object is created it cannot be changed. Stings are Immutable.
- To perform changes in original strings use the class called StringBuffer.
- String and StringBuffer classes are declared final, so there cannot be subclasses of these classes.
- The default constructor creates an empty string.
  - String s = new String();

Creating Strings

◉ **String str = "abc"; is equivalent to:**

char data[] = {'a', 'b', 'c'}; String str = new String(data);

◉Construct a string object by passing another string object. String str2 = new String(str);



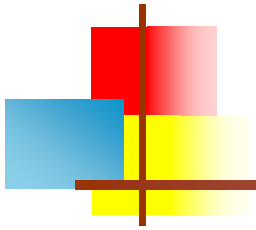
Comparison with character arrays

- Although a **`String`** has many similarities to an array of characters, there are some important differences.
- You don't need to use **`new`** to create a string unless you want to use one of the **`String`** class constructors

```
String s1 = "Hello!";
String s2 = new String("Hello!");
```

- You DO NOT use **`[]`** to access the characters in the string.
- The contents of a String cannot be changed after it has been created.
  - You can have a String variable refer to a different string, but you cannot change characters within the original string.
  - There is NO equivalent to:

```
char[] x = new char[] {'h','e','l','l','o'};
x[2] = 'q';
```



Conversion to/from character arrays

- To convert from a character array to a **`String`**:

```
char[] x = new char[] {'h','e','l','l','o'};
String s1 = new String( x );
```

- To convert from a String to a character array:

```
char[] x = aString.toCharArray( );
```

String METHODS

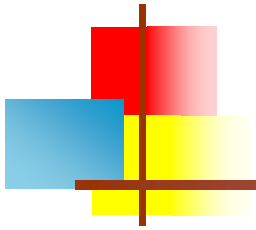
- The length() method returns the length of the string.

  Eg: System.out.println(“Hello”.length()); // prints 5

- The + operator is used to concatenate two or more strings. Eg: String myname = “Harry”

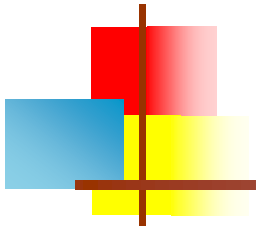
String str = “My name is” + myname+ “.”;

- For string concatenation the Java compiler converts an operand to a String whenever the other operand of the + is a String object.



- public char **charAt**(int index)
  - Returns the character at the specified index. An index ranges from 0 to length() - 1. The first character of the sequence is at index 0, the next at index 1, and so on, as for array indexing.

```
char ch;
ch = “abc”.charAt(1); // ch = “b”
```

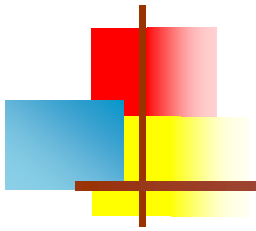


- **equals() -** Compares the invoking string to the specified object. The result is true if and only if the argument is not null and is a String object that represents the same sequence of characters as the invoking object.

public boolean **equals**(Object anObject)

- **equalsIgnoreCase()-** Compares this String to another String, ignoring case considerations. Two strings are considered equal ignoring case if they are of the same length, and corresponding characters in the two strings are equal ignoring case.

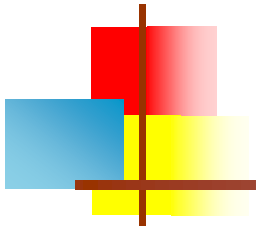
public boolean **equalsIgnoreCase**(String anotherString)



```
String myStr1 = "Hello";
String myStr2 = "Hello";
String myStr3 = "Another String";

System.out.println(myStr1.equals(myStr2)); // Returns true
because they are equal

System.out.println(myStr1.equals(myStr3)); // false
```



- **startsWith()** – Tests if this string starts with the specified prefix.

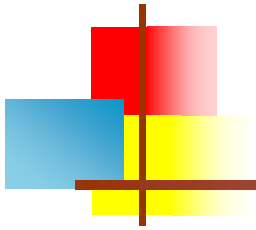
public boolean **startsWith**(String prefix)

“Figure”.startsWith(“Fig”); // true

- **endsWith() -** Tests if this string ends with the specified suffix.

public boolean **endsWith**(String suffix)

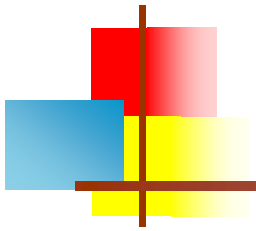
“Figure”.endsWith(“re”); // true



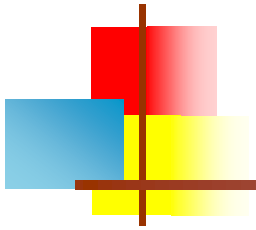
- **compareTo()** - Compares two strings.
  - The result is a negative integer if this String object lexicographically precedes the argument string.
  - The result is a positive integer if this String object lexicographically follows the argument string.
  - The result is zero if the strings are equal.
  - compareTo returns 0 exactly when the equals(Object) method would return true.

public int **compareTo**(String anotherString) public int

compareToIgnoreCase(String str)



```
String s1="hello";
String s2="hello";
String s3="meklo";
String s4="hemlo";
String s5="flag";
System.out.println(s1.compareTo(s2));//0 because both are equ
al
System.out.println(s1.compareTo(s3));//-
5 because "h" is 5 times lower than "m"
System.out.println(s1.compareTo(s4));//-
1 because "l" is 1 times lower than "m"
System.out.println(s1.compareTo(s5));//2 because "h" is 2 times
 greater than "f"
```



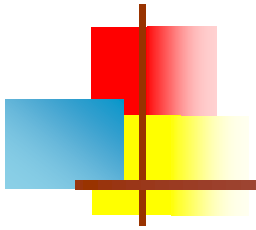
Comparing Strings

- What is the value of **`result`** for these examples?
  - Example 1:

```
String str1 = "abcde" ;
String str2 = "abcfg" ;
int result = str1.compareTo(str2);
```

  - Example 2:

```
String str1 = "abcde" ;
String str2 = "ab" ;
int result = str1.compareTo(str2);
```



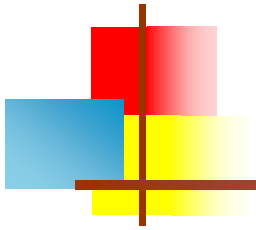
- **indexOf** – Searches for the first occurrence of a character or substring. Returns -1 if the character does not occur.

public int **indexOf**(String str) - Returns the index within this string of the first occurrence of the specified substring.

String str = “How was your day today?”;

str.indexof(‘t’);
str.indexof(“was”);

- **lastIndexOf()** –Searches for the last occurrence of a character or substring. The methods are similar to indexOf().



- **substring()** - Returns a new string that is a substring of this string. The substring begins with the character at the specified index and extends to the end of this string.

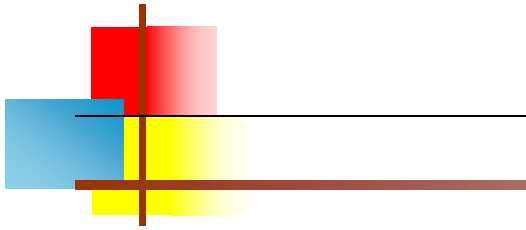
public String **substring**(int beginIndex)

Eg: "unhappy".substring(2) returns "happy"

- public String **substring**(int beginIndex, int endIndex) Eg: "smiles".substring(1, 5) returns "mile“

String METHODS

Method call	Meaning
S2=s1.toLowerCase()	Convert string s1 to lowercase
S2=s1.toUpperCase()	Convert string s1 to uppercase
S2=s1.repalce('x', 'y')	Replace occurrence x with y
S2=s1.trim()	Remove whitespaces at the beginning and end of the string s1
S1.equals(s2)	If s1 equals to s2 return true
S1.equalsIgnoreCase(s2)	If s1==s2 then return true with irrespective of case of charecters
S1.length()	Give length of s1
S1.CharAt(n)	Give nth character of s1 string
S1.compareTo(s2)	If s1<s2 –ve no If s1>s2 +ve no If s1==s2 then 0
S1.concat(s2)	Concatenate s1 and s2
S1.substring(n)	Give substring staring from nth character



String Operations

String Operations

concat() - Concatenates the specified string to the end of this string.

If the length of the argument string is 0, then this String object is returned.

Otherwise, a new String object is created, containing the invoking string with the contents of the str appended to it.

public String **concat**(String str) "to".concat("get").concat("her")

returns "together"

String Operations

- replace()- Returns a new string resulting from replacing all occurrences of oldChar in this string with newChar.

public String **replace**(char oldChar, char newChar) “iam aq iqdiaq

” . replace(‘q', ‘n')

returns “I am an indian"

String Operations

- **trim() -** Returns a copy of the string, with leading and trailing whitespace omitted.

```
public String trim()
    String s = “      Hi Mom!  “
        s.trim();
    S = “Hi Mom!”
```

- **valueOf()** – Returns the string representation of the char array argument.

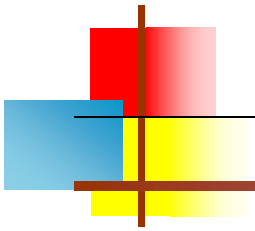
public static String **valueOf**(char[] data)

String Operations

- **toLowerCase():** Converts all of the characters in a String to lower case.
- **toUpperCase():** Converts all of the characters in this String to upper case.

public String **toLowerCase**() public String **toUpperCase**()

Eg: “HELLO THERE”.toLowerCase(); “hello there”.toUpperCase();



StringBuffer

- A StringBuffer is like a String, but can be modified.
- The length and content of the StringBuffer sequence can be changed through certain method calls.
- StringBuffer defines three constructors:
  - StringBuffer()
  - StringBuffer(int size)
  - StringBuffer(String str)

StringBuffer Operations

- The principal operations on a StringBuffer are the append and insert methods, which are overloaded so as to accept data of any type.

Here are few append methods:

StringBuffer append(String str)

StringBuffer append(int num)

- The append method always adds these characters at the end of the buffer.

StringBuffer Operations

- Insert method adds the characters at a specified point.

Here are few insert methods:

```
StringBuffer insert(int index, String str)
 StringBuffer append(int index, char ch)
```

Index specifies at which point the string will be inserted into the invoking StringBuffer object.

StringBuffer Operations

- **delete() -** Removes the characters in a substring of this StringBuffer. The substring begins at the specified start and extends to the character at index end - 1 or to the end of the StringBuffer if no such character exists. If start is equal to end, no changes are made.

```
public StringBuffer delete(int start, int end)
```

StringBuffer Operations

- **replace() -** Replaces the characters in a substring of this StringBuffer with characters in the specified String.

```
public StringBuffer replace(int start, int end,
  String str)
```

- **substring() -** Returns a new String that contains a subsequence of characters currently contained in this StringBuffer. The substring begins at the specified index and extends to the end of the StringBuffer.

```
public String substring(int start)
```

StringBuffer Operations

- **reverse() -** The character sequence contained in this string buffer is replaced by the reverse of the sequence.

```
public StringBuffer reverse()
```

- **length() -** Returns the length of this string buffer.

```
public int length()
```

- **setLength**() **-** Sets the length of the StringBuffer.

```
public void setLength(int newLength)
```

StringBuffer Operations

- **capacity() -** Returns the current capacity of the String buffer. The capacity is the amount of storage available for newly inserted characters.

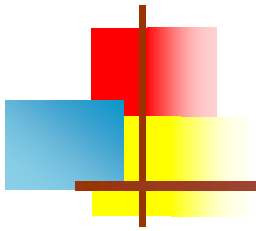
```
public int capacity()
```

- **charAt() -** The specified character of the sequence currently represented by the string buffer, as indicated by the index argument, is returned.

```
public char charAt(int index)
```

StringBuffer Methods

Methods	Meaning
S1.setCharAt(n, ‘x’)	Modify the nth character to x
S1.appen(s2)	Append s2 string at the end of s1
S1.insert(n,s2)	Insert string s2 in s1 at position n
S1.setLength(n)	Set length of string s1 to n



e.g.

StringBuffer sb = new StringBuffer(“Hello”);

sb.length(); // 5

sb.capacity(); // 21 (16 characters room is added if no size is specified)

sb.charAt(1); // e

sb.setCharAt(1,’i’); // Hillo

sb.setLength(2); // Hi

sb.append(“l”).append(“l”); // Hill

sb.insert(0, “Big “); // Big Hill

sb.replace(3, 11, “ ”); // Big

sb.reverse(); // gib

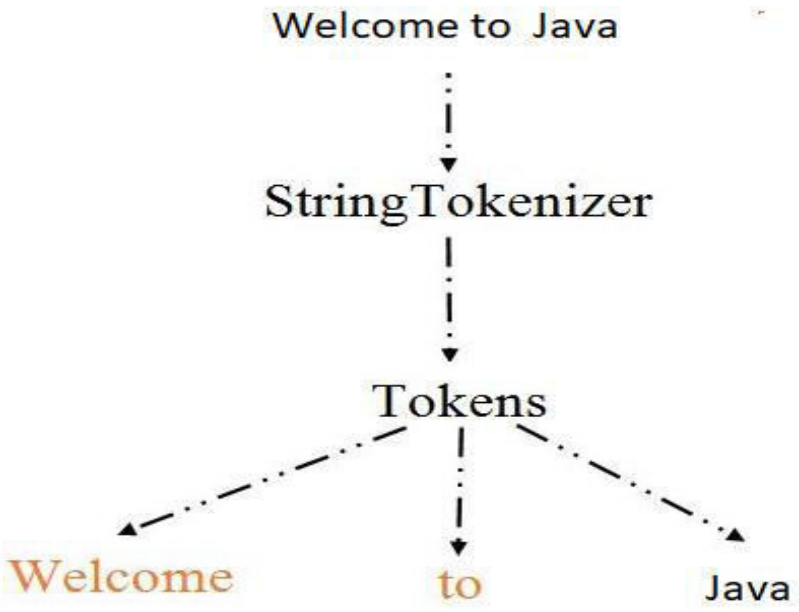


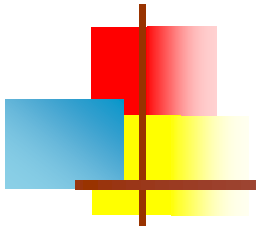
Java String tokenizer (to be updated)

In Java, StringTokenizer is used to break a string into tokens based on provided delimiter. Delimiter can be specified either at the time of object creation or on a per-token basis.

Its object internally maintains a current position within the string to be tokenized. It is located into java.util package.

In string, tokenizer objects are maintained internally and returns a token of a substring from the given string.


Welcome to Java
StringTokenizer
Tokens
Welcome
to
Java



Following are the constructors in string tokenizer

1. StringTokenizer(String str)

2. StringTokenizer(String str, String delim)

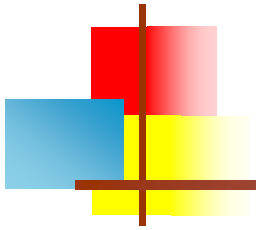
3. StringTokenizer(String str, String delim, booleanreturnValue)

Following are the methods in string tokenizer

1. booleanhasMoreTokens()

2. String nextToken()

3. String nextToken(String delim)



4. booleanhasMoreElements()

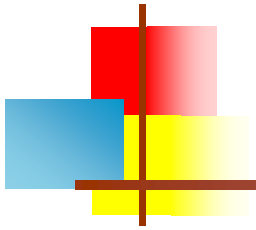
5. Object nextElement()

6. intcountTokens()

Example:

In this example, we are using Stringtokenizer to break string into **tokens based on space.**

import java.util.StringTokenizer;

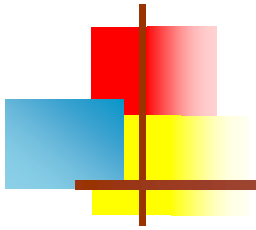


```
public class TokenDemo1
{
        public static void main(String args[])
        {
StringTokenizerobj = new StringTokenizer("Welcome to Java "," ");
                while (obj.hasMoreTokens())
                {

        System.out.println(obj.nextToken());
                }
        }
}
```

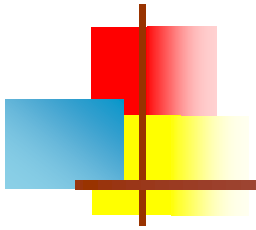
Output

Welcome to Java



Example to understand tokenizer, here we are breaking string into tokens based on the colon (:) delimiter.

```
import java.util.*;
public class TokenDemo2{
        public static void main(String args[])
        {
                String a= " : ";
                String b= "Welcome : to : Java : . : How : are : You : ?";
                StringTokenizer c = new StringTokenizer(b, a);
                int count1 = c.countTokens();
                for (int i = 0; i<count1; i++)
                        System.out.println("token [" + i + "] : "
                                + c.nextToken());
```



```
StringTokenizer d= null;
                        while (c.hasMoreTokens())

        System.out.println(d.nextToken());
        }
}
```

Chapter : Inheritance

Inheritance



- Another fundamental object-oriented technique is inheritance, used to organize and create reusable classes
- This Chapter focuses on:
  - deriving new classes from existing classes
  - creating class hierarchies
  - the `protected` modifier
  - polymorphism via inheritance
  - inheritance hierarchies for interfaces
  - inheritance used in graphical user interfaces

Inheritance



- *Inheritance* allows a software developer to derive a new class from an existing one
- The existing class is called the *parent class,* or *superclass*, or *base class*
- The derived class is called the *child class* or *subclass*.
- As the name implies, the child inherits characteristics of the parent
- That is, the child class inherits the methods and data defined for the parent class

Inheritance

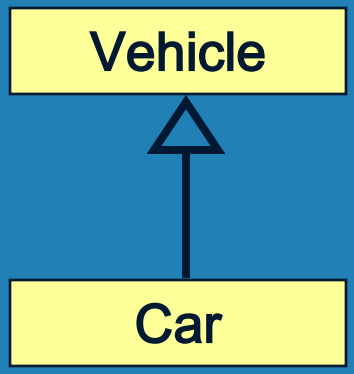


- To tailor a derived class, the programmer can add new variables or methods, or can modify the inherited ones
- *Software reuse* is at the heart of inheritance
- By using existing software components to create new ones, we capitalize on all the effort that went into the design, implementation, and testing of the existing software

Inheritance



- Inheritance relationships often are shown graphically in a UML class diagram, with an arrow with an open arrowhead pointing to the parent class



Inheritance should create an *is-a relationship*, meaning the child *is a* more specific version of the parent

Deriving Subclasses



- In Java, we use the reserved word `extends` to establish an inheritance relationship

```
class Car extends Vehicle
{
    // class contents
}
```

The protected Modifier



- Visibility modifiers determine which class members are inherited and which are not
- Variables and methods declared with `public` visibility are inherited; those with `private` visibility are not
- But `public` variables violate the principle of encapsulation
- There is a third visibility modifier that helps in inheritance situations: `protected`

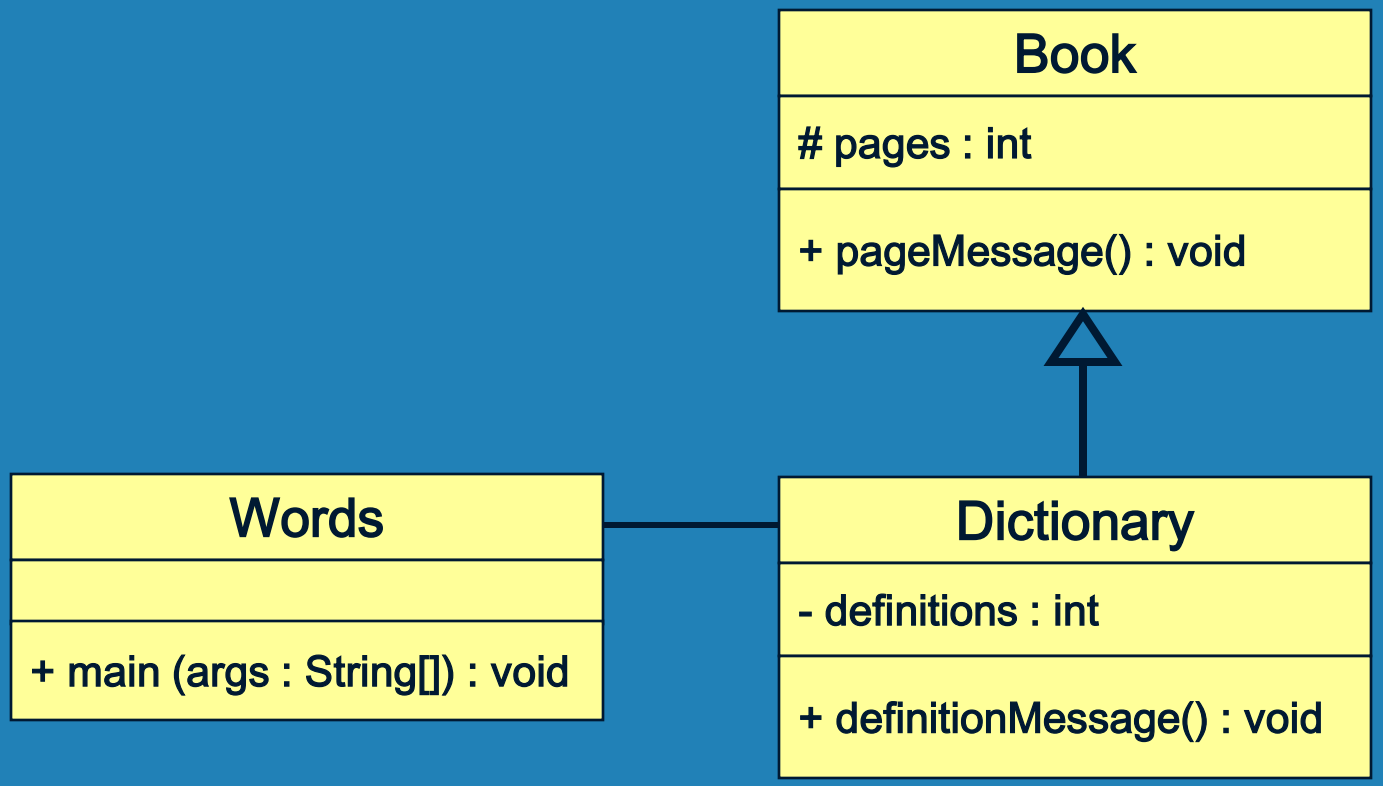
The protected Modifier



- The `protected` modifier allows a member of a base class to be inherited into a child
- Protected visibility provides more encapsulation than public visibility does
- However, protected visibility is not as tightly encapsulated as private visibility
- Protected variables and methods can be shown with a # symbol preceding them in UML diagrams

UML Diagram for Words





The super Reference



- Constructors are not inherited, even though they have public visibility
- Yet we often want to use the parent's constructor to set up the "parent's part" of the object
- The `super` reference can be used to refer to the parent class, and often is used to invoke the parent's constructor

The super Reference



- A child's constructor is responsible for calling the parent's constructor
- The first line of a child's constructor should use the `super` reference to call the parent's constructor
- The `super` reference can also be used to reference other variables and methods defined in the parent's class

Multiple Inheritance



- Java supports *single inheritance*, meaning that a derived class can have only one parent class
- *Multiple inheritance* allows a class to be derived from two or more classes, inheriting the members of all parents
- Collisions, such as the same variable name in two parents, have to be resolved
- Java does not support multiple inheritance
- In most cases, the use of interfaces gives us aspects of multiple inheritance without the overhead

Overriding Methods



- A child class can *override* the definition of an inherited method in favor of its own
- The new method must have the same signature as the parent's method, but can have a different body
- The type of the object executing the method determines which version of the method is invoked

Overriding



- A parent method can be invoked explicitly using the `super` reference
- If a method is declared with the `final` modifier, it cannot be overridden

Overloading vs. Overriding

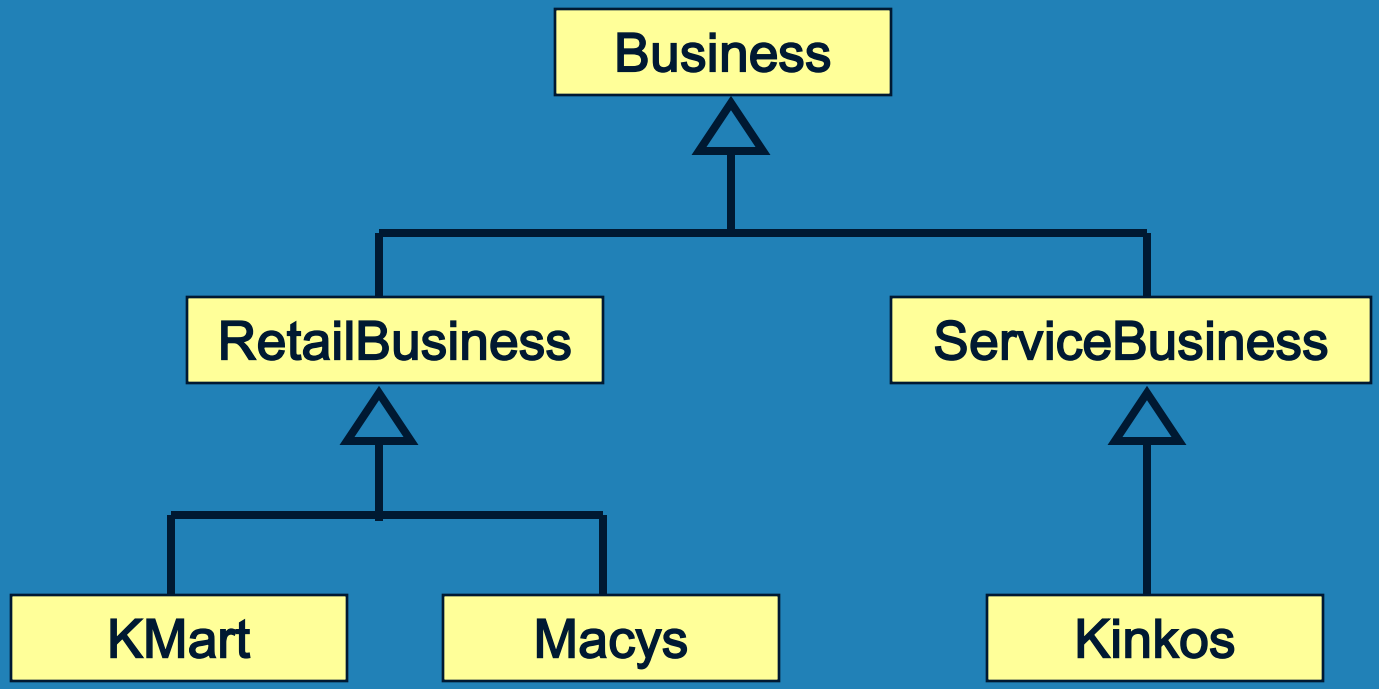


- Don't confuse the concepts of overloading and overriding
- Overloading deals with multiple methods with the same name in the same class, but with different signatures
- Overriding deals with two methods, one in a parent class and one in a child class, that have the same signature
- Overloading lets you define a similar operation in different ways for different data
- Overriding lets you define a similar operation in different ways for different object types

Class Hierarchies



- A child class of one parent can be the parent of another child, forming a *class hierarchy*



Class Hierarchies



- Two children of the same parent are called *siblings*
- Common features should be put as high in the hierarchy as is reasonable
- An inherited member is passed continually down the line
- Therefore, a child class inherits from all its ancestor classes
- There is no single class hierarchy that is appropriate for all situations

The Object Class



- A class called `Object` is defined in the `java.lang` package of the Java standard class library
- All classes are derived from the `Object` class
- If a class is not explicitly defined to be the child of an existing class, it is assumed to be the child of the `Object` class
- Therefore, the `Object` class is the ultimate root of all class hierarchies

The Object Class



- The `Object` class contains a few useful methods, which are inherited by all classes
- For example, the `toString` method is defined in the `Object` class
- Every time we have defined `toString`, we have actually been overriding an existing definition
- The `toString` method in the `Object` class is defined to return a string that contains the name of the object's class together along with some other information

The Object Class



- All objects are guaranteed to have a `toString` method via inheritance
- Thus the `println` method can call `toString` for any object that is passed to it

The Object Class



- The `equals` method of the `Object` class returns true if two references are aliases
- We can override `equals` in any class to define equality in some more appropriate way
- The `String` class (as we've seen) defines the `equals` method to return true if two `String` objects contain the same characters
- Therefore, the `String` class has overridden the `equals` method inherited from `Object` in favor of its own version

Abstract Classes



- An *abstract class* is a placeholder in a class hierarchy that represents a generic concept
- An abstract class cannot be instantiated
- We use the modifier `abstract` on the class header to declare a class as abstract:

```
public abstract class Whatever
{
   // contents
}
```

Abstract Classes



- An abstract class often contains abstract methods with no definitions (like an interface does)
- Unlike an interface, the `abstract` modifier must be applied to each abstract method
- An abstract class typically contains non-abstract methods (with bodies), further distinguishing abstract classes from interfaces
- A class declared as abstract does not need to contain abstract methods

Abstract Classes



- The child of an abstract class must override the abstract methods of the parent, or it too will be considered abstract
- An abstract method cannot be defined as `final` (because it must be overridden) or `static` (because it has no definition yet)
- The use of abstract classes is a design decision – it helps us establish common elements in a class that is too general to instantiate

Interfaces



- A Java *interface* is a collection of abstract methods and constants
- An *abstract method* is a method header without a method body
- An abstract method can be declared using the modifier `abstract`, but because all methods in an interface are abstract, usually it is left off
- An interface is used to establish a set of methods that a class will implement

Interfaces



interface is a reserved word

None of the methods in an interface are given a definition (body)

```
public interface Doable
{
    public void doThis();
    public int doThat();
    public void doThis2 (float value, char ch);
    public boolean doTheOther (int num);
}
```

A semicolon immediately follows each method header

Interfaces



- An interface cannot be instantiated
- Methods in an interface have public visibility by default
- A class formally implements an interface by:
  - stating so in the class header
  - providing implementations for each abstract method in the interface
- If a class asserts that it implements an interface, it must define all methods in the interface

Interfaces



```
public class CanDo implements Doable
{
    public void doThis ()
    {
        // whatever
    }

    public void doThat ()
    {
        // whatever
    }

    // etc.
}
```

implements is a reserved word

Each method listed in Doable is given a definition

Interfaces



- A class that implements an interface can implement other methods as well
- In addition to (or instead of) abstract methods, an interface can contain constants
- When a class implements an interface, it gains access to all its constants

Comparison with Inheritance



- Interfaces don't define any method actions… you need to fill in all the details
- It essentially just gives a basic collection of method names
- Not a strict hierarchy
- Important: A class can implement several Interfaces
- Can use this to "fake" multiple inheritance

Interfaces



- A class can implement multiple interfaces
- The class must implement all methods in all interfaces listed in the header

```
class ManyThings implements
  interface1, interface2
{
    // all methods of both interfaces
}
```

Example: The Comparable Interface



- Any class can implement `Comparable` to provide a mechanism for comparing objects of that type
- Specifically, implementing `Comparable` means that you need a method `CompareTo`

```
if (obj1.compareTo(obj2) < 0)
    System.out.println ("obj1 is less
  than obj2");
```

The Comparable Interface



- It's up to the programmer to determine what makes one object less than another
- For example, you may define the `compareTo` method of an `Employee` class to order employees by name (alphabetically) or by employee number
- The implementation of the method can be as straightforward or as complex as needed for the situation

Requiring Interfaces



- Interface names can be used like class names in the parameters passed to a method

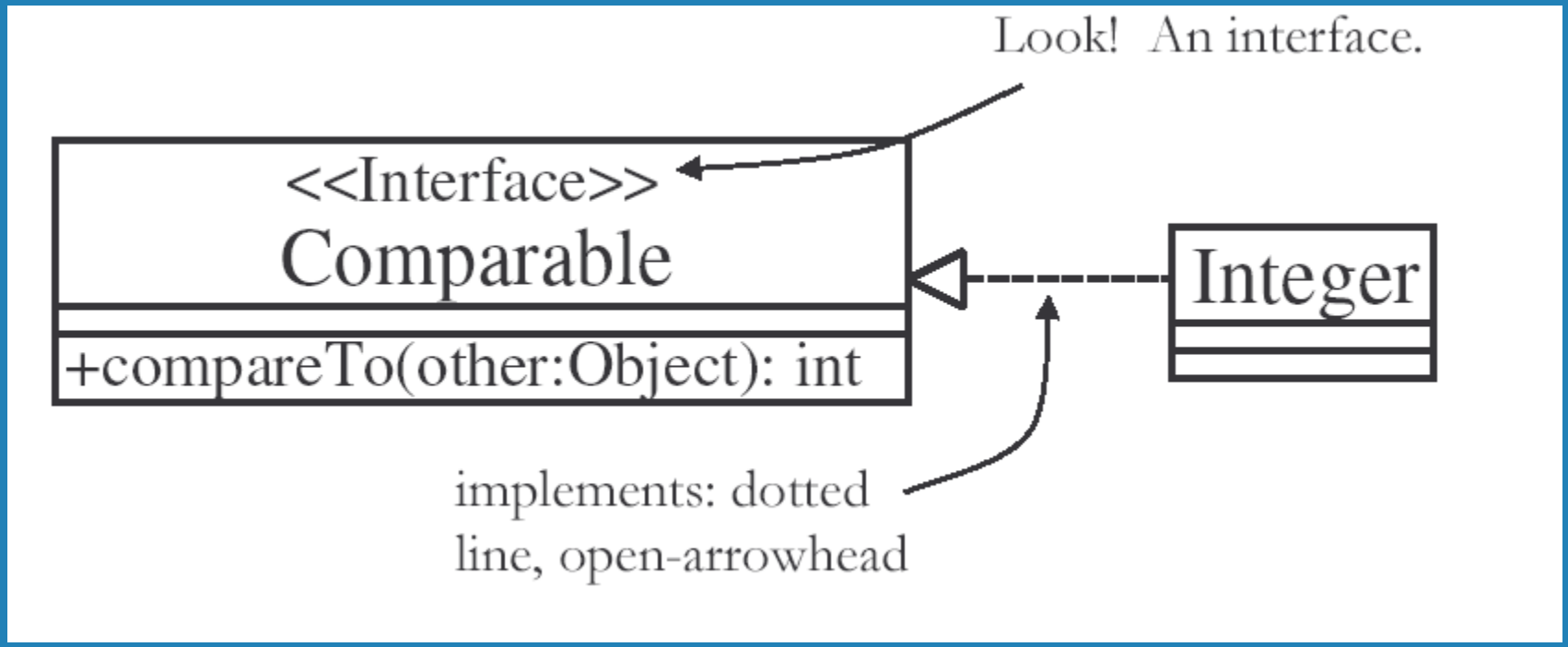
```
public boolean isLess(Comparable a,
                      Comparable b)
{
    return a.compareTo(b) < 0;
}
```

- Any class that "implements `Comparable`" can be used for the arguments to this method

Interfaces in UML



- Interfaces are easy to spot in class diagrams



Interfaces



- You could write a class that implements certain methods (such as `compareTo`) without formally implementing the interface (`Comparable`)
- However, formally establishing the relationship between a class and an interface allows Java to deal with an object in certain ways
- Interfaces are a key aspect of object-oriented design in Java

Built-in Interfaces



- The Java standard library includes lots more built-in interfaces
  - they are listed in the API with the classes
- Examples:
  - `Clonable` – implements a `clone()` method
    - Object cloning refers to the creation of an exact copy of an object. It creates a new instance of the class of the current object and initializes all its fields with exactly the contents of the corresponding fields of this object.
  - `Formattable` – can be formatted with `printf`

The Iterator Interface



- An iterator is an object that provides a means of processing a collection of objects one at a time
- An iterator is created formally by implementing the `Iterator` interface, which contains three methods
  - The `hasNext` method returns a boolean result – true if there are items left to process
  - The `next` method returns the next object in the iteration
  - The `remove` method removes the object most recently returned by the `next` method

The Iterator Interface



- By implementing the `Iterator` interface, a class formally establishes that objects of that type are iterators
- The programmer must decide how best to implement the iterator functions
- Once established, the for-each version of the `for` loop can be used to process the items in the iterator

Collections



- `Collection` is a general interface for any type that can store multiple values
- Any object `c` that implements `Collections` has these methods
  - `c.add(e)`
  - `c.remove(e)`
  - `c.size()`

Collection Sub-Interfaces



- Interfaces that are derived from `Collection`
- `Set`: unordered, can't add the same object twice
- `List`: ordered, adds new methods
  - `get(i)`: get the ith element
  - `set(i,e)`: set the ith element to `e`

Collection Implementations



- Also in the standard library: many good implementations of these interfaces
- List: `ArrayList, Stack, LinkedList`
- Sets: `HashSet, TreeSet`
- Each implementation has some differences… suitable for particular problems
  - e.g. additional methods, different type restrictions, etc.

Implementing versus Inheriting



- Implementing an Interface is very similar to inheriting a class

```
class MyClass implements MyInterface {…}
```

  - Takes everything from MyInterface and puts it in MyClass
  - Except all the methods must be implemented here
  - No previous implementations to fall back on

Interfaces vs. Abstract Classes



- Similarities
  - neither can be instantiated
  - both can be used as the starting point for a class
- Differences
  - A class can contain implementations of methods
  - A class can implement many interfaces, but only one class

Comparison



- In order of “abstractness”:
  - Interface
    - no method implementations
    - can’t be instantiated
  - Abstract class
    - some method implementations
    - can’t be instantiated
  - Non-abstract class
    - all methods implemented
    - can be instantiated

Polymorphism



- A reference can be *polymorphic*, which can be defined as "having many forms"

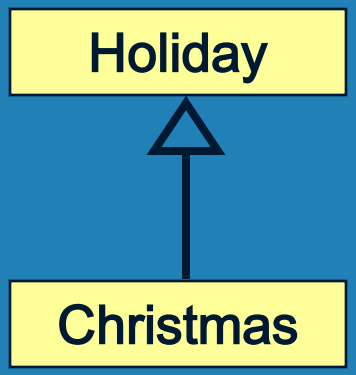
```
obj.doIt();
```

- This line of code might execute different methods at different times if the object that `obj` points to changes
- Polymorphic references are resolved at run time; this is called *dynamic binding*
- Careful use of polymorphic references can lead to elegant, robust software designs
- Polymorphism can be accomplished using inheritance or using interfaces

References and Inheritance



- An object reference can refer to an object of its class, or to an object of any class related to it by inheritance
- For example, if the `Holiday` class is used to derive a child class called `Christmas`, then a `Holiday` reference could be used to point to a `Christmas` object




```
Holiday day;
day = new Christmas();
```

References and Inheritance



- Assigning a predecessor object to an ancestor reference is considered to be a widening conversion, and can be performed by simple assignment
- Assigning an ancestor object to a predecessor reference can be done also, but it is considered to be a narrowing conversion and must be done with a cast
- The widening conversion is the most useful
- An `Object` reference can be used to refer to any object
  - An `ArrayList` is designed to hold `Object` references

Polymorphism via Inheritance



- It is the type of the object being referenced, not the reference type, that determines which method is invoked
- Suppose the `Holiday` class has a method called `celebrate`, and the `Christmas` class overrides it
- Now consider the following invocation:

```
day.celebrate();
```

- If `day` refers to a `Holiday` object, it invokes the `Holiday` version of `celebrate`; if it refers to a `Christmas` object, it invokes the `Christmas` version

AMITY UNIVERSITY

Java Programming

Course Code: IT 201

MODULE – I

Dr. Sheenu Rizvi
Asstt. Professor
Dept Of CSE/IT ASET
AUUP Lucknow.

AMITY UNIVERSITY

Polymorphism in Java

Polymorphism in Java is a concept by which we can perform a *single action in different ways*. Polymorphism is derived from 2 Greek words: poly and morphs. The word "poly" means many and "morphs" means forms. So polymorphism means many forms.

There are two types of polymorphism in Java: **compile-time polymorphism and runtime polymorphism**. We can perform polymorphism in java by method overloading and method overriding.

If you overload a static method in Java, it is the example of compile time polymorphism. Here, we will focus on runtime polymorphism in java.

AMITY UNIVERSITY

Runtime Polymorphism in Java

Runtime polymorphism or **Dynamic Method Dispatch** is a process in which a call to an overridden method is resolved at runtime rather than compile-time.

In this process, an overridden method is called through the **reference variable of a superclass**. The determination of the method to be called is based on the object being referred to by the reference variable.

Let's first understand the upcasting before Runtime Polymorphism.

Upcasting

If the reference variable of Parent class refers to the object of Child class, it is known as upcasting. For example:

Upcasting in Java



```
class A { }

class B extends A{ }


A a=new B();//upcasting
```

For **upcasting**, we can use the reference variable of class type or an interface type.
For **Example**:

```
interface I{ }
class A{ }
class B extends A implements I{ }
```

Here, the relationship of B class would be:

B IS-A A
B IS-A I
B IS-A Object

Since Object is the root class of all classes in Java, so we can write B IS-A Object.

Example of Java Runtime Polymorphism

In this example, we are creating two classes Bike and Splendor. Splendor class extends Bike class and overrides its run() method. We are calling the run method by the reference variable of Parent class. Since it refers to the subclass object and subclass method overrides the Parent class method, the subclass method is invoked at runtime.

Since method invocation is determined by the JVM not compiler, it is known as runtime polymorphism.

```
class Bike{
  void run(){System.out.println("running");}
}
class Splendor extends Bike{
  void run(){System.out.println("running safely with 60km");}

  public static void main(String args[]){
    Bike b = new Splendor();//upcasting
    b.run();
  }
}
```

output

running safely with 60km.

Lambda expression

Lambda expression is a feature of Java language which was introduced in Java 8 version. It is a function that has no name and uses a functional approach to execute code. the lambda expression is also known as an anonymous function.

It is designed to provide the implementation of a functional interface. An interface that has only a single abstract method is known as a functional interface. Java provides an annotation @FunctionalInterface, which is used to declare an interface as a functional interface.

Advantages of Lambda Expression

The body of a lambda expression can have one or more statements.

Curly brackets are optional if there is a single statement.

The return statement is optional, use only if the method signature has a return type.

We can pass zero, one, or more parameters to a lambda expression.

The type of parameters can be explicitly declared or it can be inferred from the context.

When there is a single parameter, it is not mandatory to use parentheses. Parentheses are optional.

Syntax

(list of arguments) -> { expression body}

list of arguments can be zero, one, or more.

Arrow_token : It is used to link arguments list and body of expression.

Body: It contains expressions and statement for lambda expression.

Lambda expression with the help of given basic examples:

() -> System.out.println("executing lambda expression."); // zero argument, lambda expression

(String str) -> System.out.print(str); // single argument, lambda expression

(int a, int b) -> a+b; // multiple arguments, sum of two values

(a, b) -> a+b // parameters without types, can be used to sum and concat two strings as well.

(int a, int b) -> return (a+ b); // lambda expression with return statement

(int []) -> {multiple statements; return index;} // it can have multiple statements

In these sample examples, we have variety of lambda expression such as zero argument and single statement, multiple arguments, lambda with return statement, etc. although return statement is optional and it can have multiple statements as well.

From Java 8 and later, we can implement such abstract methods using a lambda expression. This is the strength of lambda expression, notice it does not have any name that's why it is also known as an anonymous function.

Example. (with lambda)

```
interface Runnable{
   public void run();
}
public class Demo {
   public static void main(String[] args) {
      int speed=100;
       // new approach (lambda expression) to implement
      Runnable r=()->{
         System.out.println("Running at the speed of "+speed);
      };
      r.run();
   }
}
```

Output:

```
Running at the speed of 100
```

Example. (without lambda)

```
interface Runnable{
   public void run();
}
public class Demo {
   public static void main(String[] args) {
      int speed=100;
       // new approach (lambda expression) to implement
      Runnable r= new Runnable(){
         System.out.println("Running at the speed of "+speed);
      };
      r.run();
   }
}
```

Output:

```
Running at the speed of 100
```

Example: Lambda Expression With Parameter

Lambda Expression can have zero, one, or multiple parameters as we do with methods. Type of parameter is inferred by the lambda so it is optional, we may or may not mention parameter. See the example wherein second lambda expression we mentioned type of parameter.

```
interface Runnable{
   public void run (int speed);
}
public class Demo {
   public static void main(String[]args) {
      int speed=100;
       // lambda expression:
      Runnable r=(carSpeed)->{
         System.out.println("Running at the speed of "+ carSpeed);
      };
```

```
 r.run (speed);
        // specifying type of parameters
        Runnable r1=(int carSpeed)->{
            System.out.println("Running at the speed of "+ carSpeed);
        };
        r1.run(speed);
    }
}
```

output:

```
Running at the speed of 100
                                    100
```

Lambda Expression using return Statement

The return statement is optional with a lambda expression. We may use it to return a value to the caller, in this example, we used two lambda expressions in which first does not use return statement but the second one use return statement.

```
interface Runnable {
    public String run (int speed, int distance);
}
public class Demo {
    public static void main(String[] args) {

        // lambda expression: without return
        Runnable r = (carSpeed,distance)->
```

```
("Distance covered "+ distance +"Km at the speed of "+carSpeed);
        // calling
String r15 = r.run(80,150);
        System.out.println(r15);
        // lambda expression: with return statement
        Runnable r1 = (int carSpeed, int distance)->{
              return ("Distance covered "+ distance +"Km at the speed of "+carSpeed);
        };
        String fz = r1.run(100,200);
        System.out.println(fz);
            }
}
```

Output:

Distance covered 150Km at the speed of 80

Distance covered 200Km at the speed of 100

AMITY UNIVERSITY

Thank you

AMITY UNIVERSITY

Java Programming

Course Code: IT 201

MODULE – I

Dr. Sheenu Rizvi
Asstt. Professor
Dept Of CSE/IT ASET
AUUP Lucknow.

What is an Applet?

Unlike a Java application program, an applet is specifically designed to be executed within an HTML web document using an external API.

They are basically small programs – more like the web version of an application – that require a Java plugin to run on client browser. They run on the client side and are generally used for internet computing.

You can execute a Java applet in a HTML page exactly as you would include an image in a web page. When you see a HTML page with an applet in a Java-enabled web browser, the applet code gets transferred to the system and is finally run by the Java-enabled virtual machine on the browser.

Applets are also compiled using the javac command but can only run using the applet viewer command or with a browser.

A Java applet is capable of performing all kinds of operations such as play sounds, display graphics, perform arithmetic operations, create animated graphics, etc.

You can integrate an applet into a web page either locally or remotely. You can either create your own applets locally or develop them externally. When stored on a local system, it's called a local applet.

The ones which are stored on a remote location and are developed externally are called remote applets.

Browsers come with Java Runtime environment (JRE) to execute applets and these browsers are called Java-enabled browsers.

The web page contains tags which specify the name of the applet and its URL (Uniform Resource Locator) – the unique location where the applet bytecodes reside on the World Wide Web.

In simple terms, URLs refer to the files on some machine or network. Unlike applications, Java applets are executed in a more restricted environment with harsh security restrictions. They cannot access the resources on the system except the browser-specific services.

What is an Application?

It is a stand-alone Java program that runs with the support of a virtual machine in a client or server side. Also referred to as an application program, a Java application is designed to perform a specific function to run on any Java-compatible virtual machine regardless of the computer architecture.

An application is either executed for the user or for some other application program. Examples of Java applications include database programs, development tools, word processors, text and image editing programs, spreadsheets, web browsers etc.

Java applications can run with or without graphical user interface (GUI). It's a broad term used to define any kind of program in Java, but limited to the programs installed on your machine.

Any application program can access any data or information or any resources available on the system without any security restrictions.

Java application programs run by starting the Java interpreter from the command prompt and are compiled using the javac command and run using the java command.

Every application program generally stays on the machine on which they are deployed. It has a single start point which has a main() method.

AMITY UNIVERSITY

Difference between Application and Applet

Definition of Application and Applet – Applets are feature rich application programs that are specifically designed to be executed within an HTML web document to execute small tasks or just part of it. Java applications, on the other hand, are stand-alone programs that are designed to run on a stand-alone machine without having to use a browser.

Execution of Application and Applet– Applications require main method() to execute the code from the command line, whereas an applet does not require main method() for execution. An applet requires an HTML file before its execution. The browser, in fact, requires a Java plugin to run an applet.

Compilation of Application and Applet–Application programs are compiled using the “javac” command and further executed using the java command. Applet programs, on the other hand, are also compiled using the “javac” command but are executed either by using the “applet viewer” command or using the web browser

Security Access of Application and Applet – Java application programs can access all the resources of the system including data and information on that system, whereas applets cannot access or modify any resources on the system except only the browser specific services.

Restrictions of Application and Applet – Unlike applications, applet programs cannot be run independently, thus require highest level of security. However, they do not require any specific deployment procedure during execution. Java applications, on the other hand, run independently and do not require any security as they are trusted.

AMITY UNIVERSITY

Thank you

Module-2

Interfaces

Dr Supriya Raheja

Learning Objectives

- Interfaces in java
- Diamond problem
- How to implement multiple inheritance in Java

Interfaces



Another way to achieve abstraction in Java



An interface is a completely “abstract class” which is used to group related methods

Important Points



Like abstract classes, interfaces cannot be used to create objects



Interface methods do not have a body - the body is provided by the "implement" class



On implementation of an interface, you must override all of its methods



Interface methods are by default abstract and public



Interface attributes are by default public, static and final



An interface cannot contain a constructor

Syntax

```
interface InterfaceName
{
        variables declaration;
        methods declaration;
}
```

```
//variables declaration

static final type variablename=value;

//methods declaration
return-type methodname1 (parameter_list);
```

Example of interface

```
// Interface
interface Animal {
 // interface method(does not have a body)
public void animalSound();
}
```

```
class Cat implements Animal {
  public void animalSound() {
    // The body of animalSound() is provided here
    System.out.println("The Cat says: meow");
  }
```

Extending Interfaces

```
interface name2 extends name1
{
  body of name2
}
```

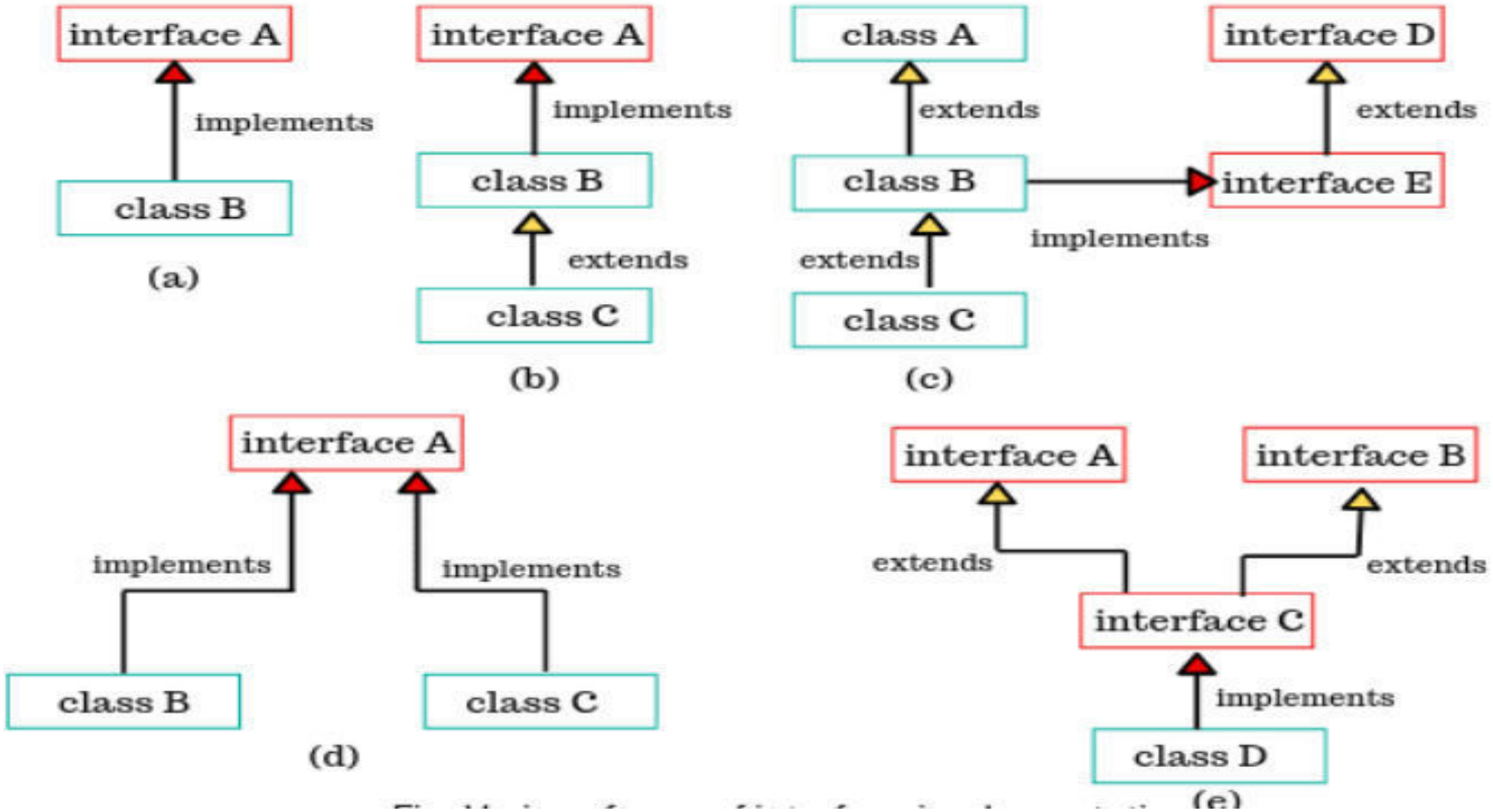
```
interface Const
{
        int age=35;
        String name="Supriya")
}
interface Methods extends Const
{
        void display();
}
```

Contd…

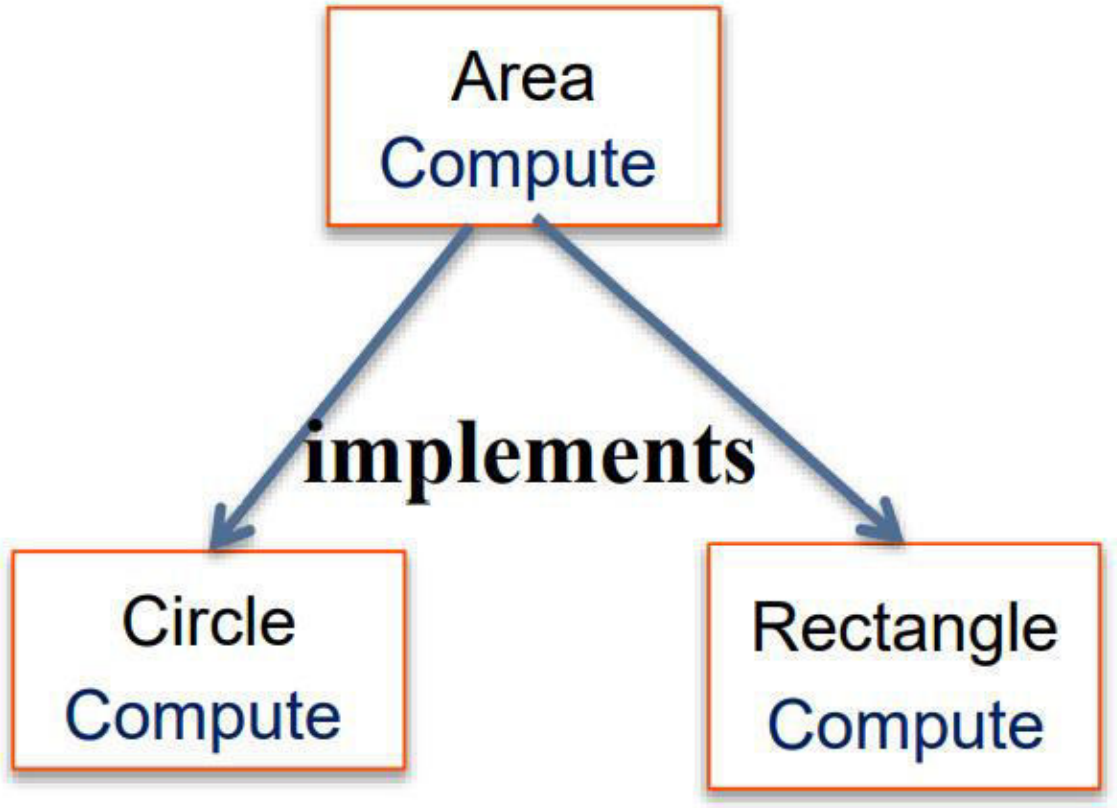
```
interface Const
{
        int age=35;
        String name="Supriya")
}
interface Methods
{
        void display();
}
```

```
interface combine extends Const, Methods
{
……….
……….
}
```

Different forms of Interfaces



Example: implement using interfaces



Example: Implementing Interfaces

```
interface Area {
static final float pi=3.14f;
float compute (float x, float y); }

class Rectangle implements Area {
public float compute(float x,float y)
{
return(x*y);
}}
class Circle implements Area{
public float compute(float x, float y)
{
return(pi*x*x);
}
}
```

```
class ICT{
public static void main(String args[]) {
Rectangle rect=new Rectangle();
Circle c=new Circle();
Area area;
area=rect;
System.out.println("Area of rectangle
="+area.compute(10,20));

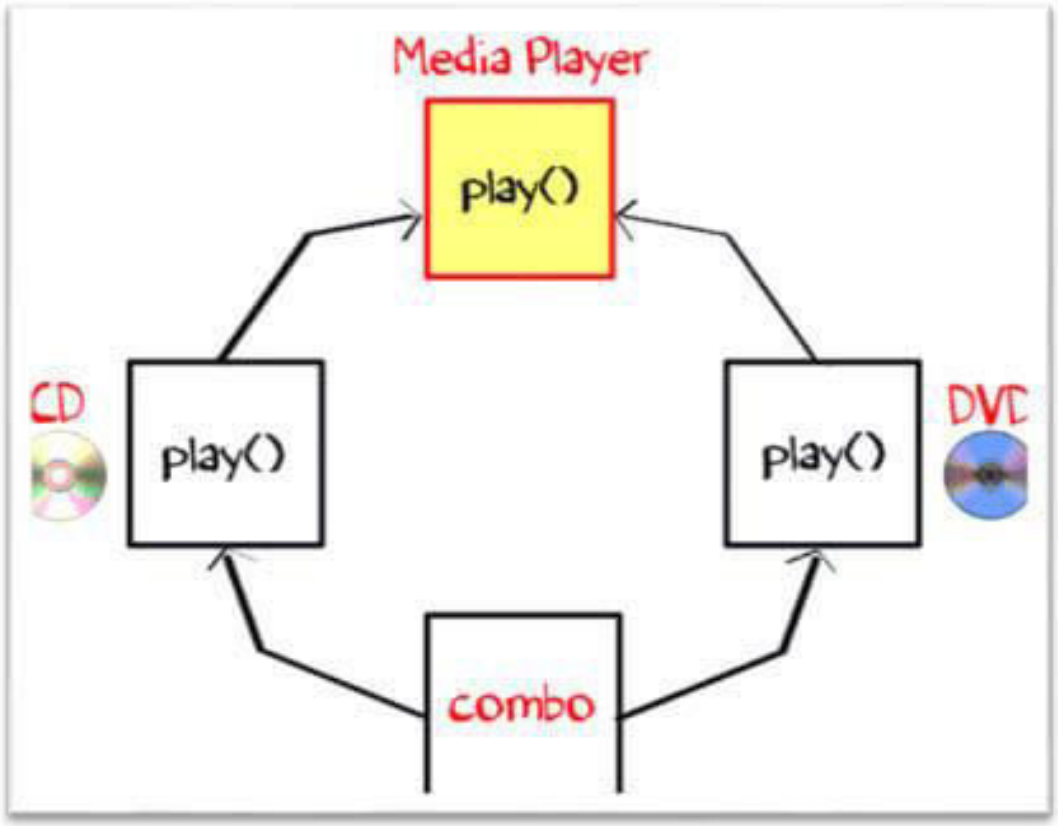
area=c;
System.out.println("Area of Circle
="+area.compute(10,0));
}}
```

```
C:\Users\supri\OneDrive\Documents>javac ICT.java

C:\Users\supri\OneDrive\Documents>java ICT
Area of rectangle =200.0
Area of Circle =314.0
```

Why And When To Use Interfaces?

- To achieve security - hide certain details and only show the important details of an object (interface).
- Java does not support "multiple inheritance" (a class can only inherit from one superclass).
- it can be achieved with interfaces, because the class can **implement** multiple interfaces.



Summary

- Importance of Interfaces
- Different forms of interfaces
- Diamond problem in multiple inheritance
- Solution to Diamond problem

Module-2

Packages

Faculty: Dr Supriya Raheja

Packages



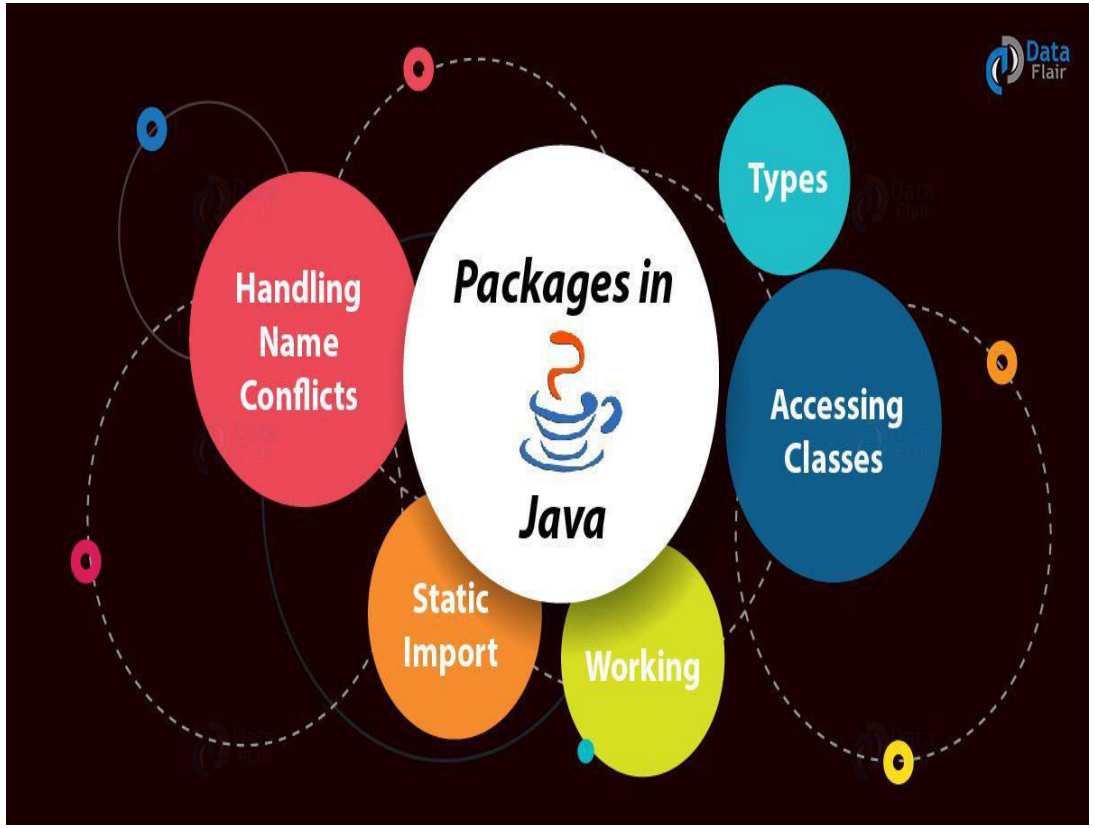
A Package is a logical organization of related classes. The java classes can be logically organized into packages.

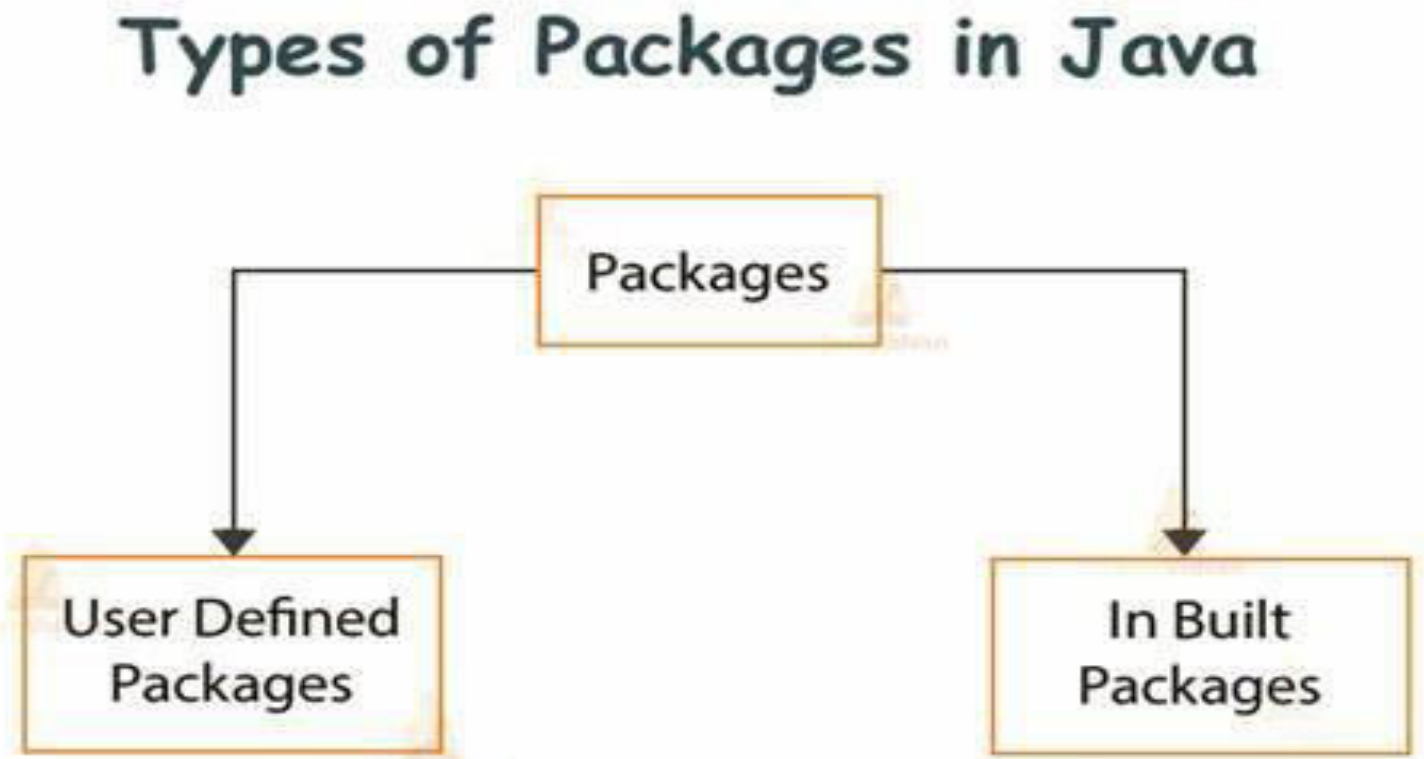


Packages acts as “containers” for classes, interfaces or subpackages.

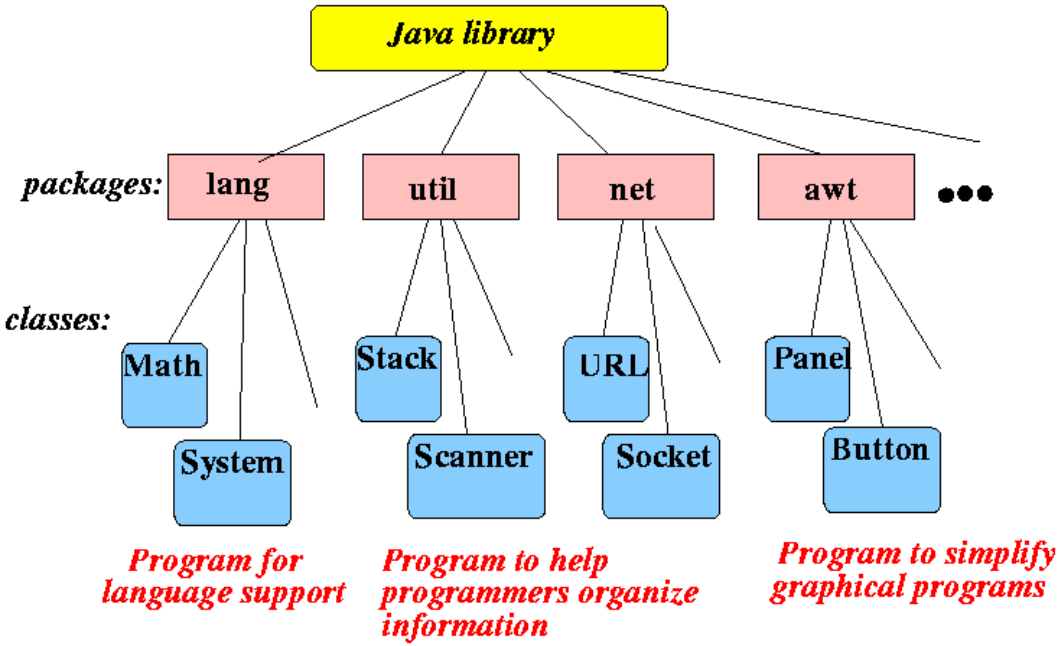
Benefits of using Packages

- Helps to reuse code
- Handles name conflicts
- Provides a way to hide classes
- Separates design from coding




Types of Packages in Java
Packages
User Defined Packages
In Built Packages

Java API Packages



java.lang: Contains language support classes(e.g classed which defines primitive data types, math operations). This package is automatically imported.

java.io: Contains classed for supporting input / output operations.

java.util: Contains utility classes like vectors, hash tables, random numbers etc.

java.applet: Contains classes for creating Applets.

java.awt: Contain classes for implementing the components for graphical user interfaces.

java.net: Contain classes for supporting networking operations.

Importing Package

//Java is package and util is subpackage

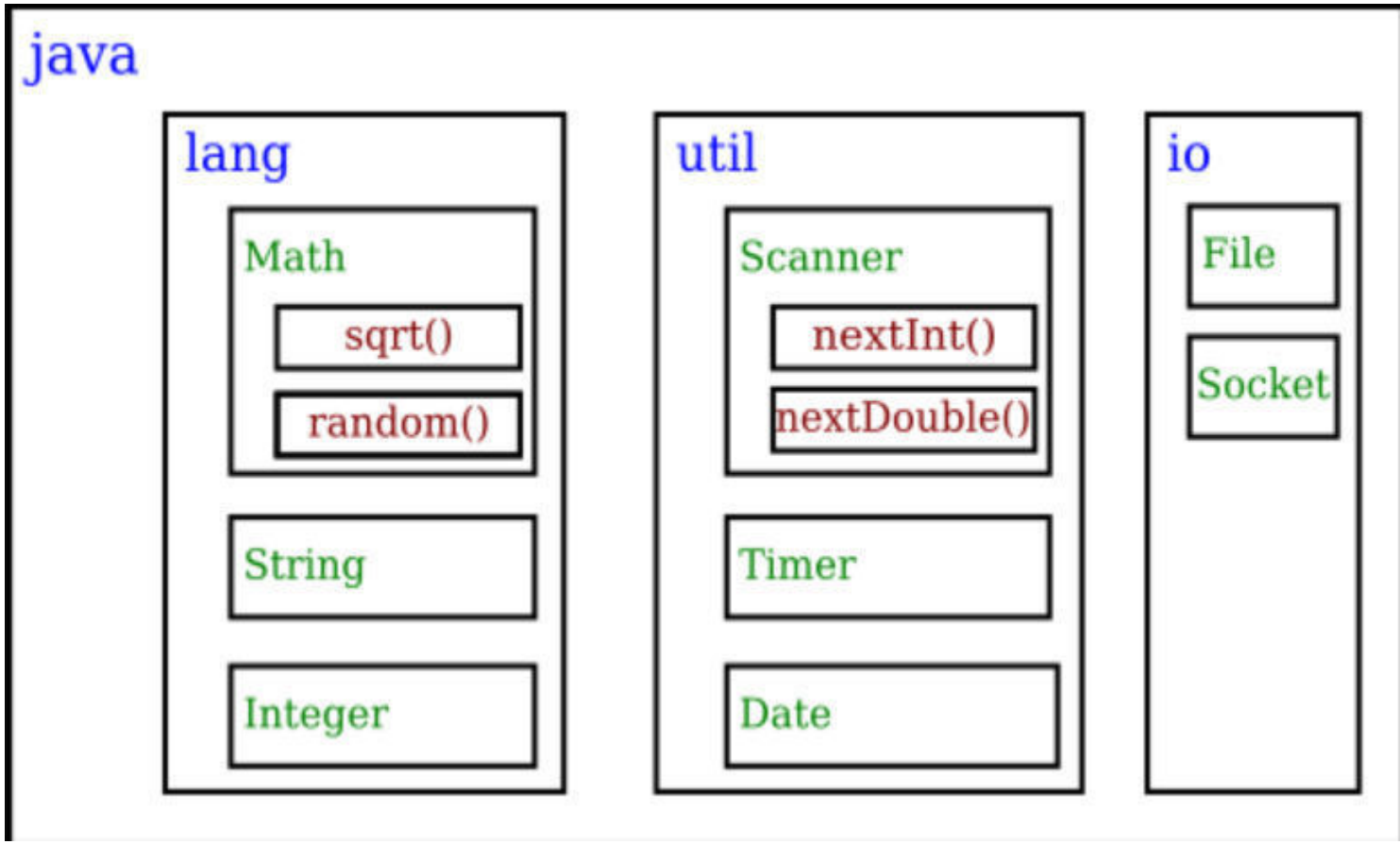
import java.util.*

//import the Vector class from util package

import java.util.Scanner;

// import all the classes from util package

import java.util.*;



User Defined Packages



JAVA uses file system to manage packages, with each package stored in its own directory.



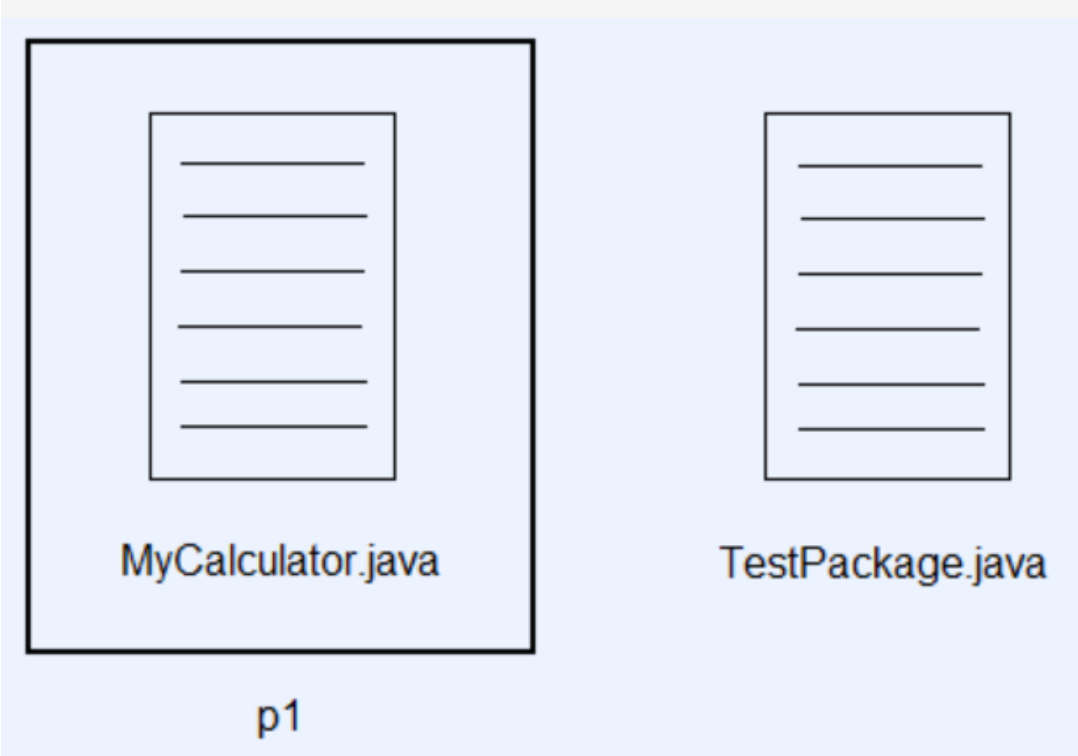
Package names are case sensitive. The directory name and the package name should be the same.



The classes declared within the package directory will belong to the specified package.

Steps to create package

1. Create a package(folder) named **p1.**
2. Create a file named **MyCalculator.java** inside **p1** package.
3. Write the code in **MyCalculator.java** file.
4. Create a file named **TestPackage.java** outside **p1** package.
5. Write the following code in **TestPackage.java** file.
6. Compile and Run **TestPackage.java** and get the output.



```
//MyCalculator.java
    package p1;
    public class MyCalculator{
        public int Add(int x,int y)
        {
            return x+y;
        }
       public int Subtract(int x,int y)
        {
            return x-y;
        }
        public int Product(int x,int y)
        {
            return x*y;
        } }
```

```
//TestPackage.java
    import p1.MyCalculator;
    class TestPackage{
        public static void main(String[] args)
        {
         MyCalculator M = new MyCalculator();
         System.out.print("\n\n\tThe Sum is : " +
M.Add(45,10));
          System.out.print("\n\n\tThe Subtract is :" +
M.Subtract(45,10));
          System.out.print("\n\n\tThe Product is :" +
M.Product(45,10));
        }
    }
```

Multiple Classes in a Package

```
//Csum.java
package calc;
public class Csum{
public int sum(int x, int y)
{
return (x+y);
}
}
```

```
//Csub.java
package calc;

public class Csub{

public int sub(int x, int y)
{
return (x-y);
}

}
```

```
import calc.*;
class MyCalculator{
public static void main(String args[])
{
Csum obj=new Csum();
Csub obj2=new Csub();

System.out.println("The sum is : "+
obj.sum(10,20));

System.out.println("The difference is :
" +obj2.sub(30,20));
}
}
```

Amity School of Engineering & Technology

Dept. of CSE

B.Tech(CSE), Sem IV

Java Programming (IT201)

Dr. Jyoti Agarwal

Course Objective

- Imparting java programming skill to students
- knowledge of object-oriented paradigm in context of Java programming language
- To learn about handling the exceptions

Table of Contents

- Exception Handling
- Exception class Hierarchy
- Java Exception Keywords
- throw keyword
- Finally block
- Difference between throw/throws

Exception Handling

- An Exception is an unwanted event that interrupts the normal flow of the program.
- When an exception occurs program execution gets terminated.
- Java exception is an object that describes an exceptional condition that has occurred in a piece of code.
- When an exception condition arises, an object representing that exception is created and thrown in the method that caused the error.

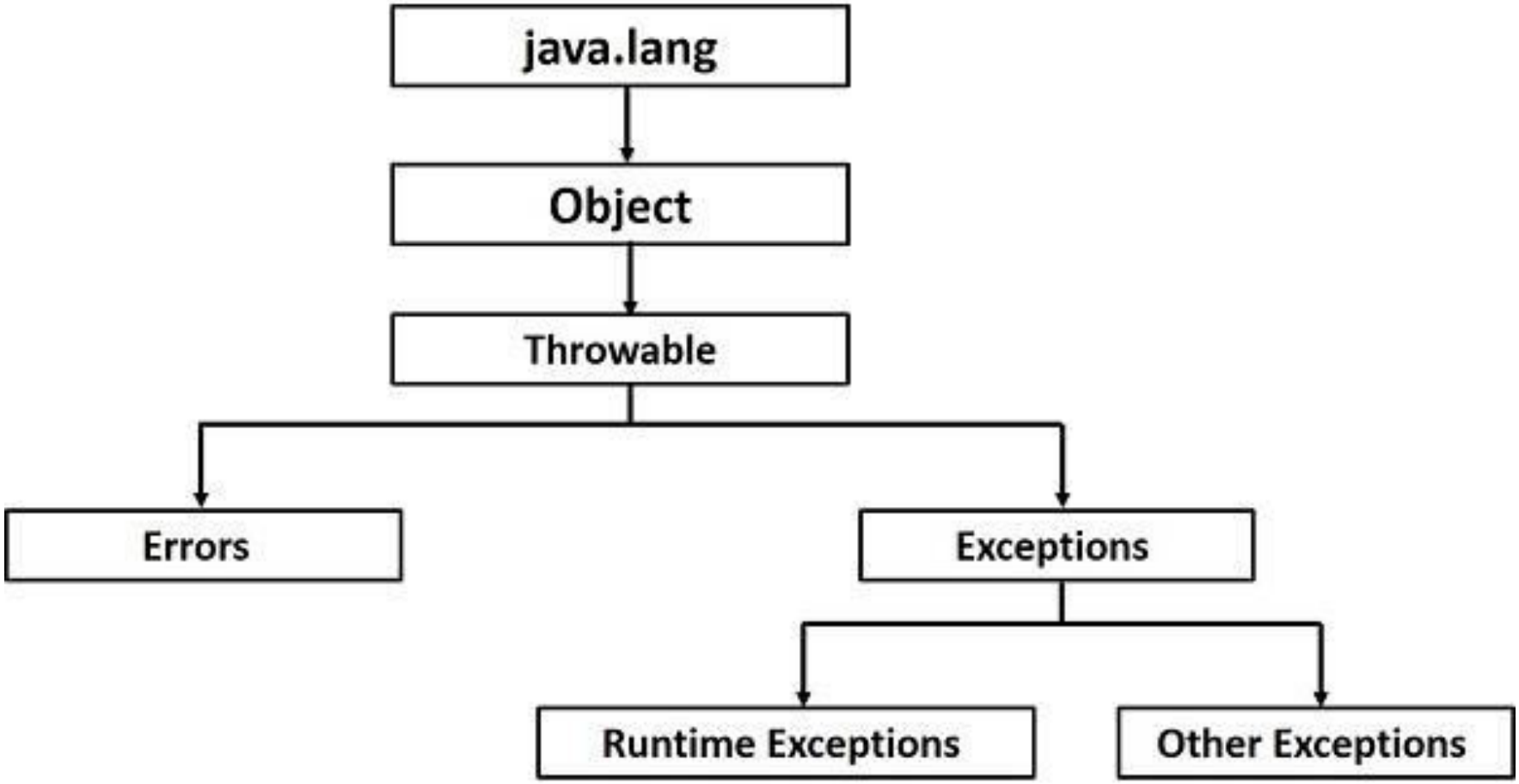
Exception Handling

- That method may choose to handle the exception itself or pass it on.
- Handled by five keywords: try, catch, throw, throws and finally.
- If we do not supply any exception handler then default exception handler provided by java caught the exception.
- Handling the exception by the programmer itself has two advantages:
- allows to fix the error.
- prevents the program from automatically terminating.

Exception Handling



Exception class Hierarchy



Exception class Hierarchy

Checked exceptions

- All exceptions other than Runtime Exceptions are known as Checked exceptions as the compiler checks them during compilation.

- If these exceptions are not handled/declared in the program, we will get compilation error.

- For example:

SQLException

IOException

ClassNotFoundException etc.

Exception class Hierarchy

Unchecked exceptions

- Runtime Exceptions are also known as Unchecked Exceptions.

- These exceptions are not checked at compile-time but it's the responsibility of the programmer to handle these exceptions.

- For example:

ArithmeticException

NullPointerException

ArrayIndexOutOfBoundsException etc.

Exception class Hierarchy

Error:

Error is irrecoverable.

Example:

OutOfMemoryError

VirtualMachineError

AssertionError .

Java Exception Keywords

try	The "try" keyword is used to specify a block where we should place exception code. The try block must be followed by either catch or finally. It means, we can't use try block alone.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the important code of the program. It is executed whether an exception is handled or not.
throw	used to throw an exception.
throws	used to declare exceptions. It doesn't throw an exception. It specifies that there may occur an exception in the method. It is always used with method signature.

Syntax


```
try
{
 // Block of code to try
}

 catch(Exception e)
{
 // Block of code to handle errors
}
```

Example


```
class B
{
 public static void main(String args[])
{
            int a=0;
                int d=5/a;
            System.out.println(d);
}
           System.out.println("Program terminated");
}
```

Handled by default exception handler.

Exception in thread "main" java.lang.ArithmeticException:

/ by zero

Example


```
class B
{
  public static void main(String args[])
{
            try
            {
                        int a=0;
                         int d=5/a;
                        System.out.println(This will not be printed”);
             }
           catch(ArithmeticException e)
        {
            System.out.println(e)
        }
            System.out.println(“After catch block”);
  }

}
```

O/P: Division by Zero

After catch block

Example


```
class B
 {
 public static void main(String args[]) {
     try {
       int a[ ] = new int[2];
       System.out.println("Access element three :" + a[3]);
     }
catch(ArrayIndexOutOfBoundsException e)
 {
       System.out.println("Exception thrown  :" + e);
     }
     System.out.println("Out of the block");
  }
}
```

O/P: Exception thrown
:java.lang.ArrayIndexOutOfBoundsExce
ption: Index 3 out of bounds for length 2
Out of the block

Multiple Catch block


```
class B
{
 public static void main(String args[])
{
      try
{
      int a=Integer.parseInt(args[0]);
      int d=5/a;
     int ar[] = new int[2];
     System.out.println("Access element three :" +
     ar[3]);
}
catch(ArithmeticException e)
{
   System.out.println(e);
}
catch(ArrayIndexOutOfBoundsException
e1)
 {
   System.out.println("Exception thrown
:" + e1);
    }

System.out.println("After catch block");

}
}
```

Multiple Catch block

- In multiple catch block, exception subclass must come before any of their superclass.

```
class E
{

public static void main(String args[])
{

  try
{
  int a=0;
  int b=12/a;
System.out.println(b);
}
catch(Exception e)
{
 System.out.println("Generic
Exception");
}
catch(ArithmeticException e1)
{
  System.out.println("no divided by
zero");
}
}}
```

compile time error: exception ArithmeticException has already been caught.

As all the exception will be caught by Exception class, which is superclass of ArithmeticException class. So we have to reverse the order of both catch statements.

throw keyword

- The throw statement allows to create a custom error.
- The throw statement is used together with an exception type like ArithmeticException, FileNotFoundException, ArrayIndex OutOfBoundsException, SecurityException, etc.
- Syntax:

```
throw exception;

throw new IOException("Error");
```

Example


```
class B
{
  void checkAge(int age)
{
   if (age < 18)
{
    throw new ArithmeticException("Can not give vote – age should be >=18);
   }
   else
 {
    System.out.println("Access granted - You are old enough!");
   }
 }

 public static void main(String[] args)
{
    B obj=new B();
    obj.checkAge(15);
}
}
```

Output:
Exception in thread "main" java.lang.ArithmeticException: Can not give vote – age should be >=18

Throw NullPointerException

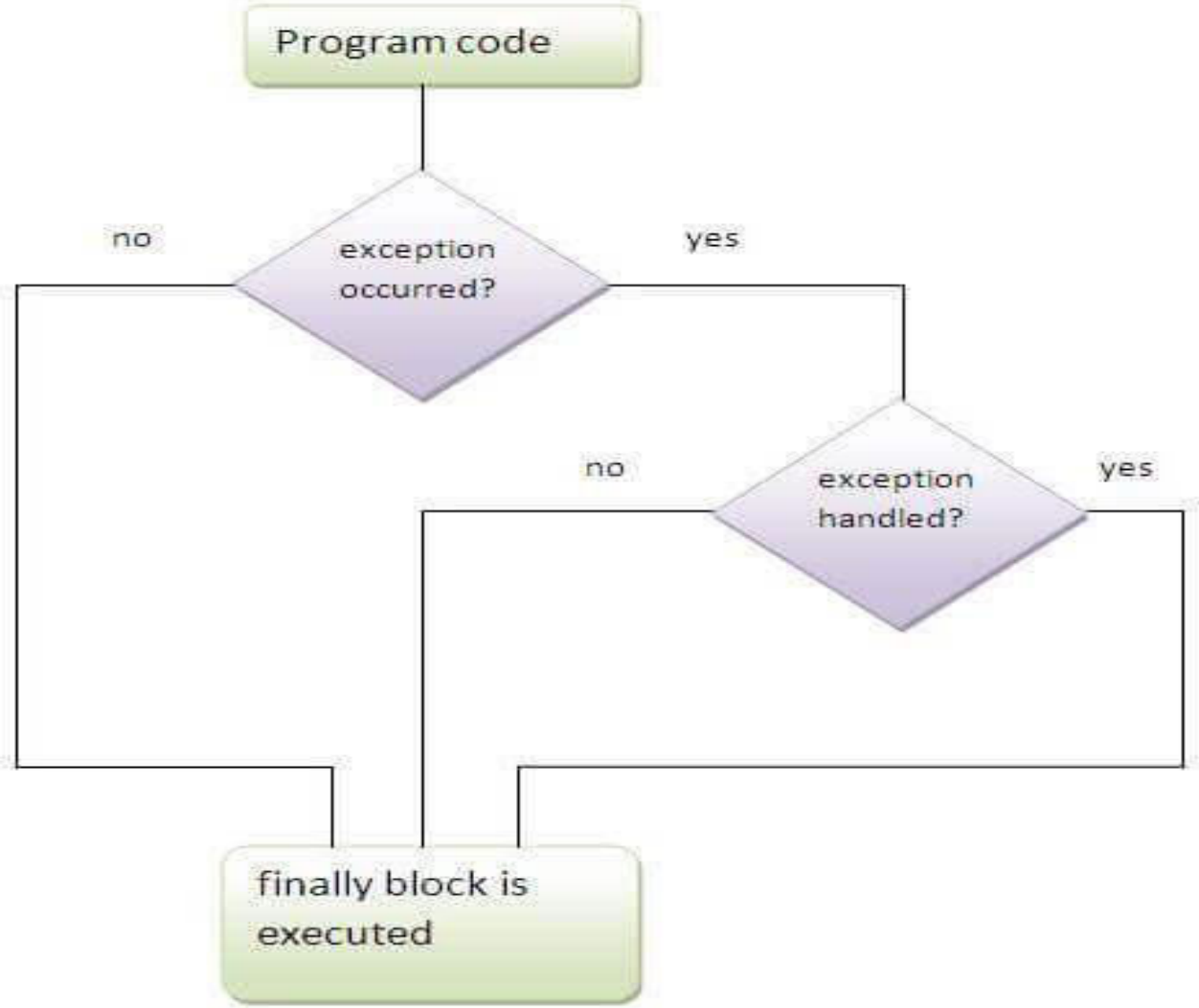

```
class Check1
{
  public static void main(String args[])
{
   Check1 a=new Check1();
    a=null;
    if(a==null)
        throw new NullPointerException("Null");
    else
        System.out.println("Hello");
}
}
```

O/P: Exception in thread "main"
java.lang.NullPointerException: Null

Finally block

- The finally block follows a try block or a catch block.
- A finally block of code always executes, irrespective of occurrence of an exception.
- appears at the end of catch block.

Finally block



Example


```
public class ExcepTest
{
   public static void main(String args[])
 {
     int a[] = new int[2];
     try
{
 System.out.println("Access element three :" +
a[3]);
     }
 catch (ArrayIndexOutOfBoundsException e)
{
       System.out.println("Exception thrown  :" + e);
    }
finally
 {
        a[0] = 6;
System.out.println("First element
value: " + a[0]);
System.out.println("The finally
statement is executed");
     }
  }
}
```

O/P: Exception thrown :java.lang.ArrayIndexOutOfBoundsException: 3
First element value: 6 The finally statement is executed

Case 1: where exception doesn't occur


```
class Test
{
 public static void main(String args[])
{
 try
{
  int data=25/5;
  System.out.println(data);
 }
 catch(NullPointerException e)
{
System.out.println(e);
}
 finally{System.out.println("finally block is always executed");}
 System.out.println(“End of Code");
 }
}
```

Output:
5
Finally block is always executed
End of Code

Case 2: where exception occurs and not handled


```
class Test
{
 public static void main(String args[])
{
 try
{
  int data=25/0;
  System.out.println(data);
 }
 catch(NullPointerException e)
{
System.out.println(e);
}
 finally
{
System.out.println("finally block is always executed");
}
 System.out.println(“End of Code");  }
}
```

finally block is always executed Exception in thread "main" java.lang.ArithmeticException: / by zero

Case 3: where exception occurs and handled


```
public class TestFinallyBlock2{
  public static void main(String args[])
{
  try{
   int data=25/0;
   System.out.println(data);
  }
  catch(ArithmeticException e)
{
System.out.println(e);
}
finally
{
System.out.println("finally block is always
 executed");
}
  System.out.println(“End of Code”);
  }
}
```

java.lang.ArithmeticException: / by zero
finally block is always executed
End of code.

Situations When finally block does not execute

- The death of a thread.
- Using of the System. exit() method.
- Due to an exception arising in the finally block.

Diff between throw and throws

throw	throws
Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
Throw is followed by an instance. throw new ArithmeticException("Arithmetic Exception");	Throws is followed by class. throws ArithmeticException;
Throw is used within the method.	Throws is used with the method signature.
You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method()throws IOException,SQLException

References

- Java: The Complete Reference , Seventh Edition by Herbert Schildt

- https://www.w3schools.com

- https://www.javatpoint.com

Amity School of Engineering & Technology

B.Tech CSE, Semester 5

Java Threads (Unit 3)

Objectives

After completing this section, students will be able to

- Understand Threads in Java and their use
- Create multithreaded programs
- Use synchronized methods or blocks to synchronize threads

AMITY UNIVERSITY

Contents

1. What is a Thread ?
2. Creating, Implementing and Extending a Thread
3. The life-cycle of a Thread
4. Interrupt a Thread
5. Thread synchronization

What is a Thread ?

- A sequential (or single-threaded) program is one that, when executed, has only one single flow of control.
  - i.e., at any time instant, there is at most only one instruction (or statement or execution point) that is being executed in the program.

What is a Thread ?

- A multi-threaded program is one that can have multiple flows of control when executed.
  - At some instance, there may exist multiple instructions or execution points that are being executed in the program

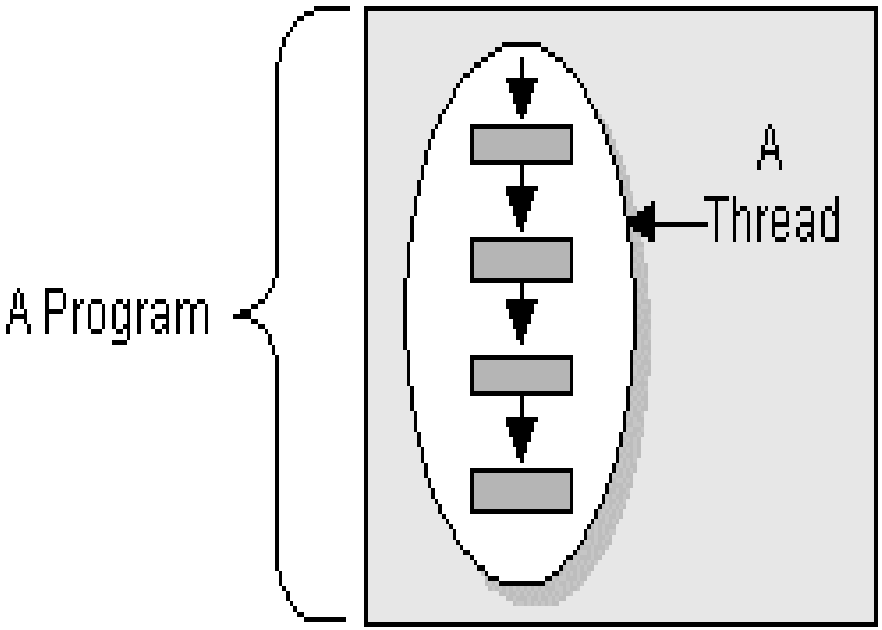
What is a Thread ?

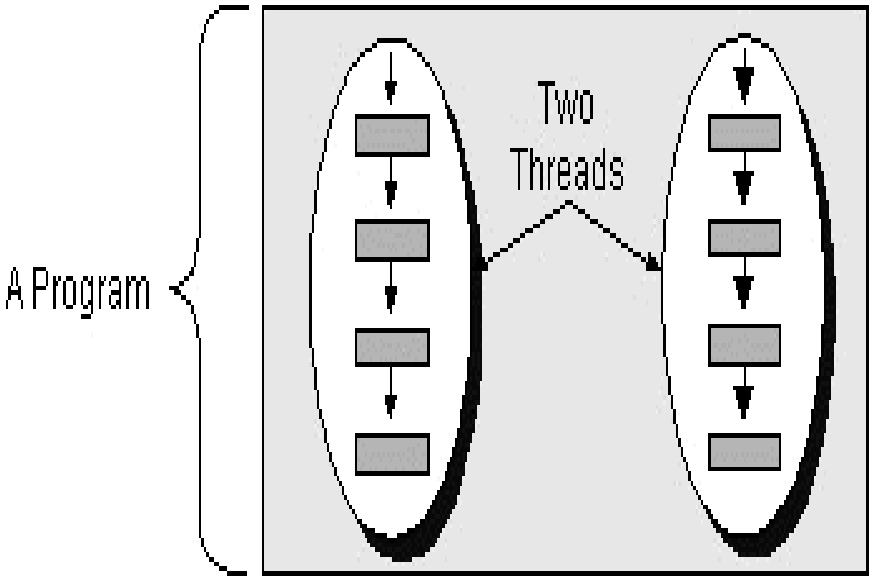
- Ex: in a Web browser we may do the following tasks at the same time:
  - 1. scroll a page,
  - 2. download an applet or image,
  - 3. play sound,
  - 4 print a page

- A thread is a single sequential flow of control within a program.

Thread vs Program?

Single-Threaded vs Multithreaded Programs

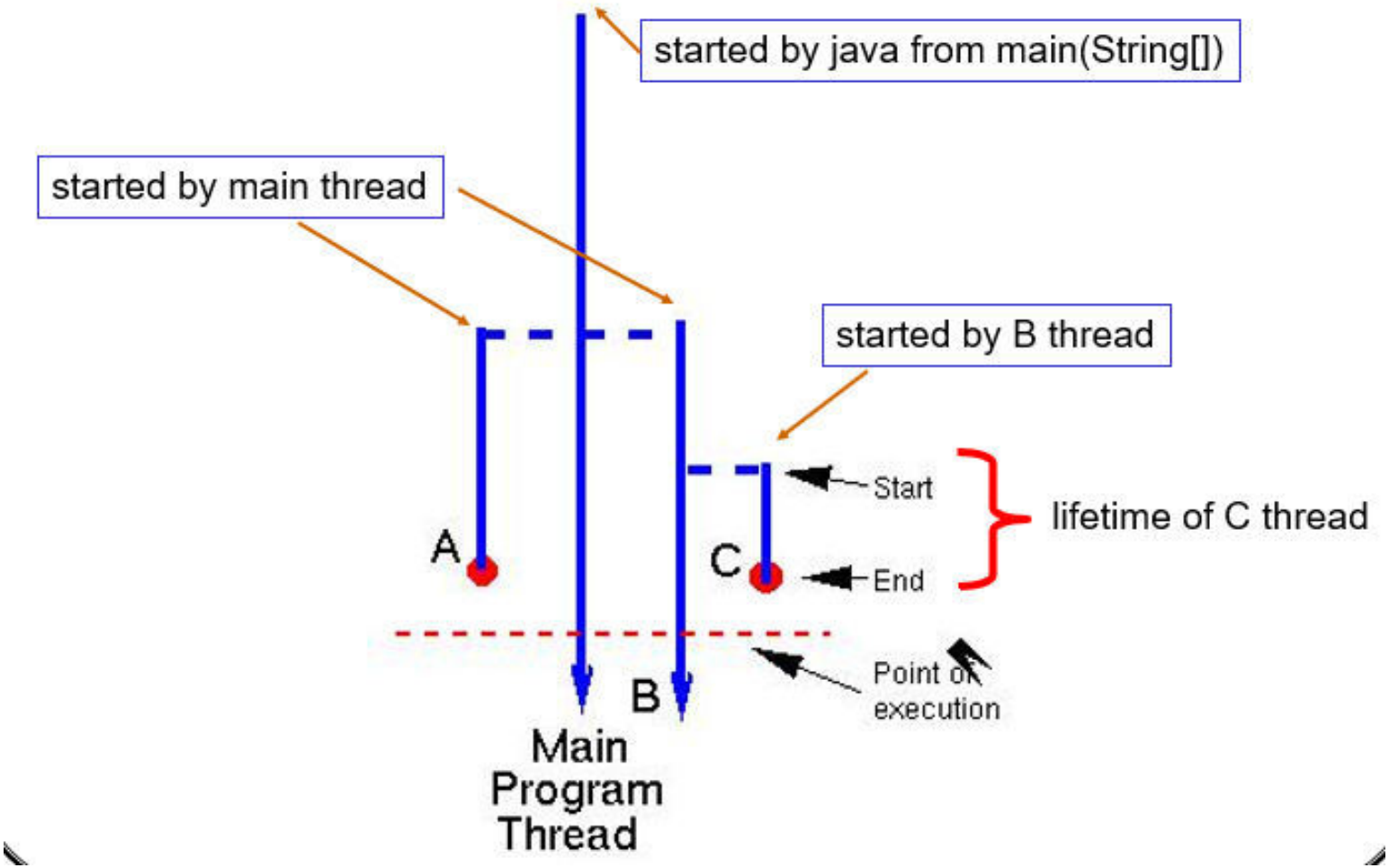





```
{ A(); A1();  A2();   A3();
  B1();  B2();  }
```

```
{  A();
   newThreads {
      { A1(); A2(); A3() };
      {B1(); B2() }
   }
}
```

Thread Ecology in a Java Program



Can we check Threads in Microsoft Word ?

Define and Launch a Java Thread

- Each Java Run time thread is encapsulated in a java.lang.Thread instance.

- Two ways to define a thread:
  1. Extend the Thread class
  2. Implement the Runnable interface :

Define and Launch a Java Thread

Steps for extending the Thread class:

1. Subclass the Thread class;

2. Override the default Thread method run(), which is the entry point of the thread, like the main(String[]) method in a java program.

Define and Launch a Java Thread

Implement the Runnable interface :

```
package java.lang;
public interface Runnable {  public void
run() ; }
```

Define a Thread


```
// Example:
public class Print2Console extends Thread {
    public void run() {   // run() is to a thread what
        main() is to a java program
       for (int b = -128; b < 128; b++)
        System.out.println(b);   }
   … // additional methods, fields …
     }
```

Define a Thread

- Impement the Runnable interface if you need a parent class:

```
public class Print2GUI implements Runnable {
    public void run() {
        for (int b = -128; b < 128; b++)
         System.out.println(b);}
     }
```

How to Launch a Thread

1. Create an instance of [a subclass of] of Thread, say thread.

   Thread thread = new Print2Console()

2. call its start() method, thread.start();. // note: not call run() !!

How to Launch a Thread

Ex:

- Printer2Console t1 = new Print2Console(); // t1 is a thread instance !
- t1.start() ; // this will start a new thread, which begins its execution by calling t1.run()
- … // parent thread continue immediately here without waiting for the child thread to complete its execution. cf: t1.run();
- Print2GUI jtext = new Print2GUI();
- Thread t2 = new Thread(jtext);
- t2.start();
- …

An Example


```
public class SimpleThread extends Thread {
        public SimpleThread(String str) { super(str);  }
        public void run() {
            for (int i = 0; i < 10; i++) {
                System.out.println(i + " " + getName());
                try { // at this point, current thread is 'this'.
                    Thread.sleep((long)(Math.random() * 1000));
                } catch (InterruptedException e) { }
            }
            System.out.println("DONE! " + getName());
        }       }
```

Main Program


```
public class TwoThreadsTest {
    public static void main (String[] args) {
    new SimpleThread(“Thread1").start();
    new SimpleThread(“Thread2").start();
} }
```

Output??

Main Program

0 Thread1
0 Thread2
1 Thread2
1 Thread1
2 Thread1
2 Thread2
3 Thread2
3 Thread1
4 Thread1
4 Thread2
5 Thread1
5 Thread2
6 Thread2
6 Thread1
7 Thread1
7 Thread2
8 Thread2
9 Thread2
8 Thread1
DONE! Thread2
9 Thread1
DONE! Thread1

Life Cycle of a Java Thread

New → (Runnable → blocked/waiting) * → Runnable → dead(terminated)

Life Cycle of a Java Thread

- sleep(long ms [,int ns]) // sleep (ms + ns x 10^{-3}) milliseconds and then continue

- [IO] blocked by synchronized method/block
  - synchronized(obj) { … } // synchronized statement
  - synchronized m(…) { … } // synchronized method
  - // return to runnable if IO complete

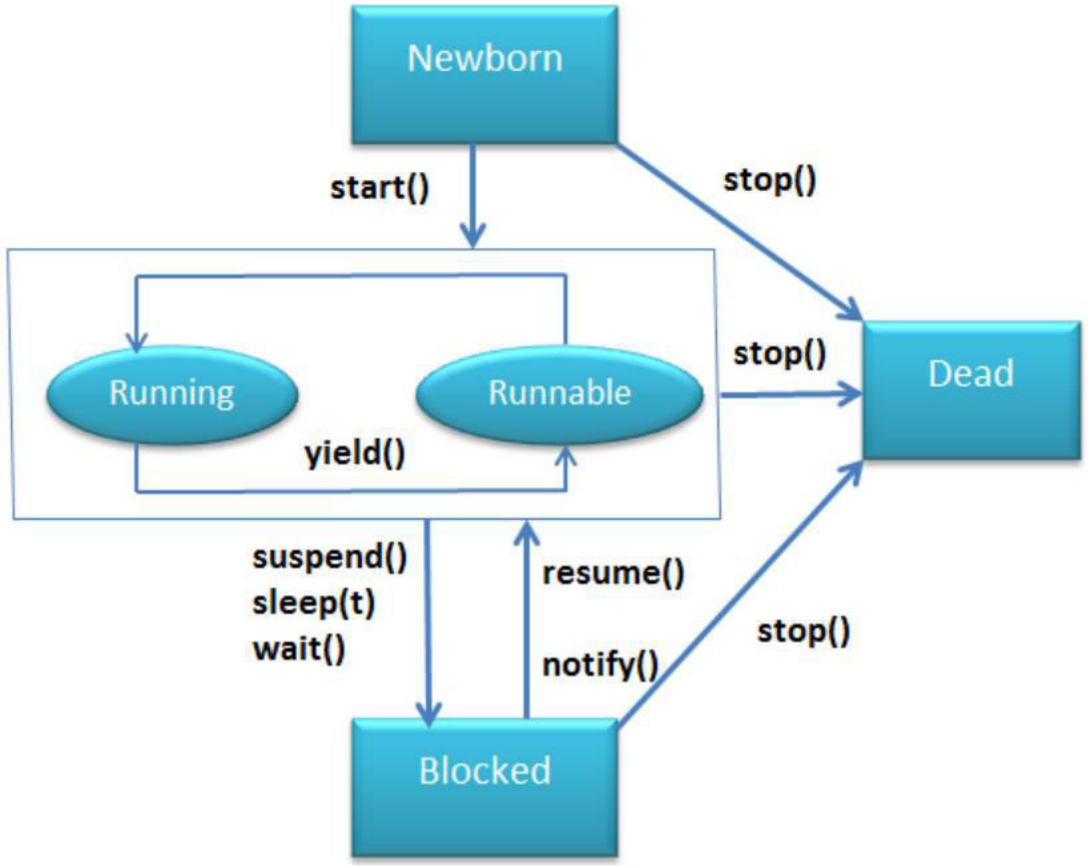
Life Cycle of a Java Thread

- obj.wait() // retrun to runnable by obj.notify() or obj.notifyAll()

- join(long ms [,int ns]) // Waits at most ms milliseconds plus ns nanoseconds for this thread to die.

The States (life cycle) of a Thread


```
public class Thread { .. // enum in 1.5  is a special class for finite
    type.
 public static enum State { //use Thread.State for referring to this
    nested class
  NEW,   //   after new Thread(), but before start().
  RUNNABLE, // after start(), when running or ready
  BLOCKED,   //  blocked by monitor lock
                          // blocked by a synchronized method/block
  WAITING,     // waiting for to be notified; no time out set
     // wait(), join()
 TIMED_WAITING, // waiting for to be notified; time out set
    // sleep(time), wait(time), join(time)
TERMINATED  // complete execution or after stop()
} …}
```


Newborn
start()
stop()
Running
Runnable
yield()
stop()
Dead
suspend()
sleep(t)
wait()
resume()
notify()
stop()
Blocked

Thread Synchronization

- Problem with any multithreaded Java program :
  - Two or more Thread objects access the same pieces of data.
- too little or no synchronization ==> there is inconsistency, loss or corruption of data.
- too much synchronization ==> deadlock or system frozen.
- In between there is unfair processing where several threads can starve another one hogging all resources between themselves

Can Multithreading incur inconsistency?

Multithreading may incur inconsistency : an Example

Two concurrent deposits of 50 into an account with 0 initial balance.:

```
void deposit(int amount) {
   int x = account.getBalance();
   x += amount;
   account.setBalance(x);  }
```

Multithreading may incur inconsistency : an Example

- deposit(50) : // deposit 1

```
x = account.getBalance()  //1
x += 50; //2
account.setBalance(x) //3
```

- deposit(50) : // deposit 2

```
x = account.getBalance()  //4
x += 50; //5
account.setBalance(x) //6
```

The execution sequence:
1,4,2,5,3,6 will result in unwanted result !!
Final balance is 50 instead of 100!!

Synchronized Methods and Statements

- Multithreading can lead to racing hazards where different orders of interleaving produce different results of computation.
  - Order of interleaving is generally unpredictable and is not determined by the programmer.

Synchronized Methods and Statements

- Java's synchronized method (as well as synchronized statement) can prevent its body from being interleaved by relevant methods.
  - synchronized(obj) { … } // synchronized statement with obj as lock
  - synchronized … m(…) {… } //synchronized method with this as lock
  - When one thread executes (the body of) a synchronized method/statement, all other threads are excluded from executing any synchronized method with the same object as lock.

Synchronizing Threads

- Java use the monitor concept to achieve mutual exclusion and synchronization between threads.
- Synchronized methods /statements guarantee mutual exclusion.
  - Mutual exclusion may cause a thread to be unable to complete its task. So monitor allow a thread to wait until state change and then continue its work.

Synchronizing Threads

- wait(), notify() and notifyAll() control the synchronization of threads.
  - Allow one thread to wait for a condition (logical state) and another to set it and then notify waiting threads.
  - condition variables => instance Boolean variables
  - wait => wait();
  - notifying => notify(); notifyAll();

Typical Use


```
synchronized void doWhenCondition() {
   while ( !condition )
     wait(); // wait until someone notifies us of
   changes in condition
   … // do what needs to be done when
   condition is true
}
```

Typical Use


```
synchronized void changeCondition {
   // change some values used in condition test
   notify(); // Let waiting threads know
    something changed
}
```

MODULE **IV**

ABSTRACT WINDOWING TOOLKIT (AWT)

By: **Dr. Ram Paul Hathwal**
Dept of CSE, ASET, AUUP

Objectives

- The objectives of this chapter are:
  - To discuss the classes present in the **java.awt** package
  - To understand the inheritance hierarchy of the AWT
  - To outline the basic structure of GUIs
  - To show how to add components to containers
  - To understand how to use Layout Managers
  - To understand basic graphics processing under the AWT

Vocabulary

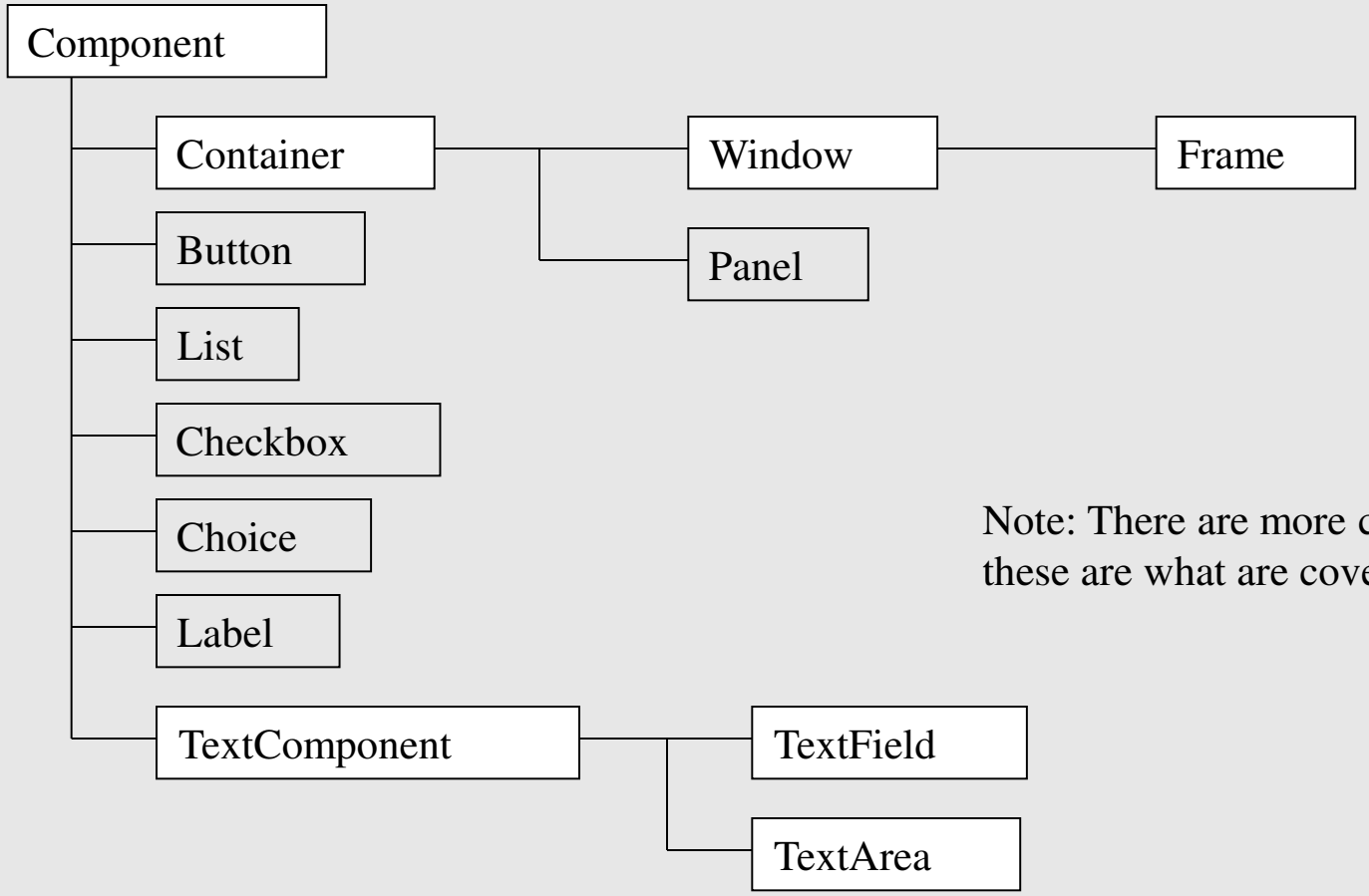
- AWT – The Abstract Window Toolkit provides basic graphics tools (tools for putting information on the screen)
- Swing – A much better set of graphics tools.
- Container – a graphic element that can hold other graphic elements (and is itself a Component)
- Component – a graphic element (such as a Button or a TextArea) provided by a graphics toolkit
- listener – A piece of code that is activated when a particular kind of event occurs
- layout manager – An object whose job it is to arrange Components in a Container.

Abstract Windowing Toolkit

- Present in all Java implementations.
- The AWT is roughly broken into three categories
  - Components
  - Layout Managers
  - Graphics

- Many AWT components have been replaced by Swing components.
- Uses the controls defined by your OS
  - therefore it's “least common denominator”
- It is generally not considered a good idea to mix Swing components and AWT components. Choose to use one or the other.

AWT – Class Hierarchy



Note: There are more classes, however, these are what are covered in this chapter

How to build a GUI...

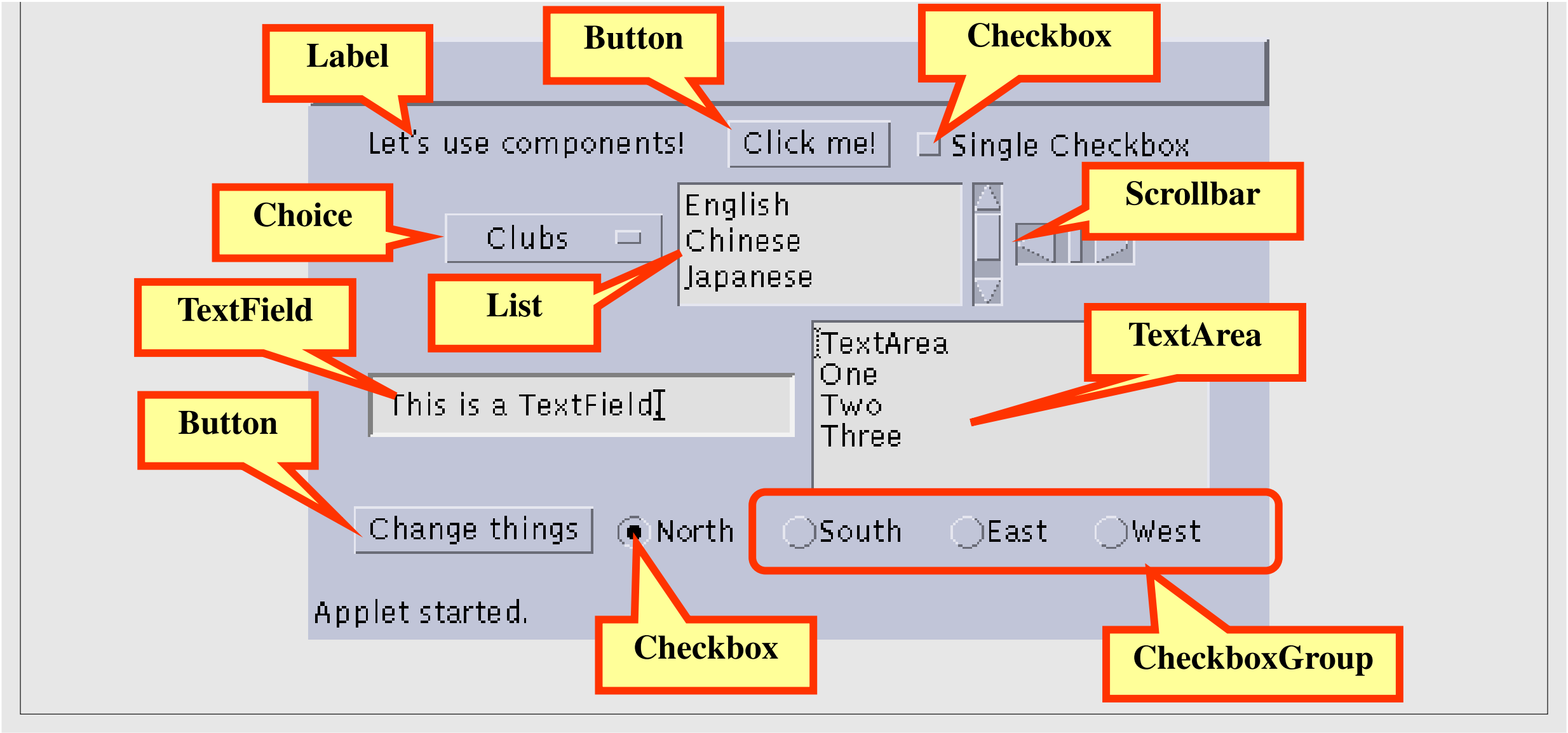
- Make somewhere to display things—usually a Frame or Dialog (for an application), or an Applet
- Create some Components, such as buttons, text areas, panels, etc.
- Add your Components to your display area.
- Arrange, or *lay out*, your Components.
- Attach Listeners to your Components.
  - Interacting with a Component causes an Event to occur
  - A Listener gets a message when an interesting event occurs, and executes some code to deal with it.
  - For each Listener you implement, supply the methods that it requires

Component

- Component is the superclass of most of the displayable classes defined within the AWT. **Note:** it is abstract.
- MenuComponent is another class which is similar to Component except it is the superclass for all GUI items which can be displayed within a drop-down menu.
- The Component class defines data and methods which are relevant to all Components:

```
setBounds
setSize
setLocation
setFont
setEnabled
setVisible
setForeground                -- colour
setBackground                -- colour
```

Some types of Components



Creating Components


```
Label lab = new Label ("Hi, Om!");

Button but = new Button ("Click me!");

Checkbox toggle = new Checkbox ("toggle");

TextField txt =
    new TextField ("Initial text.", 20);

Scrollbar scrolly = new Scrollbar(Scrollbar.HORIZONTAL, initialValue,
                                  bubbleSize, minValue, maxValue);
```

Container

- Container is a subclass of Component. (ie. All containers are themselves, Components)
- A Container is also a Component:
  - This allows Containers to be nested
- For a component to be placed on the screen, it must be placed within a Container
- The Container class defined all the data and methods necessary for managing groups of Components

  add
  getComponent
  getMaximumSize
  getMinimumSize
  getPreferredSize
  remove
  removeAll

Windows and Frames

- The Window class defines a top-level Window with no Borders or Menu bar.
  - Usually used for application splash screens
- Frame defines a top-level Window with Borders and a Menu Bar
  - Frames are more commonly used than Windows
- Once defined, a Frame is a Container which can contain Components

```
Frame aFrame = new Frame("Hello World");
aFrame.setSize(100,100);
aFrame.setLocation(10,10);
aFrame.setVisible(true);
```

Panels

- When writing a GUI application, the GUI portion can become quite complex.
- To manage the complexity, GUIs are broken down into groups of components. Each group generally provides a unit of functionality.
- A Panel is a rectangular Container whose sole purpose is to hold and manage components within a GUI.

```
Panel aPanel = new Panel();
aPanel.add(new Button("Ok"));
aPanel.add(new Button("Cancel"));

Frame aFrame = new Frame("Button Test");
aFrame.setSize(100,100);
aFrame.setLocation(10,10);

aFrame.add(aPanel);
```

Buttons

- This class represents a push-button which displays some specified text.
- When a button is pressed, it notifies its Listeners. (More about Listeners in the next chapter).
- To be a Listener for a button, an object must implement the ActionListener Interface.

```
Panel aPanel = new Panel();
Button okButton = new Button("Ok");
Button cancelButton = new Button("Cancel");

aPanel.add(okButton));
aPanel.add(cancelButton));

okButton.addActionListener(controller2);
cancelButton.addActionListener(controller1);
```

Labels

- This class is a Component which displays a single line of text.
- Labels are read-only. That is, the user cannot click on a label to edit the text it displays.
- Text can be aligned within the label

```
Label aLabel = new Label("Enter password:");
aLabel.setAlignment(Label.RIGHT);

aPanel.add(aLabel);
```

List

- This class is a Component which displays a list of Strings.
- The list is scrollable, if necessary.
- Sometimes called Listbox in other languages.
- Lists can be set up to allow single or multiple selections.
- The list will return an array indicating which Strings are selected

```
List aList = new List();
aList.add("Calgary");
aList.add("Edmonton");
aList.add("Regina");
aList.add("Vancouver");

aList.setMultipleMode(true);
```

Checkbox

- This class represents a GUI checkbox with a textual label.
- The Checkbox maintains a boolean state indicating whether it is checked or not.
- If a Checkbox is added to a CheckBoxGroup, it will behave like a radio button.

```
Checkbox creamCheckbox = new CheckBox("Cream");
Checkbox sugarCheckbox = new CheckBox("Sugar");
[]..
if (creamCheckbox.getState())
{
    coffee.addCream();
}
```

Choice

- This class represents a dropdown list of Strings.
- Similar to a list in terms of functionality, but displayed differently.
- Only one item from the list can be selected at one time and the currently selected element is displayed.

```
Choice aChoice = new Choice();
aChoice.add("Calgary");
aChoice.add("Edmonton");
aChoice.add("Alert Bay");
[]..

String selectedDestination= aChoice.getSelectedItem();
```

TextField

- This class displays a single line of optionally editable text.
- This class inherits several methods from TextComponent.
- This is one of the most commonly used Components in the AWT

```
TextField emailTextField = new TextField();
TextField passwordTextField = new TextField();
passwordTextField.setEchoChar("*");
[…]

String userEmail = emailTextField.getText();
String userpassword = passwordTextField.getText();
```

TextArea

- This class displays multiple lines of optionally editable text.
- This class inherits several methods from TextComponent.
- TextArea also provides the methods: appendText(), insertText() and replaceText()

```
// 5 rows, 80 columns
TextArea fullAddressTextArea = new TextArea(5, 80);
[]..

String userFullAddress= fullAddressTextArea.getText();
```

Layout Managers

- Since the Component class defines the setSize() and setLocation() methods, all Components can be sized and positioned with those methods.
- Problem: the parameters provided to those methods are defined in terms of pixels. Pixel sizes may be different (depending on the platform) so the use of those methods tends to produce GUIs which will not display properly on all platforms.
- Solution: Layout Managers. Layout managers are assigned to Containers. When a Component is added to a Container, its Layout Manager is consulted in order to determine the size and placement of the Component.
- **NOTE:** If you use a Layout Manager, you can no longer change the size and location of a Component through the setSize and setLocation methods.

Layout Managers (cont)

- There are several different LayoutManagers, each of which sizes and positions its Components based on an algorithm:
  - FlowLayout
  - BorderLayout
  - GridLayout
- For Windows and Frames, the default LayoutManager is BorderLayout. For Panels, the default LayoutManager is FlowLayout.

Flow Layout

- The algorithm used by the FlowLayout is to lay out Components like words on a page: Left to right, top to bottom.
- It fits as many Components into a given row before moving to the next row.

```
Panel aPanel = new Panel();
aPanel.add(new Button("Ok"));
aPanel.add(new Button("Add"));
aPanel.add(new Button("Delete"));
aPanel.add(new Button("Cancel"));
```

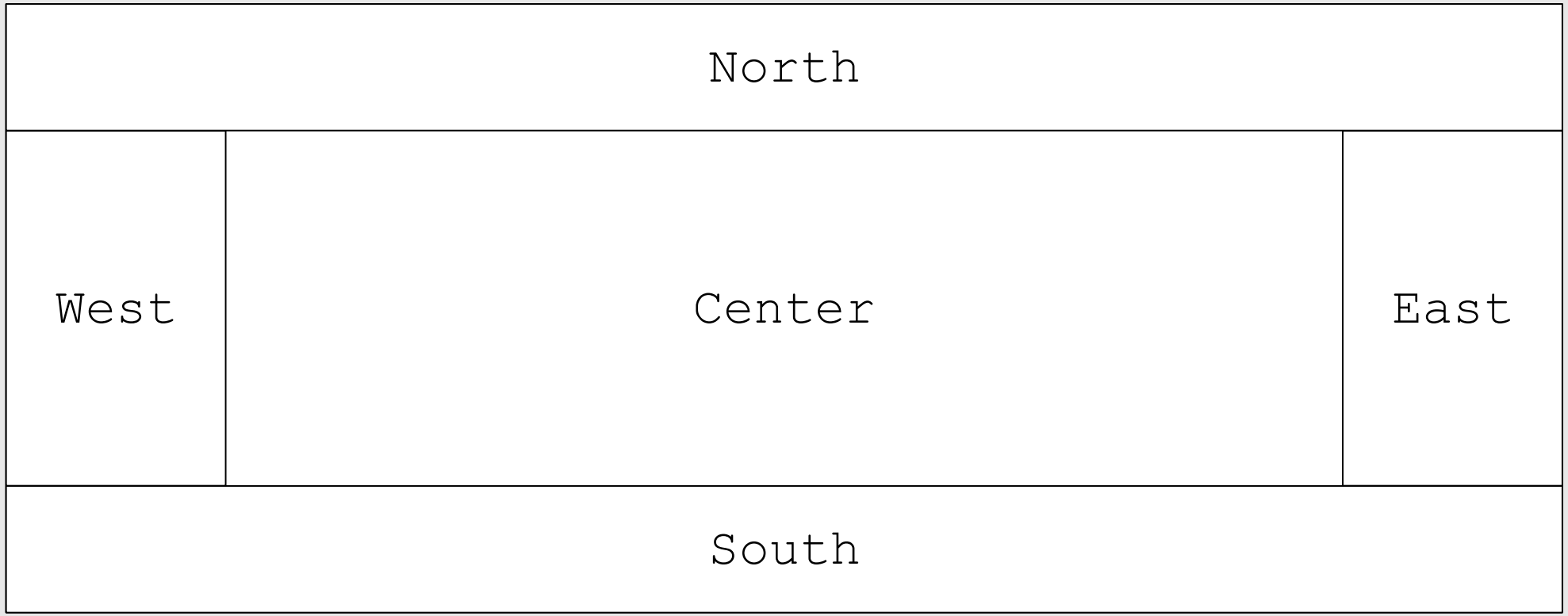
Border Layout

- The BorderLayout Manager breaks the Container up into 5 regions (North, South, East, West, and Center).
- When Components are added, their region is also specified:

```
Frame aFrame = new Frame();
aFrame.add("North", new Button("Ok"));
aFrame.add("South", new Button("Add"));
aFrame.add("East", new Button("Delete"));
aFrame.add("West", new Button("Cancel"));
aFrame.add("Center", new Button("Recalculate"));
```

Border Layout (cont)

- The regions of the BorderLayout are defined as follows:



Grid Layout

- The GridLayout class divides the region into a grid of equally sized rows and columns.
- Components are added left-to-right, top-to-bottom.
- The number of rows and columns is specified in the constructor for the LayoutManager.

```
Panel aPanel = new Panel();
GridLayout theLayout = new GridLayout(2,2);
aPanel.setLayout(theLayout);

aPanel.add(new Button("Ok"));
aPanel.add(new Button("Add"));
aPanel.add(new Button("Delete"));
aPanel.add(new Button("Cancel"));
```

What if I dont want a LayoutManager?

- LayoutManagers have proved to be difficult and frustrating to deal with.
- The LayoutManager can be removed from a Container by invoking its setLayout method with a null parameter.

```
Panel aPanel = new Panel();
aPanel.setLayout(null);
```

Graphics

- It is possible to draw lines and various shapes within a Panel under the AWT.
- Each Component contains a Graphics object which defines a Graphics Context which can be obtained by a call to getGraphics().
- Common methods used in Graphics include:

drawLine
drawOval
drawPolygon
drawPolyLine
drawRect
drawRoundRect
drawString
draw3DRect
fill3DRect
fillArc

- fillOval
- fillPolygon
- fillRect
- fillRoundRect
- setColor
- setFont
- setPaintMode
- drawImage

MODULE **IV**

EVENTS HANDLING

By: **Dr. Ram Paul Hathwal**
Dept of CSE, ASET, AUUP

Events Handling

- Any program that uses GUI (graphical user interface) such as Java application written for windows, is event driven.
- Event describes the change of state of any object.
- Events are generated as result of user interaction with the graphical user interface components.
- Changing the state of an object is known as an event.
- **For example**: clicking on a button, entering a character in Textbox, moving the mouse, selecting an item from list, scrolling the page, *etc*.
- The ***java.awt.event*** package provides many event classes and Listener interfaces for event handling.

Delegation Event Model

- The modern approach to handling events is based on the delegation event model.
- The delegation event model provides a standard mechanism for a source to generate an event and send it to a set of listeners.
- The listener simply waits until it receives an event.
- Once received, the listener processes the event and then return.
- In the delegation event model, listener must register with a source in order to receive an event notification.
- Notification are sent only to listeners that want to receive them.
- There are mainly three parts in delegation event model.
  - Events.
  - Event sources.
  - Event Listeners.

Components of Event Handling

- Event handling has three main components,
  - **Events**: An event is a change of state of an object.
  - **Events Source**: Event source is an object that generates an event.
  - **Listeners**: A listener is an object that listens to the event. A listener gets notified when an event occurs.

Events in Java

- An event is an object that describes a state change in a source.
- It can be generated as a consequence of a person interacting with the elements in a graphical user interface.
- Some of the activities that cause events to be generated are pressing a button, entering a character via the keyboard, selecting an item in a list and clicking the mouse.
- Events may also occur that are not directly caused by interactions with a user interface. We are free to define events that are appropriate for our application.

Event Sources

- A source is an object that generates an event. This occurs when the internal state of that object changes in some way.
- Sources may generate more than one type of event.
- A source must register listeners in order for the listeners to receive notifications about a specific type of event.
- Each type of event has its own registration method.
- Here is the general form:
  - public void addTypeListener(TypeListener el)
    - For example: b.addActionListener(this);
- Here, type is the name of the event, and el is a reference to the event listener.
- For example, the method that registers a keyboard event listener is called addKeyListener().

Conti…

- The method that registers a mouse motion listener is called addMouseMotionListener().
- When an event occurs, all registered listeners are notified and receive a copy of the event object. This is known as multicasting the event.
- In all cases, notifications are sent only to listeners that register to receive them.
- A source must also provide a method that allows a listener to unregister an interest in a specific type of event. The general form of such a method is this:
  - Public void removeTypeListener(TypeListener el)
- Here, type is an object that is notified when an event listener. For example, to remove a keyboard listener, you would call removeKeyListener()

Sources Generating Events

Button	Generates action events when the button is pressed.
Checkbox	Generates item events when the check box is selected or deselected.
Choice	Generates item events when the choice is changed.
List	Generates action events when an item is double-clicked; Generates item events when an item is selected or deselected.
Menu Item	Generates action events when a menu item is selected; generates item events when a checkable menu item is selected or deselected.
Scrollbar	Generates adjustment events when the scroll bar is manipulated.
Text Component	Generates text events when the user enters a character.
Window	Generates widow events when a window is activated, closed, deactivated, deiconified, iconified, opened or quit.

Event Listeners

- A listener is an object that is notified when an event occurs.
- It has two major requirements. First, it must have been registered with one or more sources to receive notifications about specific types of events.
- Second, it must implement methods to receive and process these notifications.
- The method that receive and process events are defined in a set of interfaces found in ***java.awt.event.***
- For example, the ***MouseMotionListener*** interface defines two methods to receive notifications when the mouse is dragged or moved.
- Any object may receive and process one or both of these events if it provides an implementation of this interface.

Important Event Classes and Interface

Event Classes	Description	Listener Interface
ActionEvent	generated when button is pressed, menu-item is selected, list-item is double clicked	ActionListener
MouseEvent	generated when mouse is dragged, moved, clicked, pressed or released and also when it enters or exit a component	MouseListener
KeyEvent	generated when input is received from keyboard	KeyListener
ItemEvent	generated when check-box or list item is clicked	ItemListener
TextEvent	generated when value of textarea or textfield is changed	TextListener
MouseWheelEvent	generated when mouse wheel is moved	MouseWheelListener
WindowEvent	generated when window is activated, deactivated, deiconified, iconified, opened or closed	WindowListener
ComponentEvent	generated when component is hidden, moved, resized or set visible	ComponentEventListener
ContainerEvent	generated when component is added or removed from container	ContainerListener
AdjustmentEvent	generated when scroll bar is manipulated	AdjustmentListener
FocusEvent	generated when component gains or loses keyboard focus	FocusListener

Events Handling steps

- Two Steps to handle events:
  - Implement appropriate interface in the class.
  - Register the component with the listener.

Registration Methods

- For registering the component with the Listener, many classes provide the registration methods.
- For example:

Button

```
public void addActionListener(ActionListener a){}
```

MenuItem

```
public void addActionListener(ActionListener a){}
```

TextField

```
public void addActionListener(ActionListener a){}
public void addTextListener(TextListener a){}
```

TextArea

```
public void addTextListener(TextListener a){}
```

Checkbox

```
public void addItemListener(ItemListener a){}
```

Choice

```
public void addItemListener(ItemListener a){}
```

List

```
public void addActionListener(ActionListener a){}
public void addItemListener(ItemListener a){}
```

ActionEvent

- An ActionEvent is generated when a button is pressed, a list item is double-clicked, or a menu item is selected.
- The ActionEvent class defines four integer constants that can be used to identify any modifiers associated with an action event:
  - public static final int ALT_MASK
    - The **alt** modifier. An indicator that the alt key was held down during the event. (8)
  - public static final int SHIFT_MASK
    - The **shift** modifier. An indicator that the shift key was held down during the event. (1)
  - public static final int CTRL_MASK
    - The **control** modifier. An indicator that the control key was held down during the event. (2)
  - public static final int META_MASK

Syntax Example

- PublicActionEvent(Object source, int id, String command, int modifiers)
  - Constructs an ActionEvent object with modifier keys.
  - Parameters: source - the object that originated the event
  - id - an integer that identifies the event
  - command - a string that may specify a command (possibly one of several) associated with the event
  - modifiers - the modifier keys held down during this action
- Example:

  public ActionEvent(Object source, int id, String command, long when, int modifiers)
  - Constructs an ActionEvent object with the specified modifier keys and timestamp.
  - **Parameters:**
    - **source** - the object that originated the event.
    - **id** - an integer that identifies the event.
    - **command** - a string that may specify a command (possibly one of several) associated with the event.
    - **when** - the time the event occurred.
    - **modifiers** - the modifier keys held down during this action.

Methods

- public String getActionCommand()
  - Returns the command string associated with this action.
- public long getWhen()
  - Returns the timestamp of when this event occurred.
- int getModifiers()
  - Returns the modifier keys held down during this action event.
- String paramString()
  - Returns a parameter string identifying this action event.

ActionListener Interface

- This interface defines the actionPerformed() method that is invoked when an action event occurs.
- Its general form is shown here:
  - void actionPerformed(ActionEvent ae)

Programming Example


```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;

/* <applet code="ActionEventExample" width=200 height=200>
</applet> */

public class ActionEventExample extends Applet implements ActionListener
{
    String actionMessage="";

      public void init()
      {
            Button Button1 = new Button("Ok");
            Button Button2 = new Button("Cancel");

            add(Button1);
            add(Button2);

            Button1.addActionListener(this); //Listener Registered
            Button2.addActionListener(this); //Listener Registered
       }
```

Conti…


```
public void paint(Graphics g)
      {
            g.drawString(actionMessage,10,50);
       }

public void actionPerformed(ActionEvent ae)
      {

            String action = ae.getActionCommand();

            if(action.equals("Ok"))
                  actionMessage = "Ok Button Pressed";
            else if(action.equals("Cancel"))
                  actionMessage = "Cancel Button Pressed";

            repaint();
      }
}
```

Output


Applet Viewer:...
Applet
Ok
Cancel
Ok Button Pressed
Applet started.

ComponentEvent class

- A low-level event which indicates that a component moved, changed size, or changed visibility.
- This class has following constants.
- public static final int COMPONENT_MOVED
  - This event indicates that the component's position changed.
- public static final int COMPONENT_RESIZED
  - This event indicates that the component's size changed.
- public static final int COMPONENT_SHOWN
  - This event indicates that the component was made visible.
- public static final int COMPONENT_HIDDEN
  - This event indicates that the component was become invisible.

Conti…

- public ComponentEvent(Component source, int id)
  - Constructs a ComponentEvent object.
  - Parameters:
  - source - the Component that originated the event
  - id - an integer indicating the type of event
- Component getComponent()
  - Returns the creator of the event.
  - the Component object that originated the event, or null if the object is not a Component.

ComponentListener interface

- The listener interface for receiving component events.
- void componentResized(ComponentEvent e)
  - Invoked when the component's size changes.
- void componentMoved(ComponentEvent e)
  - Invoked when the component's position changes
- void componentShown(ComponentEvent e)
  - Invoked when the component has been made visible.
- void componentHidden(ComponentEvent e)
  - Invoked when the component has been made invisible.

Programming Example


```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ComponentEventExample1
{
  public static void main(String[] args)
  {
     JFrame frame = new JFrame("ComponentEventExample");
     frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
     TextArea txtArea = new TextArea();
     Checkbox checkbox1 = new Checkbox("Checkbox 1");
     Checkbox checkbox2 = new Checkbox("Checkbox 2");
     frame.add(txtArea, BorderLayout.CENTER);
     frame.add(checkbox1, BorderLayout.NORTH);
     frame.add(checkbox2, BorderLayout.SOUTH);
     frame.setVisible(true);
     ComponentListener componentListener = new MyComponentListener();
     frame.addComponentListener(componentListener);
  }
}
```

Conti…

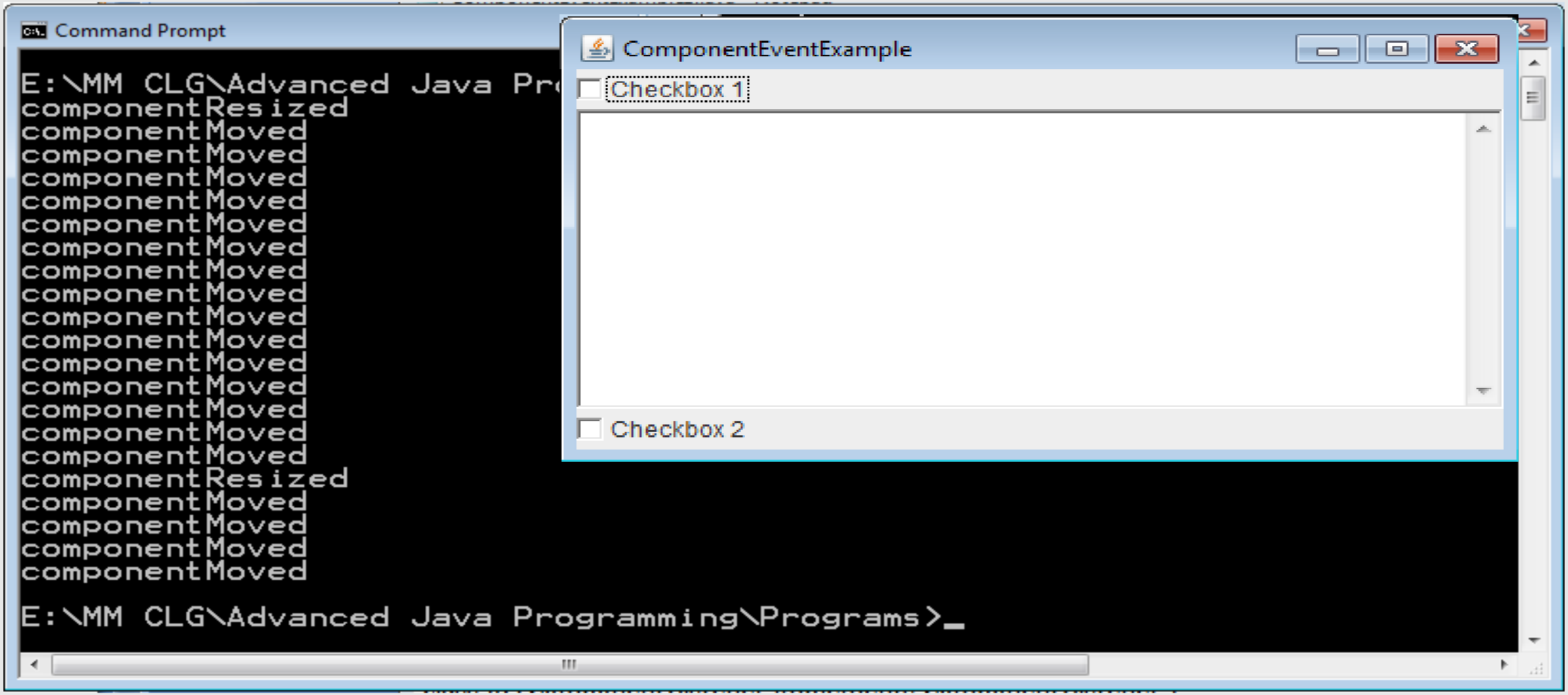

```
class MyComponentListener implements ComponentListener
{
    public void componentShown(ComponentEvent evt)
    {
        System.out.println("componentShown");
    }

    public void componentHidden(ComponentEvent evt)
    {
        System.out.println("componentHidden");
     }

    public void componentMoved(ComponentEvent evt)
    {
        System.out.println("componentMoved");
    }

     public void componentResized(ComponentEvent evt)
     {
         System.out.println("componentResized");
     }
}
```

Output


Command Prompt
E:\MM CLG\Advanced Java Pr
componentResized
componentMoved
componentMoved
componentMoved
componentMoved
componentMoved
componentMoved
componentMoved
componentMoved
componentMoved
componentMoved
componentMoved
componentMoved
componentMoved
componentMoved
componentResized
componentMoved
componentMoved
componentMoved
componentMoved
E:\MM CLG\Advanced Java Programming\Programs>
ComponentEventExample
Checkbox 1
Checkbox 2

ContainerEvent class

- A low-level event which indicates that a container's contents changed because a component was added or removed
- This class has following constants.
  - public static final int COMPONENT_ADDED
    - This event indicates that a component was added to the container.
  - public static final int COMPONENT_REMOVED
    - This event indicates that a component was removed from the container.
  - public ContainerEvent(Component source, int id, Component child)
    - Constructs a ContainerEvent object.
    - Parameters:
      - source - the Component object (container) that originated the event
      - id - an integer indicating the type of event
      - child - the component that was added or removed

Conti…

- public Container getContainer()
  - Returns the originator of the event.
  - Returns the Container object that originated the event, or null if the object is not a Container.
- public Component getChild()
  - Returns the component that was affected by the event.
  - Returns the Component object that was added or removed.
  - The listener interface for receiving container events.
- void componentAdded(ContainerEvent e)
  - Invoked when a component has been added to the container.
- void componentRemoved (ContainerEvent e)
  - Invoked when a component has been removed from the container.

Programming Example


```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ContainerEventExample
{
  public static void main(String args[])
  {
    JFrame frame = new JFrame();
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    Container contentPane = frame.getContentPane();

    ContainerListener cont = new ContainerListener()
    {
        ActionListener listener = new ActionListener()
        {
            public void actionPerformed(ActionEvent e)
          {
              System.out.println("Selected: " + e.getActionCommand());
          }
        };
```

Conti…


```
public void componentAdded(ContainerEvent e)
{
    Component c = e.getChild();
    if (c instanceof JButton)
    {
        JButton b = (JButton) c;
        b.addActionListener(listener);
    }
}

public void componentRemoved(ContainerEvent e)
{
    Component c = e.getChild();
     if (c instanceof JButton)
     {
         JButton b = (JButton) c;
         b.removeActionListener(listener);
     }
   }
};
```

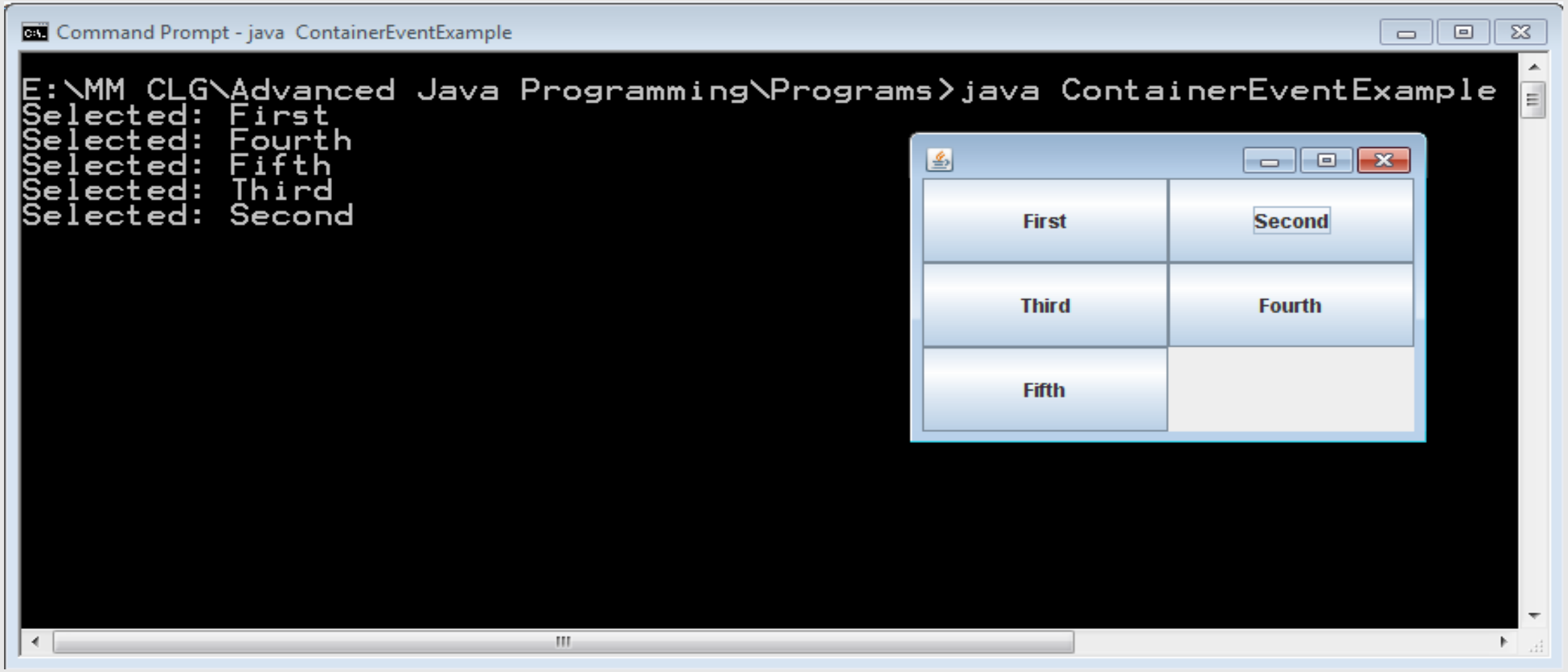
Conti…


```
    contentPane.addContainerListener(cont);

    contentPane.setLayout(new GridLayout(3, 2));
    contentPane.add(new JButton("First"));
    contentPane.add(new JButton("Second"));
    contentPane.add(new JButton("Third"));
    contentPane.add(new JButton("Fourth"));
    contentPane.add(new JButton("Fifth"));

    frame.setSize(300, 200);
    frame.show();
  }
}
```

Output


Command Prompt - java ContainerEventExample
E:\MM CLG\Advanced Java Programming\Programs>java ContainerEventExample
Selected: First
Selected: Fourth
Selected: Fifth
Selected: Third
Selected: Second
First
Second
Third
Fourth
Fifth

FocusEvent class

- A low-level event which indicates that a Component has gained or lost the input focus.
- This class has following constants.
- public static final int FOCUS_GAINED
  - This event indicates that the Component is now the focus owner.
- public static final int FOCUS_LOST
  - This event indicates that the Component is no longer the focus owner.

Constructors

- publicfocusEvent(Component source,int id,boolean temporary, Component opposite)
  - source - the Component that originated the event
  - id - FOCUS_GAINED or FOCUS_LOST
  - temporary - true if the focus change is temporary; false otherwise
  - opposite - the other Component involved in the focus change, or null
- publicFocusEvent(Component source,int id,booleantemporary)
  - id - an integer indicating the type of event
  - temporary - true if the focus change is temporary; false otherwise.
- public FocusEvent(Component source,int id)
  - source - the Component that originated the event
  - id - an integer indicating the type of event

FocusListener interface

- void focusGained(FocusEvent e)
  - Invoked when a component gains the keyboard focus.

- void focusLost(FocusEvent e)
  - Invoked when a component loses the keyboard focus.

Programming Example

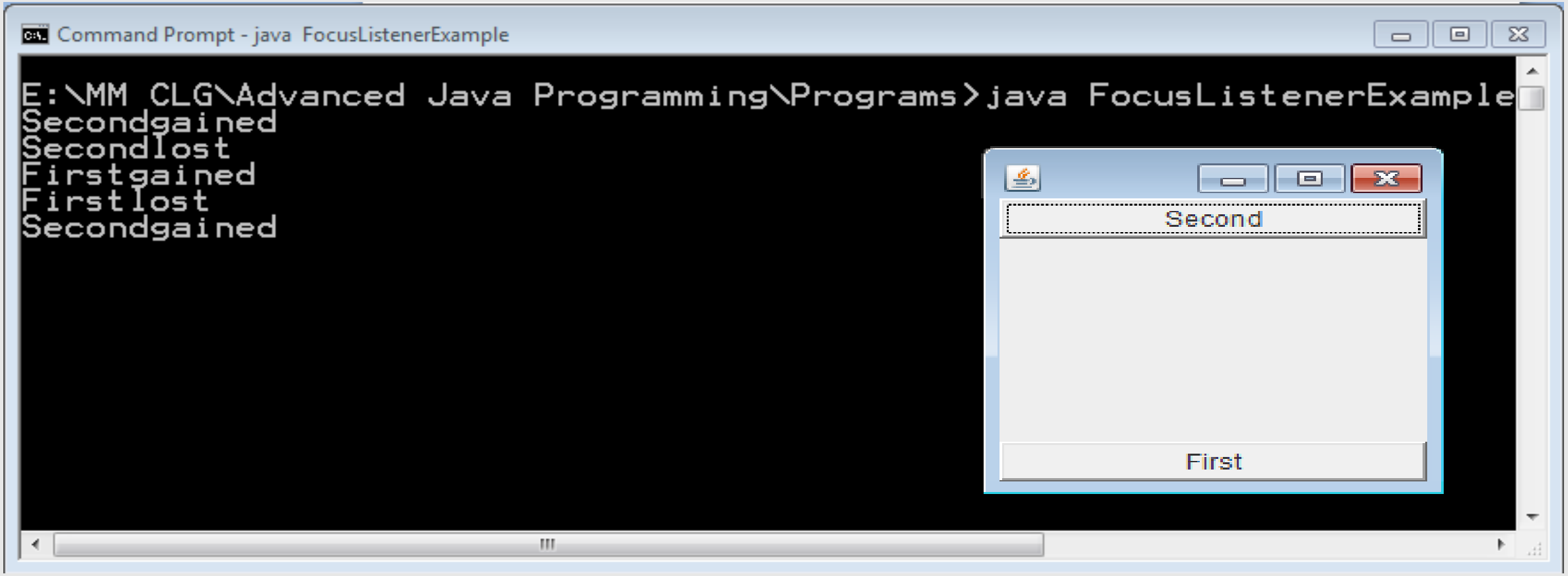

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class FocusListenerExample extends JFrame implements FocusListener
{
    Button b1,b2;
    public FocusListenerExample()
    {
        b1=new Button ("First");
        b2=new Button ("Second");
        add(b1,BorderLayout.SOUTH);
        add(b2,BorderLayout.NORTH);
        b1.addFocusListener(this);
        b2.addFocusListener(this);
        setSize(200,200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }
```

Conti…


```
public void focusGained(FocusEvent fe) //method of focuslistener
    {
       if(fe.getSource()==b1)
              System.out.println(b1.getLabel()+"gained");
       if(fe.getSource()==b2)
              System.out.println(b2.getLabel()+"gained");
       if(fe.isTemporary())
              System.out.println("Temporary Focus");
    }
    public void focusLost(FocusEvent fe) //in focusevent "getID()"is a method
    {
       if(fe.getSource()==b1)
              System.out.println(b1.getLabel()+"lost");
       if(fe.getSource()==b2)
              System.out.println(b2.getLabel()+"lost");
    }
    public static void main(String a[])
    {
       new FocusListenerExample();
    }
}
```

Output


Command Prompt - java FocusListenerExample
E:\MM CLG\Advanced Java Programming\Programs>java FocusListenerExample
Secondgained
Secondlost
Firstgained
Firstlost
Secondgained
Second
First

ItemEvent class

- A semantic event which indicates that an item was selected or deselected.
- This high-level event is generated by an ItemSelectable object (such as a List) when an item is selected or deselected by the user.
- This class has following constants.
- public static final int SELECTED
  - This state-change value indicates that an item was selected.
- public static final int DESELECTED
  - This state-change-value indicates that a selected item was deselected
- public ItemEvent (ItemSelectable source, int id, Object item, int stateChange)
  - Constructs an ItemEvent object.
  - Parameters:
    - source - the ItemSelectable object that originated the event
    - id - an integer that identifies the event type
    - item - an object -- the item affected by the event
    - stateChange - an integer that indicates whether the item was selected or deselected

Methods of ItemEvent Class

- public ItemSelectable getItemSelectable()
  - Returns the creator of the event.
  - Returns: the ItemSelectable object that originated the event.
- public Object getItem()
  - Returns the item affected by the event.
  - Returns: the item (object) that was affected by the event.
- public int getStateChange()
  - Returns the type of state change (selected or deselected).
  - Returns: an integer that indicates whether the item was selected or deselected

ItemListener interface

- The listener interface for receiving item events.
- void itemStateChanged(ItemEvent e)
- Invoked when an item has been selected or deselected by the user.
- The code written for this method performs the operations that need to occur when an item is selected (or deselected).

Programming Example


```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;

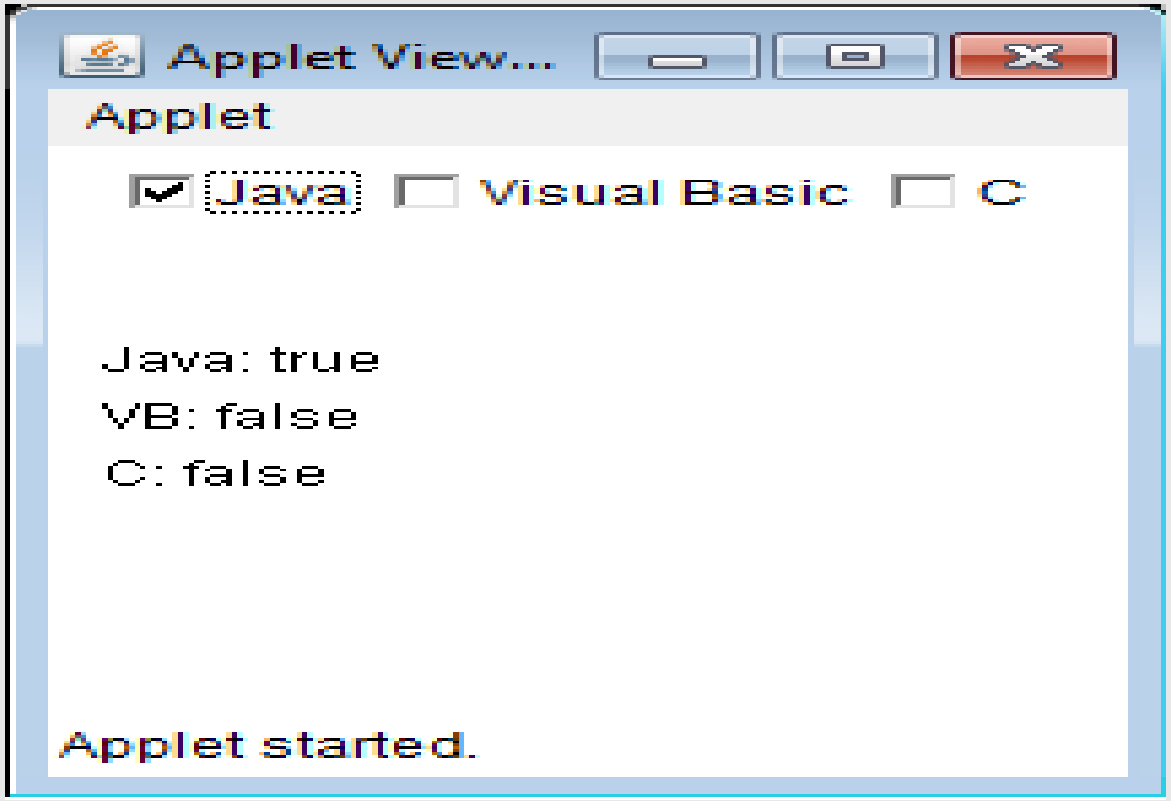
 /* <applet code="ItemListenerExample" width=200 height=200>
</applet> */

public class ItemListenerExample extends Applet implements ItemListener
{

        Checkbox java = null;
        Checkbox vb = null;
        Checkbox c = null;
```

```
    public void init()
    {
        java = new Checkbox("Java");
        vb = new Checkbox("Visual Basic");
        c = new Checkbox("C");
          add(java);
        add(vb);
        add(c);
        java.addItemListener(this);
        vb.addItemListener(this);
        c.addItemListener(this);
    }
    public void paint(Graphics g)       {
        g.drawString("Java: " + java.getState(),10,80);
        g.drawString("VB: " + vb.getState(), 10, 100);
        g.drawString("C: " + c.getState(), 10, 120);
    }
   public void itemStateChanged(ItemEvent ie)
   {
        repaint();
    }
}
```

Output


Applet View...
Applet
Java
Visual Basic
C
Java: true
VB: false
C: false
Applet started.

KeyEvent class

- An event which indicates that a keystroke occurred in a component.
- This class has following constant.
  - public static final int KEY_PRESSED
    - The "key pressed" event. This event is generated when a key is pushed down.
  - public static final int KEY_RELEASED
    - The "key released" event. This event is generated when a key is let up.
  - public static final int KEY_TYPED
    - The "key typed" event.
    - This event is generated when a character is entered. In the simplest case, it is produced by a single key press.
    - Often, however, characters are produced by series of key presses, and the mapping from key pressed events to key typed events may be many-to-one or many-to-many.

Methods of KeyEvent class

- public int getKeyCode()
  - Returns the integer keyCode associated with the key in this event.
  - Returns: the integer code for an actual key on the keyboard.
- public char getKeyChar()
  - Returns the character associated with the key in this event.
  - For example, the KEY_TYPED event for shift + "a" returns the value for "A".
  - boolean isActionKey()
  - Returns true if the key firing the event is an action key. Examples of action keys include Page Up, Caps Lock, the arrow and function keys.

KeyListener Interface

- Key events indicate when the user is typing at the keyboard.
- Key events are fired by the component with the keyboard focus when the user presses or releases keyboard keys.
- Notifications are sent about two basic kinds of key events:
  - The typing of a Unicode character
  - The pressing or releasing of a key on the keyboard
- The first kind of event is called a key-typed event.
- To know when the user types a Unicode character ? whether by pressing one key such as 'a' or by pressing several keys in sequence ?
- The second kind is either a key-pressed or key-released event.
- To know when the user presses the F1 key, or whether the user pressed the '3' key on the number pad, you handle key-pressed events.

Methods of KeyListener Interface

Method	Purpose
keyTyped(KeyEvent)	Called just after the user types a Unicode character into the listened-to component.
keyPressed(KeyEvent)	Called just after the user presses a key while the listened-to component has the focus.
keyReleased(KeyEvent)	Called just after the user releases a key while the listened-to component has the focus.

Programming Example


```
import java.awt.*;
import java.awt.event.*;
import javax.swing.JApplet;
public class EventDemo6 extends JApplet implements KeyListener
{
   String event; // description of keyboard event
   public void init() // set up UI
   {
       setLayout(new FlowLayout());
       event = ""; addKeyListener(this); // listen for keyboard events
       setFocusable(true); // force applet to receive KeyEvent
    }
   public void paint(Graphics g) // draw message to applet
   {
      super.paint(g);
      g.drawRect(0, 0, getWidth(), getHeight()); // show bounds of applet
      g.drawString(event, 10, 50);
   }
```

Conti…


```
    public void keyPressed(KeyEvent e) // handle key presses
    {
        event = e.getKeyChar() + " pressed"; repaint();
    }
    public void keyReleased(KeyEvent e) // handle key releases
    {
        event = e.getKeyChar() + " released"; repaint();
    }
    public void keyTyped(KeyEvent e) // handle typing on applet
    {
        event = e.getKeyChar() + " typed"; repaint();
     }
}
```

TextEvent class

- A semantic event which indicates that an object's text changed.
- This high-level event is generated by an object (such as a TextComponent) when its text changes.
- public TextEvent(Object source,int id)
  - Constructs a TextEvent object.
  - Parameters:
    - source - the (TextComponent) object that originated the event
    - id - an integer that identifies the event type

TextListener interface

- The listener interface for receiving text events.
- void textValueChanged(TextEvent e)
- Invoked when the value of the text has changed.
- The code written for this method performs the operations that need to occur when text changes.

WindowEvent class

- A low-level event indicates that a window has changed its status.
- This event is generated by a Window object when it is opened, closed, activated, deactivated, iconified, or deiconified, or when focus is transferred into or out of the Window.

int constants

- WINDOW_ACTIVATED
- WINDOW_CLOSED
- WINDOW_CLOSING
- WINDOW_DEACTIVATED
- WINDOW_DEICONIFIED
- WINDOW_GAINED_FOCUS
- WINDOW_ICONIFIED
- WINDOW_LOST_FOCUS
- WINDOW_OPENED
- WINDOW_STATE_CHANGED

Constructors

- public WindowEvent(Window source,int id)
  - Constructs a WindowEvent object.
  - Parameters: source - the Window object that originated the event
  - id - an integer indicating the type of event
- publicWindowEvent(Window source, int id, Window opposite, int oldState, int newState)
  - Note that passing in an invalid id results in unspecified behavior.
  - This method throws an **IllegalArgumentException** if source is null.
  - Parametrs:
    - source - the Window object that originated the event
    - id - an integer indicating the type of event.
    - opposite - the other window involved in the focus or activation change, or null
    - oldState - previous state of the window for window state change event
    - newState - new state of the window for window state change event

WindowListener interface

- void windowClosing(WindowEvent e)
  - Invoked when the user attempts to close the window from the window's system menu.
- void windowClosed(WindowEvent e)
  - Invoked when a window has been closed as the result of calling dispose on the window
- void windowIconified(WindowEvent e)
  - Invoked when a window is changed from a normal to a minimized state.
- void windowOpened(WindowEvent e)
  - Invoked the first time a window is made visible.
- void windowDeiconified(WindowEvent e)
  - Invoked when a window is changed from a minimized to a normal state.
- void windowActivated(WindowEvent e)
  - Invoked when the Window is set to be the active Window.
- void windowDeactivated(WindowEvent e)
  - Invoked when a Window is no longer the active Window

WindowFocusListener interface

- The listener interface for receiving WindowEvents, including WINDOW_GAINED_FOCUS and WINDOW_LOST_FOCUS events.
- void windowGainedFocus(WindowEvent e)
  - Invoked when the Window is set to be the focused Window, which means that the Window, or one of its subcomponents, will receive keyboard events.
  - This event indicates a mouse action occurred in a component. This low-level event is generated by a component object for Mouse Events and Mouse motion events.

Constructor

➢MouseEvent(Component source, int id, long when, int modifiers, int x, int y, int clickCount, boolean popupTrigger)

- Constructs a MouseEvent object with the specified
  - source- source component,
  - id- type of event,
  - when- system time at mouse event occurred
  - modifiers-to know what modifiers were pressed after event was occurred,
  - x & y- coordinates of the mouse ,
  - clickCount- click count
  - popupTrigger- whether popup menu appeared

Method	Purpose
int getClickCount()	Returns the number of quick, consecutive clicks the user has made (including this event). For example, returns 2 for a double click.
int getButton()	Returns which mouse button, if any, has a changed state. One of the following constants is returned: NOBUTTON, BUTTON1, BUTTON2, or BUTTON3.
int getX() int getY()	Return the (x,y) position at which the event occurred, relative to the component that fired the event.
Point getPoint()	Returns the x,y position of the event rlative to the source component.

MouseListener Interface

- Mouse events notify when the user uses the mouse (or similar input device) to interact with a component.
- Mouse events occur when the cursor enters or exits a component's onscreen area and when the user presses or releases one of the mouse buttons.

Methods of MouseListener Interface

Method	Purpose
mouseClicked(MouseEvent)	Called just after the user clicks the listened-to component.
mouseEntered(MouseEvent)	Called just after the cursor enters the bounds of the listened-to component.
mouseExited(MouseEvent)	Called just after the cursor exits the bounds of the listened-to component.
mousePressed(MouseEvent)	Called just after the user presses a mouse button while the cursor is over the listened-to component.
mouseReleased(MouseEvent)	Called just after the user releases a mouse button after a mouse press over the listened-to component.

MouseMotionListener Interface

- Mouse-motion events notify when the user uses the mouse (or a similar input device) to move the onscreen cursor.
- If an application requires the detection of both mouse events and mouse-motion events, use the MouseInputAdapter class.
- It implements the MouseInputListener a convenient interface that implements both the MouseListener and MouseMotionListener interfaces.

Programming Example


```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class MouseEventDemo extends JApplet implements MouseListener
{
   private int x;            // x coordinate of mouse event
   private int y;            // y coordinate of mouse event
   private String event;     // description of mouse event

   public void init()                        // set up GUI
   {
      setLayout(new FlowLayout());

      addMouseListener(this);                // listen for mouse events

      x = -1;                           // set x negative for no initial message
   }
```

```
public void paint(Graphics g)                    // draw message to screen
   {
     super.paint(g);

     g.drawRect(0, 0, getWidth(), getHeight());   // show bounds of applet

     if(x != - 1)                                // display event during repainting only
     {
       g.drawString("Mouse event " + event +
                    " at (" + x + ", " + y + ")",
                    10, 50);
     }
   }
public void mousePressed(MouseEvent e)          // save coordinates of presses
   {
     x = e.getX();
     y = e.getY();
     event = "press";

     repaint();
   }
```

```
public void mouseClicked(MouseEvent e)          // save coordinates of clicks
    {
      x = e.getX();
      y = e.getY();
      event = "click";

      repaint();
    }

 public void mouseReleased(MouseEvent e)          // save coordinates of releases
    {
      x = e.getX();
      y = e.getY();
      event = "release";

      repaint();
    }
```

```
  public void mouseEntered(MouseEvent e)     // save coordinates when mouse enters applet
    {
      x = e.getX();
      y = e.getY();
      event = "enter";

      repaint();
    }

    public void mouseExited(MouseEvent e)     // save coordinates when mouse leaves applet
    {
      x = e.getX();
      y = e.getY();
      event = "exit";

      repaint();
    }
}
/*<applet code=MouseEventDemo height=300 width=300></applet>*/
```

Annotations

Introduction

- Java provides feature that enables to embed supplemental information into the source file.
- This information is called Annotation and does not change the action of the program
- e.g. @Override
- Annotation leaves the semantics of the program unchanged

Example

```
 class A
{     void show()
      { System.out.print(“Hello”);}
}
 class B extends A
{
     void show()
     {System.out.print(“Bye”)’ }
}

public class Ann
{
  public static void main(String args[])
{
   A a1=new A();
   B b1= new B();
  a1.show();
   b1.show();
}
```

```
 class A
{
    void showmygrade()
     { System.out.print(“CGPA is 9”);}
}
 class B extends A
{
    void showmyGrade()
     {System.out.print(“CGPA is 8”) }
}

public class Ann
{
  public static void main(String args[])
{
   B b1= new B();
   b1.showmygrade();
}
```

Example of @Override

```
class A
{
    void showmygrade()
    { System.out.print(“CGPA is 9”);}
}
 class B extends A
{ @Override
    void showmygrade()
     {System.out.print(“CGPA is 8”) }
}

public class Ann
{
  public static void main(String args[])
{
   B b1= new B();
   b1.showmygrade();
}
```

```
class A
{
   void showmygrade()
   { System.out.print(“CGPA is 9”);}
}
 class B extends A
{
@Deprecated
  void show
{ }
@Override
   void showmygrade()
    {System.out.print(“CGPA is 8”) }
}

public class Ann
{
  public static void main(String args[])
{
   B b1= new B();
   b1.showmygrade();
}
```

```
class A
{
    void showmygrade()
    { System.out.print(“CGPA is 9”);}
}
 class B extends A
{
@Deprecated
  void show ( )
{ }
@Override
    void showmygrade()
     {System.out.print(“CGPA is 8”) }
}

public class Ann
{
  public static void main(String args[])
{
   B b1= new B();
  b1.show();
   b1.showmygrade();
}
```

Example of @Functional Interface

```
@Functional Interface
Interface A
{
  void fun();
}
class B extends A
{
@Deprecated
  void show ( )
{ }
@Override
    void showmygrade()
     {System.out.print(“CGPA is 8”) }
}

public class Ann
{
  public static void main(String args[])
{
   B b1= new B();
  b1.show();
   b1.showmygrade();
}
```

Create your own Annotations

```
@interface Smartphone
{
   String os();
   int ver();
}


@Smartphone (os="Android", ver=6)
class NokiaASeries
{
  String model;
   int size;
}
public class Demo
{
 public static void main(String args[])
{
}
}
```

```
@interface Smartphone
{
   String os() default “Symbian”;
   int ver() default 1;
}


@Smartphone
class NokiaASeries
{
  String model;
   int size;
}
public class Demo
{
 public static void main(String args[])
{
}
}
```

```
@interface Smartphone
{
   String os() default “Symbian”;
   int ver() default 1;
}

@Smartphone (os=“Android”, ver =6)
class NokiaASeries
{
  String model;
   int size;
  public NokiaAseries(String model, int size)
 {
    this.model=model;
   this.size=size;
  }
}
public class Demo
{
 public static void main(String args[])
{
   NokiaAseries A1= new NokiaAseries(“XYZ”, 5);
  System.out.print(A1.model);
}
}
```

```
public class Demo
{
 public static void main(String args[])
{
    NokiaAseries A1= new NokiaAseries(“XYZ”, 5);
  System.out.print(A1.model);
 Class c =a1.getClass();      //Reflection API to fetch the value of Annotation
 Annotation an= c.getAnnotation(Smartphone.class);
 Smaprtphone s= (Smartphone) an;
System.out.print(s.os());
}
}
```

Meta Annotation

@Target(ElementType.Type) //where this annotation will be used. Here class

@Retention (RetentionPolicy.RUNTIME)

@interface Smartphone

{}

- Marker Annotation – no value
- Single Value Annotation- single value
- Multiple Value Annotation – multiple value

MAVEN

What is Maven?

- Hosted by Apache Software Foundation
- While Eclipse is an IDE that provides environment for developing project, Maven is used for building the code
- Traditionally for software development in Java we need to use third party libraries or dependencies or libraries (jar files) to execute Spring or hibernate
- Spring needs 10-12 libraries that need to downloaded and add them manually. Further, in case of version change, this step must be repeated.
- Maven relieves you from this

- Maven solves the problem related to dependencies.
- You just need to specify the dependencies that you need in pom.xml file
- In case of version change, you just need to change pom.xml
- Maven is a build automation tool
- Maven is a project management tool
- Maven handles the complete build process

Maven is used for Java based project and addresses two critical aspects of building software

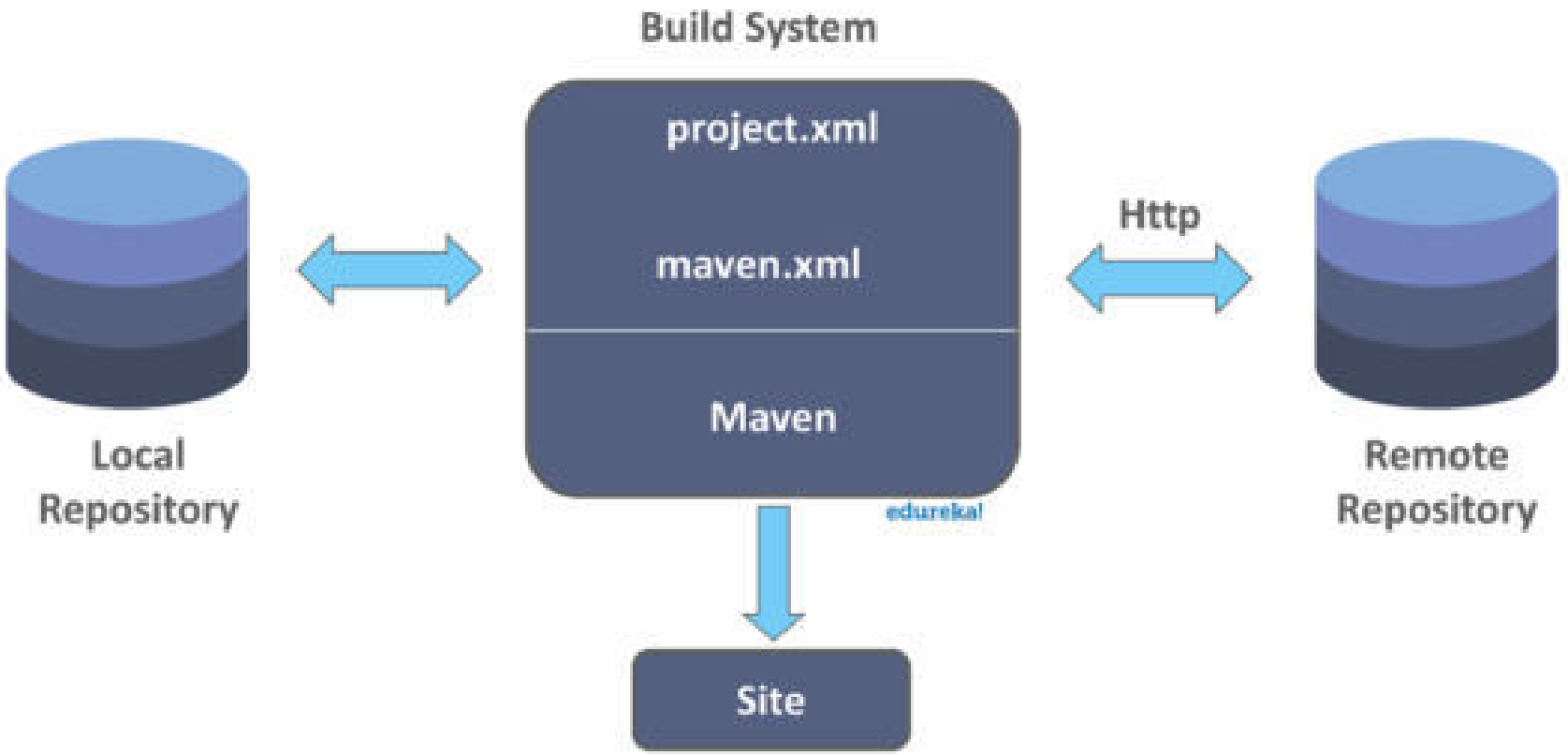
(i) Describes how software is built

(ii) Describe the dependencies

- Maven uses conventions for build procedures and only exceptions need to be written down
- An XML file describes the software project that is built, its dependencies on other external modules and components, the built order, the directories and required plugin.

- It comes with pre-defined targets for performing well-defined tasks such as compilation of code and its packaging
- Maven dynamically downloads java libraries and maven plugins from one or more repositories such as maven central repository and stores in local cache.
- This local cache of the downloaded artifacts can also be updated with the artifacts created with the local repository
- Public repository can also be updated
- Maven can also be used to build and manage projects written in c#, scala etc

Maven Architecture



- When you specify a dependency in pom.xml file of maven ---→
- Maven will look for a file in Central Repository ---→
- If a file is present in Central Repository it will copy into the local machine and if not then it will fetch the file from the remote repository via the Internet.

Maven Life Cycle – to deploy & distribute target project

Clean Lifecyle
pre-clean
clean
post-clean

Default Lifecyle	
validate	test-compile
initialize	process-test-classes
generate-sources	test
process-sources	prepare-package
generate-resources	**package**
process-resources	pre-integration-test
compile	integration-test
process-classes	post-integration-test
generate-test-sources	verify
process-test-sources	**install**
generate-test-resources	**deploy**
processs-test-resources	

Site Lifecyle
pre-site
site
post-site
site-deploy

Clean Life Cycle

- Clean Project
- Removes all files by previous built
- It has 3 phases

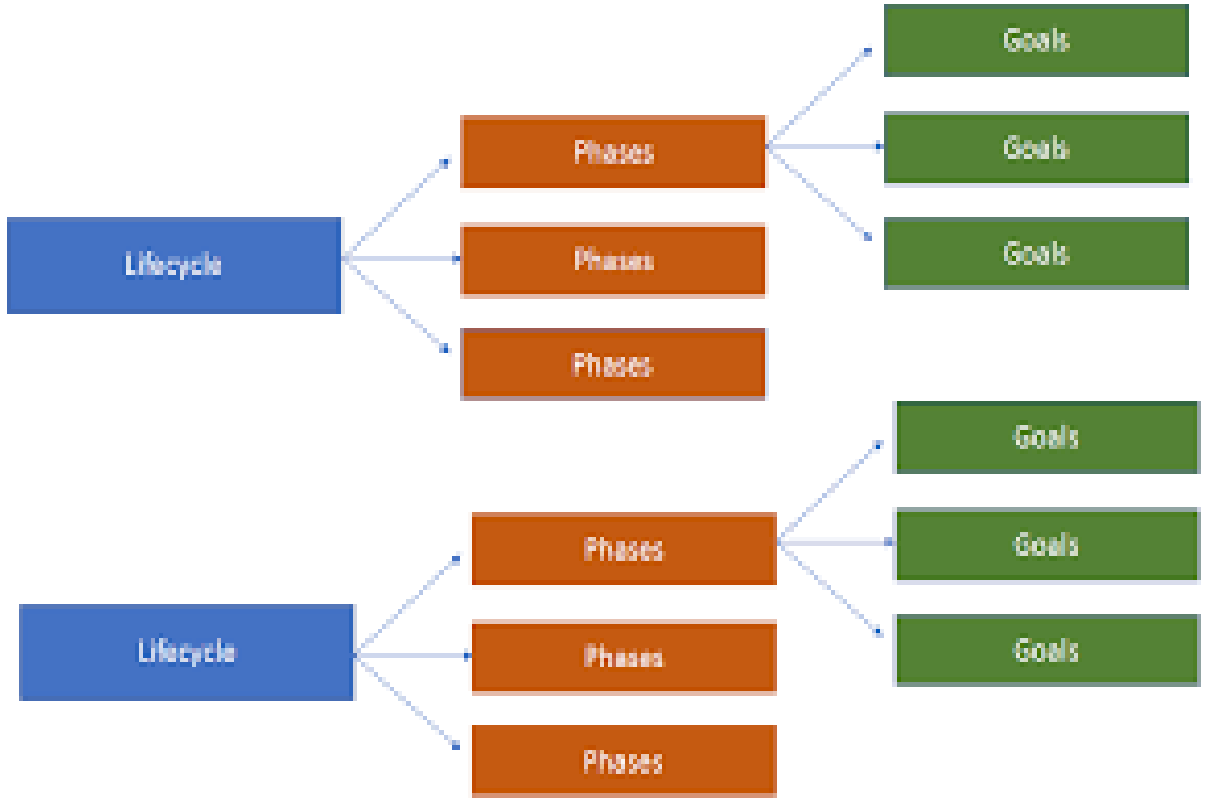
Default life cycle

- Main life cycle
- Responsible for project deployment
- It has 23 phases

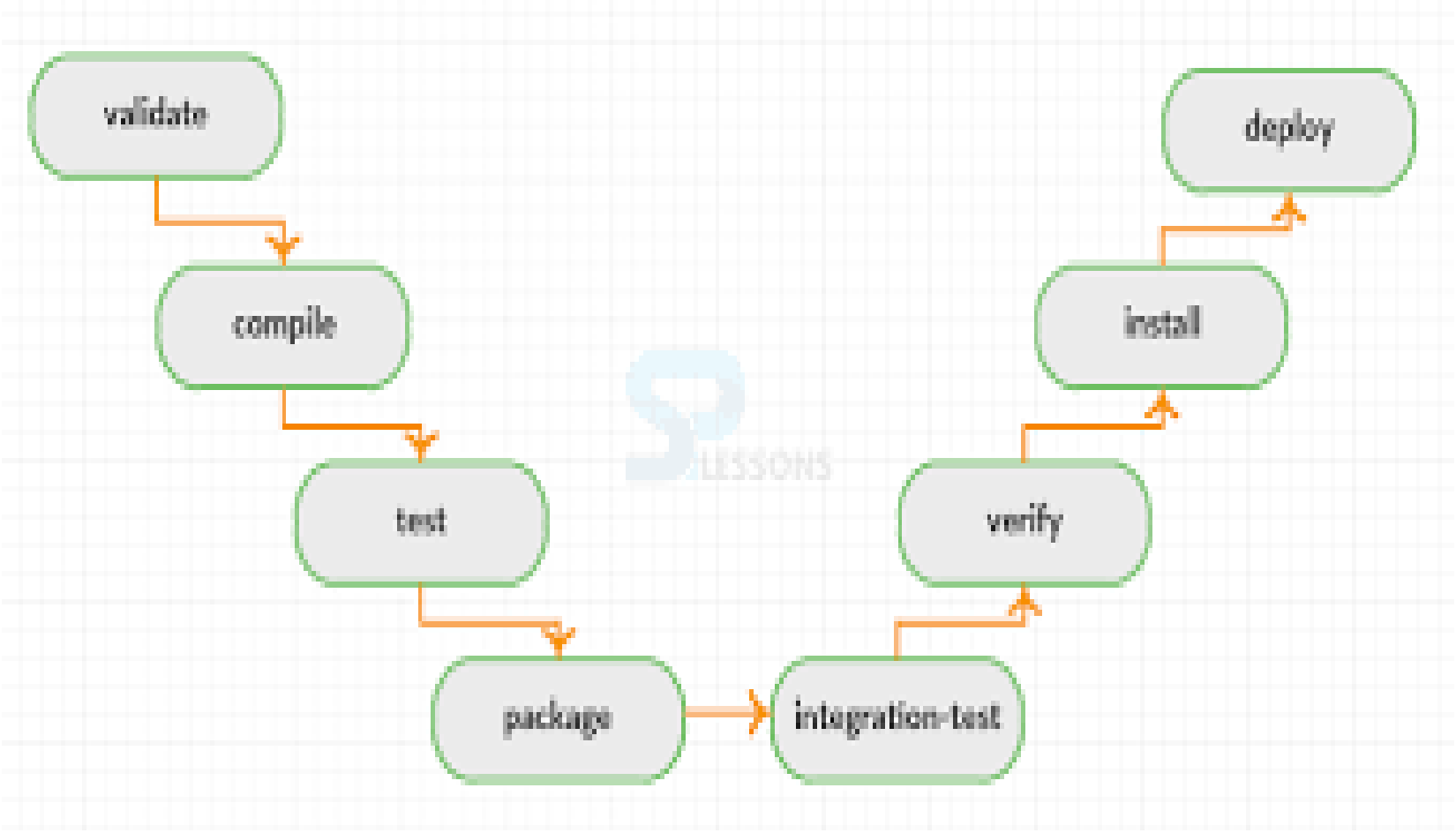
Site Lifecycle

- Needed for site documentation
- It has 4 phases

-A lifecycle is organized into sequence of phases.
-Each phase is responsible for a specific task.
-Phases give order to a set of goals.
-These goals are chosen and bound by the packaging type of the project being acted on



Important phases in the default life cycle



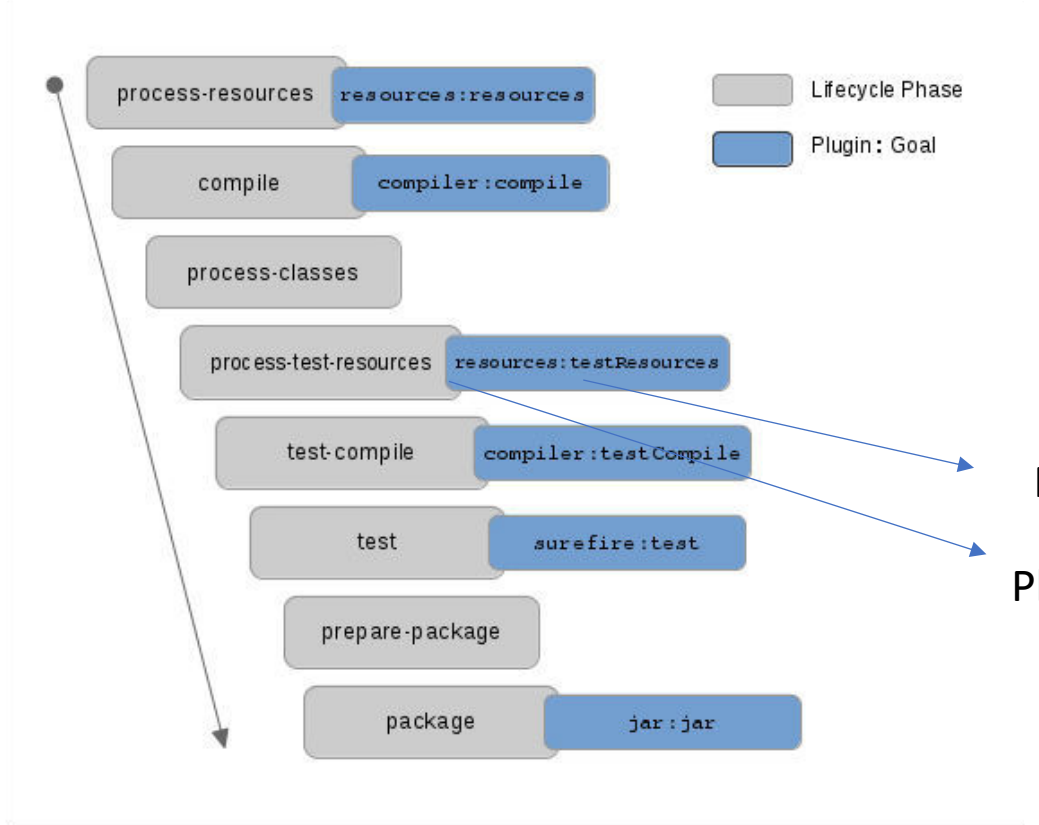
Phases of Default life cycle

- **Validate**
  - Checks if all the information necessary for the build is available;
- **Compile**
  - Compiles the source code of the project
- **Test compile**
  - Compiles the test source code
  - Responsible for running the unit test of the application
- **Packaging**
  - Packaging the compiled source code into the distributable format (i.e jar format)
- **Integration**
  - In this phase the process and deployment of the package is needed
- **Install**
  - Install the package to a local repository
- **Deploy**
  - Copy the package to a remote repository

- -Each phase has a set of goals and these goals are executed by a plug-in.
  -Phases give order to the goals
  -Each goal is responsible for a specific task
  -When we run a phase, all goals bound to that phase are executed in order

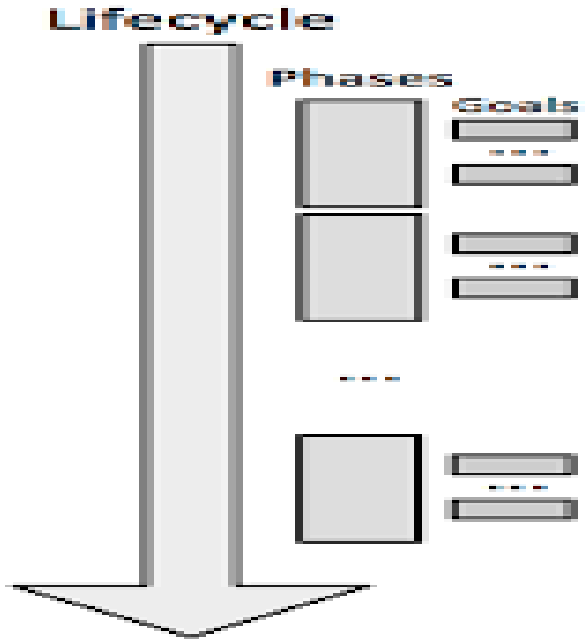
- All Maven plug-ins are a group of goals
- All execution is done by plug-ins
- A plug-in is mapped to a goal and is executed as a part of it
- We can invoke a specific goal while maven execution.
- A plug-in configuration can be modified using a plug-in declaration
- Example: Compiler plug-in compiles the java source code

  Compiler plug-in has 2 goals- Compile and test compile



A plug-in is bound to a phase

Lifecycle->Phases-> Goals
Plug-in are mapped to goals and executed as part of goals



- Project -> Anything that we create in Maven
- Pom.xml is the main file where we describe the dependencies

<project xmlns….

<dependency>

<groupID> com.anchal //unique identifier

<artifactID> demo

<version>

<dependency>

<groupID> org.Springframework </groupID>

<artifactID> Spring-xyz </artifactID>

<version> 4.3.8 release </version>

MVN Repository-> Spring -> copy dependency details and copy to pom.xml

Maven Coordinates (GAV)

- The following three fields act like an address and timestamp.

- groupID
- artifactID
- Version

- http://maven.apache.org/pom.html#Maven_Coordinates

Maven Tutorial Practical

- https://www.youtube.com/watch?v=uEYjXpMDJiU

What is POM?

- A Project Object Model or POM is the fundamental unit of work in Maven.
- It is an XML file that contains information about the project and configuration details used by Maven to build the project.
- It contains default values for most projects.
- Example- build directory (target), source directory(src/main/java), test directory (src/test/java)
- When executing a task or goal, Maven looks for the POM in the current directory. It reads the POM, gets the needed configuration information, then executes the goal

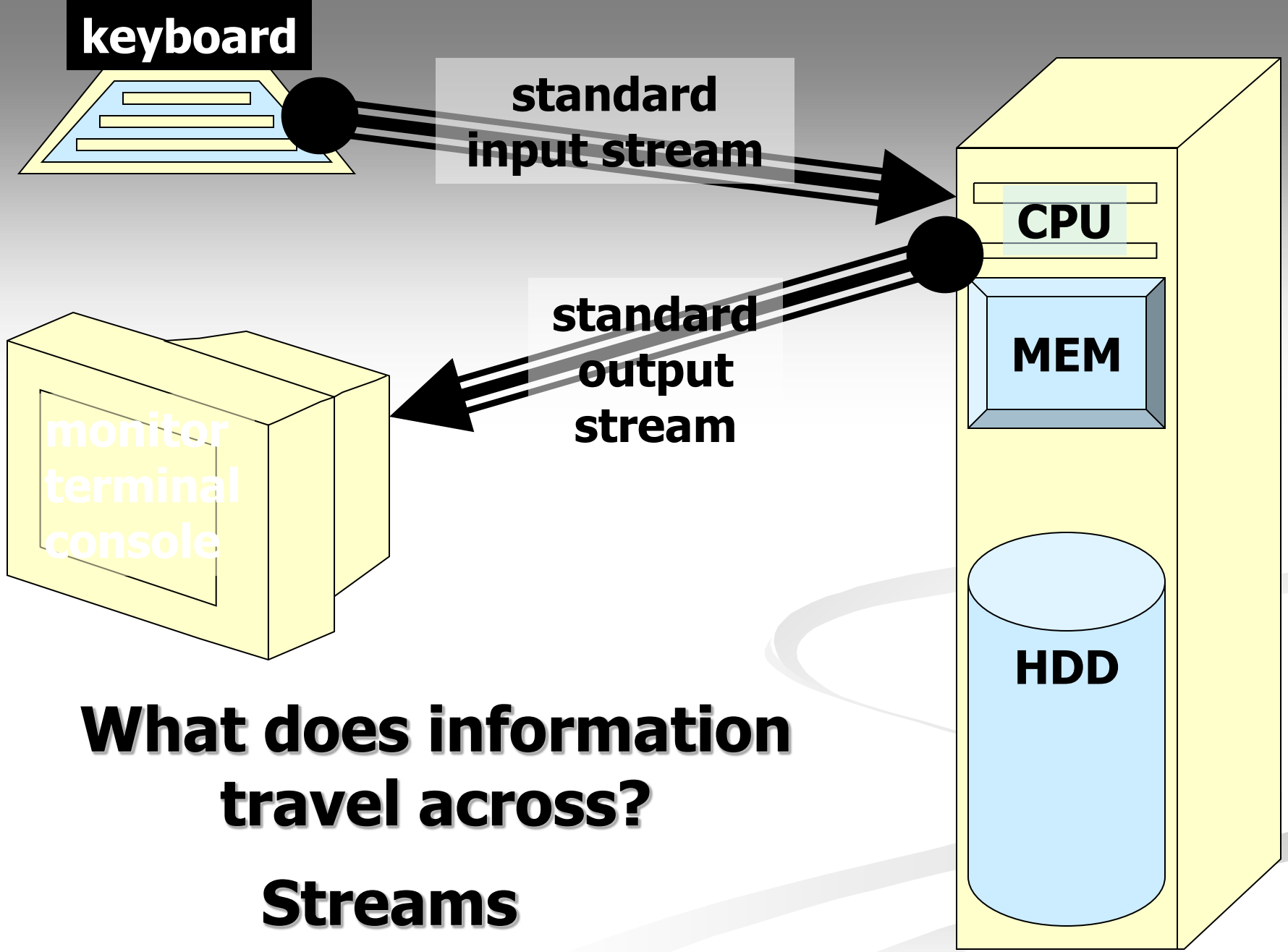
Super POM

- http://maven.apache.org/guides/introduction/introduction-to-the-pom.html

Files and Streams

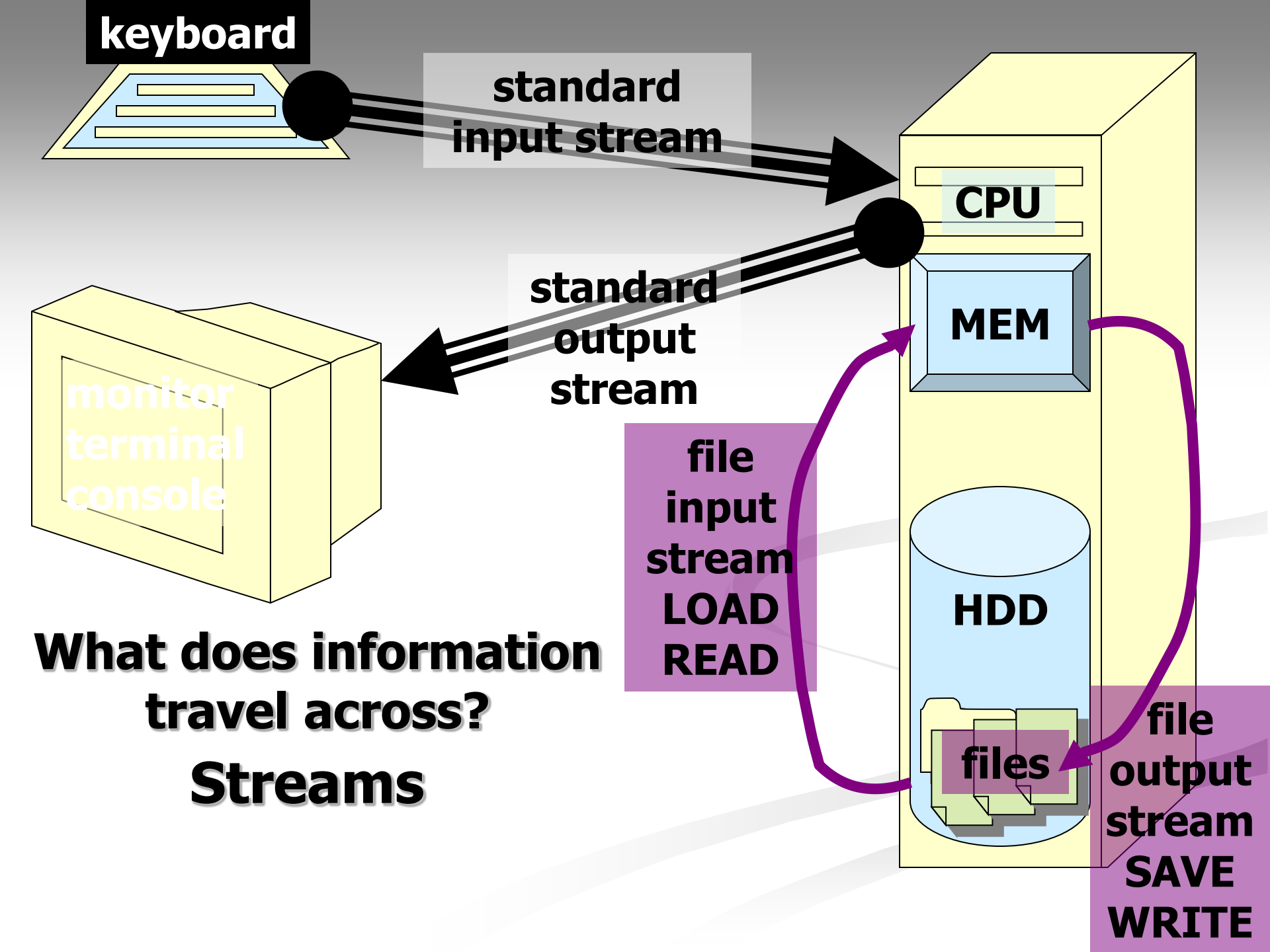
Goals

- To be able to read and write text files
- To become familiar with the concepts of text and binary formats
- To learn about encryption
- To understand when to use sequential and random file access
- To be able to read and write objects using serialization



What does information travel across?

Streams



What does information travel across?

Streams

Reading and Writing Text Files

- Text files – files containing simple text
  - Created with editors such as notepad, html, etc.
- Simplest way to learn it so extend our use of **`Scanner`**
  - Associate with files instead of **`System.in`**
- All input classes, except Scanner, are in java.io
  - **`import java.io.*;`**

Review: Scanner

- We've seen Scanner before
- The constructor takes an object of type **`java.io.InputStream`** – stores information about the connection between an input device and the computer or program
  - Example: **`System.in`**
- Recall – only associate ***one*** instance of **`Scanner`** with **`System.in`** in your program
  - Otherwise, get bugs

Numerical Input

- 2 ways (we've learned one, seen the other)
  - Use **`int`** as example, similar for **`double`**

- First way:
  - Use **`nextInt()`**

```
int number = scanner.nextInt();
```

- Second way:
  - Use **`nextLine()`, `Integer.parseInt()`**

```
String input = scanner.nextLine();
int number = Integer.parseInt(input);
```

Numerical Input

- Exceptions
  - **`nextInt()`** throws **`InputMismatchException`**
  - **`parseInt()`** throws **`NumberFormatException`**
- Optimal use
  - **`nextInt()`** when there is multiple information on one line
  - **`nextLine() + parseInt()`** when one number per line

Reading Files

- The same applies for both console input and file input
- We can use a different version of a Scanner that takes a ***File*** instead of **`System.in`**
- Everything works the same!

Reading Files

- To read from a disk file, construct a **`FileReader`**
- Then, use the **`FileReader`** to construct a **`Scanner`** object

```
FileReader rdr = newFileReader("input.txt");
Scanner fin = new Scanner(rdr);
```

Reading Files

- You can use **`File`** instead of **`FileReader`**
  - Has an **`exists()`** method we can call to avoid **`FileNotFoundException`**

```
File file = new File ("input.txt");
Scanner fin;
if(file.exists()){
  fin = new Scanner(file);
} else {
  //ask for another file
}
```

Reading Files

- Once we have a Scanner, we can use methods we already know:
  - **`next`**, **`nextLine`**, **`nextInt`**, etc.

- Reads the information from the file instead of console

File Class

- **`java.io.File`**
  - associated with an actual file on hard drive
  - used to check file's status
- Constructors
  - **`File(<full path>)`**
  - **`File(<path>, <filename>)`**
- Methods
  - **`exists()`**
  - **`canRead()`**, **`canWrite()`**
  - **`isFile()`**, **`isDirectory()`**

File Class

- **`java.io.FileReader`**
  - Associated with **`File`** object
  - Translates data bytes from File object into a stream of characters (much like InputStream vs. InputStreamReader)
- Constructors
  - **`FileReader( <File object> );`**
- Methods
  - **`read()`**, **`readLine()`**
  - **`close()`**

Writing to a File

- We will use a **`PrintWriter`** object to write to a file
  - What if file already exists? → Empty file
  - Doesn't exist? → Create empty file with that name

- How do we use a **`PrintWriter`** object?
  - Have we already seen one?

Writing to a File

- The out field of the System class is a **`PrintWriter`** object associated with the console
  - We will associate our **`PrintWriter`** with a file now

```
PrintWriter fout = new PrintWriter("output.txt");
fout.println(29.95);
fout.println(new Rectangle(5, 10, 15, 25));

fout.println("Hello, World!");
```

- This will print the exact same information as with **`System.out`** (except to a file "output.txt")!

Closing a File

- Only main difference is that we have to close the file stream when we are done writing
- If we do not, not all output will written
- At the end of output, call **`close()`**

```
fout.close();
```

Closing a File

- Why?
  - When you call **`print()`** and/or **`println()`**, the output is actually written to a buffer. When you close or flush the output, the buffer is written to the file
  - The slowest part of the computer is hard drive operations – much more efficient to write once instead of writing repeated times

File Locations

- When determining a file name, the default is to place in the same directory as your .class files
- If we want to define other place, use an absolute path (e.g. c:\My Documents)

```
in  = new
  FileReader("c:\\homework\\input.dat");
```

- Why **** ?

Sample Program

- Two things to notice:
  - Have to import from java.io
  - I/O requires us to catch checked exceptions
    - **`java.io.IOException`**

Java Input Review

CONSOLE:

```
Scanner stdin = new Scanner( System.in );
```

FILE:

```
Scanner inFile = new Scanner( new
  FileReader(srcFileName ));
```

Java Output Review

- CONSOLE:

```
System.out.print("To the screen");
```

- FILE:

```
PrintWriter fout =
  new PrintWriter(new File("output.txt");
fout.print("To a file");
```

```
import java.io.FileReader;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Scanner;

public class LineNumberer{
  public static void main(String[] args){
      Scanner console = new Scanner(System.in);
      System.out.print("Input file: ");
      String inFile = console.next();

      System.out.print("Output file: ");
      String outFile = console.next();

      try{
            FileReader reader = new FileReader(inFile);
            Scanner in = new Scanner(reader);
```

```
        PrintWriter out = new
          PrintWriter(outputFileName);
        int lineNumber = 1;

        while (in.hasNextLine()){
          String line = in.nextLine();
          out.println("/* " + lineNumber + " */ " +
          line);
          lineNumber++;
        }

        out.close();
      } catch (IOException exception){
          System.out.println("Error processing file: "
            + exception);
      }
   }
}
```

An Encryption Program

- Demonstration: Use encryption to show file techniques

- File encryption
  - To scramble a file so that it is readable only to those who know the encryption method and secret keyword
  - (Big area of CS in terms of commercial applications – biometrics, 128-bit encryption breaking, etc.)

Modifications of Output

- Two constraints so far:
  - Files are overwritten
  - Output is buffered and not written immediately
- We have options to get around this

File Class

- **`java.io.FileWriter`**
  - Associated with **`File`** object
  - Connects an output stream to write bytes of info
- Constructors
  - **`FileWriter( <filename>, <boolean> );`**
    - **true to append data, false to overwrite all of file**
- This will overwrite an existing file
  - To avoid, create File object and see if `exists()` is true

Java File Output

- **`PrintWriter`**
  - composed from several objects

```
PrintWriter out =
  new PrintWriter(
  new FileWriter( dstFileName, false ), true );
```

  - requires **`throws FileNotFoundException`**, which is a sub class of **`IOException`**

- Methods
  - **`print()`**, **`println()`**: **buffers** data to write
  - **`flush()`**: sends buffered output to destination
  - **`close()`**: flushes and closes stream

Java File Output

```
// With append to an existing file
PrintWriter outFile1 =
  new PrintWriter(
    new FileWriter(dstFileName,true),false);

// With autoflush on println
PrintWriter outFile2 =
  new PrintWriter(
    new FileWriter(dstFileName,false),true);

outFile1.println( "appended w/out flush" );
outFile2.println( "overwrite with flush" );
```

To flush or not to flush

- Advantage to flush:
  - Safer – guaranteed that all of our data will write to the file
- Disadvantage
  - Less efficient – writing to file takes up time, more efficient to flush once (on close)

Caeser Cipher

- Encryption key – the function to change the value
- Simple key – shift each letter over by 1 to 25 characters
  - If key = 3, A → D B → E etc.
- Decryption = reversing the encryption
  - Here we just subtract the key value

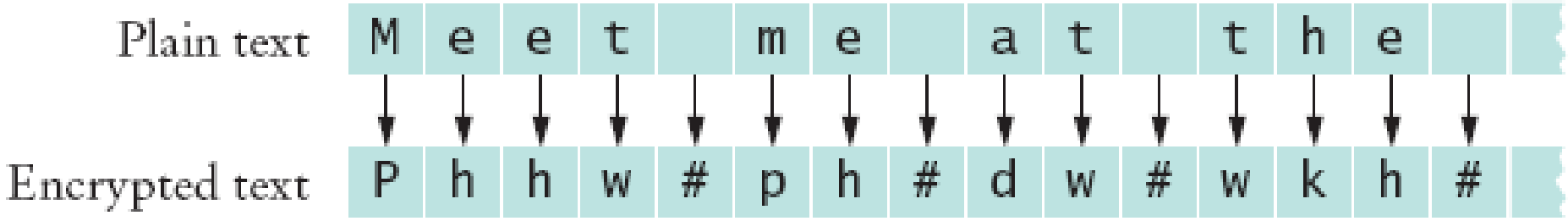


Figure 2 The Caesar Cipher

Binary File Encryption

```
int next = in.read();
if (next == -1)
  done = true;
else {
  byte b = (byte) next;
  //call the method to encrypt the byte
  byte c = encrypt(b);
  out.write(c);
}
```

```
01: import java.io.File;
02: import java.io.FileInputStream;
03: import java.io.FileOutputStream;
04: import java.io.InputStream;
05: import java.io.OutputStream;
06: import java.io.IOException;
07:
08: /**
09:    An encryptor encrypts files using the Caesar cipher.
10:    For decryption, use an encryptor whose key is the
11:    negative of the encryption key.
12: */
13: public class Encryptor
14: {
15:    /**
16:       Constructs an encryptor.
17:       @param aKey the encryption key
18:    */
19:    public Encryptor(int aKey)
20:    {
21:       key = aKey;
22:    }
23:
24:    /**
25:       Encrypts the contents of a file.
26:       @param inFile the input file
27:       @param outFile the output file
28:    */
29:    public void encryptFile(String inFile, String outFile)
30:          throws IOException
31:    {
32:       InputStream in = null;
33:       OutputStream out = null;
34:
35:       try
```

```
37:          in = new FileInputStream(inFile);
38:          out = new FileOutputStream(outFile);
39:          encryptStream(in, out);
40:       }
41:       finally
42:       {
43:          if (in != null) in.close();
44:          if (out != null) out.close();
45:       }
46:    }
47:
48:    /**
49:       Encrypts the contents of a stream.
50:       @param in the input stream
51:       @param out the output stream
52:    */
53:    public void encryptStream(InputStream in, OutputStream out)
54:          throws IOException
55:    {
56:       boolean done = false;
57:       while (!done)
58:       {
59:          int next = in.read();
60:          if (next == -1) done = true;
61:          else
62:          {
63:             byte b = (byte) next;
64:             byte c = encrypt(b);
65:             out.write(c);
66:          }
67:       }
68:    }
69:
70:    /**
71:       Encrypts a byte.
72:       @param b the byte to encrypt
73:       @return the encrypted byte
74:    */
75:    public byte encrypt(byte b)
76:    {
77:       return (byte) (b + key);
78:    }
79:
80:    private int key;
81: }
```

```
01: import java.io.IOException;
02: import java.util.Scanner;
03:
04: /**
05:    A program to test the Caesar cipher encryptor.
06: */
07: public class EncryptorTester
08: {
09:    public static void main(String[] args)
10:    {
11:       Scanner in = new Scanner(System.in);
12:       try
13:       {
14:          System.out.print("Input file: ");
15:          String inFile = in.next();
16:          System.out.print("Output file: ");
17:          String outFile = in.next();
18:          System.out.print("Encryption key: ");
19:          int key = in.nextInt();
20:          Encryptor crypt = new Encryptor(key);
21:          crypt.encryptFile(inFile, outFile);
22:       }
23:       catch (IOException exception)
24:       {
25:          System.out.println("Error processing file: " + exception);
26:       }
27:    }
28: }
29:
```

Object Streams

- Last example read BankAccount field individually
  - Easier way to deal with whole object
- **`ObjectOutputStream`** class can save a entire objects to disk
- **`ObjectOutputStream`** class can read objects back in from disk
- Objects are saved in binary format; hence, you use streams and not writers

Write out an object

- The object output stream saves all instance variables

```
BankAccount b = . . .;

ObjectOutputStream out = new ObjectOutputStream(
   new FileOutputStream("bank.dat"));

out.writeObject(b);
```

Read in an object

- **`readObject`** returns an **`Object`** reference
- Need to remember the types of the objects that you saved and use a cast

```
ObjectInputStream in = new ObjectInputStream(
   new FileInputStream("bank.dat"));
BankAccount b = (BankAccount) in.readObject();
```

Exceptions

- `readObject` method can throw a `ClassNotFoundException`
- It is a checked exception
- You must catch or declare it

Writing an Array

- Usually want to write out a collection of objects:

```
BankAccount[] arr = new BankAccount[size];

// Now add size BankAccount objects into arr
out.writeObject(arr);
```

Reading an Array

- To read a set of objects into an array

```
BankAccount[] ary = (BankAccount[])
   in.readObject();
```

Object Streams

- Very powerful features
  - Especially considering how little we have to do

- The **`BankAccount`** class as is actually will not work with the stream
  - Must implement **`Serializable`** interface in order for the formatting to work

Object Streams

```
class BankAccount implements Serializable
{
   . . .
}
```

- IMPORTANT: **`Serializable`** interface has no methods.
- No effort required

Serialization

- Serialization: process of saving objects to a stream
  - Each object is assigned a serial number on the stream
  - If the same object is saved twice, only serial number is written out the second time
  - When reading, duplicate serial numbers are restored as references to the same object

Serialization

- Why isn't everything serializable?
  - Security reasons – may not want contents of objects printed out to disk, then anyone can print out internal structure and analyze it
  - Example: Don't want SSN ever being accessed
  - Could also have temporary variables that are useless once the program is done running

Tokenizing

- Often several text values are in a single line in a file to be compact

  `"25 38 36 34 29 60 59"`

- The line must be broken into parts (i.e. ***tokens***)

  `"25"`

  `"38"`

  `"36"`

- tokens then can be parsed as needed

  `"25"` can be turned into the integer `25`

Tokenizing

- Inputting each value on a new line makes the file very long
- May want a file of customer info – name, age, phone number all on one line
- File usually separate each piece of info with a **delimiter** – any special character designating a new piece of data (space in previous example)

Tokenizing in Java

- use a **`StringTokenizer`** object
  - default delimiters are: space, tab, newline, return
  - requires: **`import java.util.*`**
- Constructors
  - **`StringTokenizer(String line)//default dlms`**
  - **`StringTokenizer(String ln, String dlms)`**
- Methods
  - **`hasMoreTokens()`**
  - **`nextToken()`**
  - **`countTokens()`**

StringTokenizing in Java

```
Scanner stdin = new…
System.out.print( "Enter a line with comma
  seperated integers(no space): " );
String input = stdin.nextLine();

StringTokenizer st;
String delims = ",";
st = new StringTokenizer( input, delims );

while ( st.hasMoreTokens() )
{
  int n = Integer.parseInt(st.nextToken());
  System.out.println(n);
}
```

```
File gradeFile = new File("scores.txt");
if(gradeFile.exists()){
  Scanner inFile = new Scanner(gradeFile);

  String line = inFile.nextLine();

  while(line != null){
      StringTokenizer st = new
            StringTokenizer(line, ":");
      System.out.print(" Name: " + st.nextToken());

      int num = 0;
      double sum = 0;

      while ( st.hasMoreTokens() )
      {
            num++;
            sum += Integer.parseInt(st.nextToken());
      }
      System.our.println(" average = "+ sum/num);
      line = inFile.nextLine();
```

```
    }

    inFile.close();
}
```

If you call `nextToken()` and there are no more tokens, `NoSuchElementException` is thrown

Tokenizing

- Scanner tokenizes already…

```
Scanner in = new Scanner(…);
while(in.hasNext()) {
  String str = in.next();
  …
}
```